JavaScript — Learn Everything

The complete course, from `var/let/const` to algorithms, i18n & production.

52 lessons · 5 projects · generated 2026-06-08

Contents

Lessons

01 · VARIABLES — var, let, const

```
/* ============================================================================
 * 01 · VARIABLES — var, let, const
 * ============================================================================
 * Run:  node lessons/01-variables.js
 *
 * WHAT YOU'LL LEARN
 *   • How to declare variables with var, let, and const
 *   • The difference between declaration, initialization, and reassignment
 *   • Scope: function-scope (var) vs block-scope (let/const)
 *   • Hoisting and the "temporal dead zone"
 *
 * RULE OF THUMB
 *   Use `const` by default. Use `let` only when you must reassign.
 *   Avoid `var` — it has confusing scoping and is considered legacy.
 *
 * NOTE: reserved keywords (var, let, const, if, else, for, while, function,
 *       return, ...) cannot be used as variable names.
 * ============================================================================ */

// ── 1. Declaration & initialization ──────────────────────────────────────────
// "Declaration" = create the variable. "Initialization" = give it a value.

let count;            // declaration only — value is `undefined` for now
count = 12;           // initialization
console.log(count);   // => 12

const TAX_RATE = 0.2; // const MUST be declared AND initialized together
console.log(TAX_RATE); // => 0.2

// const RATE;        // ❌ SyntaxError: Missing initializer in const declaration



// ── 2. Reassignment ──────────────────────────────────────────────────────────
let score = 10;
score = 25;           // ✅ let can be reassigned
console.log(score);   // => 25

// TAX_RATE = 0.3;    // ❌ TypeError: Assignment to constant variable
console.log(TAX_RATE); // => 0.2  (const can never be reassigned)

// IMPORTANT: `const` means the *binding* can't change — but if it holds an
// object or array, the contents can still be mutated:
```

```
const user = { name: 'Sam' };
user.name = 'Alex';   // ✅ allowed — we mutate the object, not the binding
console.log(user);    // => { name: 'Alex' }



// ── 3. Redeclaration ──────────────────────────────────────────────────────
// "Redeclaration" = declaring a variable with the SAME name twice in the SAME
// scope. This is different from reassignment (which only changes the value).

// `var` ALLOWS redeclaration — the second `var` is silently merged into the
// first. This is a common source of bugs, since you can clobber a variable
// without realising it:
var total = 1;
var total = 2;        // ✅ no error — `var` lets you redeclare
console.log(total);   // => 2

// `let` and `const` FORBID redeclaration in the same scope:
// let count = 1;
// let count = 2;     // ❌ SyntaxError: Identifier 'count' has already been declared
//
// const PI = 3.14;
// const PI = 3.15;   // ❌ SyntaxError: Identifier 'PI' has already been declared

// NOTE: redeclaring in a *different* (nested) block is fine — that's a brand new
// variable that "shadows" the outer one, not a redeclaration:
let level = 'outer';
{
  let level = 'inner'; // ✅ separate variable in its own block
  console.log(level);  // => inner
}
console.log(level);    // => outer



// ── 4. Scope ──────────────────────────────────────────────────────────────
// `var` is FUNCTION-scoped. `let`/`const` are BLOCK-scoped ({ ... }).

function demoScope() {
  var functionScoped = 'I live in the whole function';
  let blockScoped = 'I live only in my block';
  console.log(functionScoped, '|', blockScoped);
}
demoScope(); // => I live in the whole function | I live only in my block
// console.log(functionScoped); // ❌ ReferenceError — not visible outside

// var ignores block boundaries:
{
  var leaks = 'I escape the block';
}
console.log(leaks); // => I escape the block  (this is why var is confusing)
```

```
// let/const respect block boundaries:
{
  let trapped = 'I stay inside';
}
// console.log(trapped); // ❌ ReferenceError: trapped is not defined


// ── 5. Hoisting & the Temporal Dead Zone (TDZ) ──────────────────────
// `var` declarations are "hoisted" to the top and start as `undefined`.
console.log(hoistedVar); // => undefined  (declared below, but already exists)
var hoistedVar = 'now I have a value';

// `let`/`const` are also hoisted, but you CANNOT use them before their line.
// That gap is the "Temporal Dead Zone".
try {
  console.log(hoistedLet);   // ❌ throws — we're in the TDZ
} catch (err) {
  console.log(err.message);  // => Cannot access 'hoistedLet' before initialization
}
let hoistedLet = 'safe to use now';


/* PRACTICE ------------------------------------------------------------------
 *   1. Declare a const for your name and log it.
 *   2. Declare a let counter, set it to 0, then reassign it to 5.
 *   3. Try reassigning a const and read the exact error message.
 * --------------------------------------------------------------------------- */
```

02 · DATA TYPES

```
/* ============================================================================
 * 02 · DATA TYPES
 * ============================================================================
 * Run:  node lessons/02-data-types.js
 *
 * WHAT YOU'LL LEARN
 *   • The 7 primitive types and the object type
 *   • How `typeof` reports each type (and its one famous bug)
 *   • The difference between value types and reference types
 * ========================================================================== */

// ── 1. The primitive types ──────────────────────────────────────────
// Primitives are immutable, single values. There are 7:

const aString  = 'hello';         // string
const aNumber  = 42;              // number (integers AND decimals)
```

```
const aBoolean = true;            // boolean — true / false
const nothing  = null;            // null — an intentional "empty" value
let   notSet;                     // undefined — declared but no value yet
const big      = 9007199254740993n; // bigint — integers beyond Number's safe range
const unique   = Symbol('id');    // symbol — a guaranteed-unique value

console.log(typeof aString);  // => string
console.log(typeof aNumber);  // => number
console.log(typeof aBoolean); // => boolean
console.log(typeof notSet);   // => undefined
console.log(typeof big);      // => bigint
console.log(typeof unique);   // => symbol

// ⚠ FAMOUS BUG: typeof null is "object". It's a 30-year-old mistake, kept for
// backwards compatibility. Just memorize it.
console.log(typeof nothing);  // => object  (NOT "null"!)



// ── 2. Objects (everything that isn't a primitive) ──────────────────────
const obj   = { a: 1 };
const arr   = [1, 2, 3];
const fn    = function () {};

console.log(typeof obj); // => object
console.log(typeof arr); // => object   (arrays are objects! use Array.isArray)
console.log(typeof fn);  // => function (functions are callable objects)

console.log(Array.isArray(arr)); // => true   ← the correct way to detect arrays
console.log(Array.isArray(obj)); // => false



// ── 3. Value vs reference ──────────────────────────────────────────────
// Primitives are copied BY VALUE — each variable holds its own copy.
let x = 10;
let y = x;   // y gets a COPY of 10
y = 99;
console.log(x, y); // => 10 99   (changing y did not affect x)

// Objects/arrays are copied BY REFERENCE — both names point to the SAME data.
const original = { score: 1 };
const alias = original;   // alias points to the same object
alias.score = 100;
console.log(original.score); // => 100  (they share one object!)

// To truly copy an object, spread it (shallow copy):
const copy = { ...original };
copy.score = 5;
console.log(original.score, copy.score); // => 100 5
```

```
// ── 4. null vs undefined ──────────────────────────────────────────────
// undefined → "this has not been given a value yet" (usually by JS)
// null      → "this is intentionally empty" (usually set by you)
let unassigned;
console.log(unassigned);     // => undefined
const cleared = null;
console.log(cleared);        // => null



/* PRACTICE ----------------------------------------------------------------
 *   1. Log the typeof for: "5", 5, true, null, undefined, [], {}.
 *   2. Copy an object two ways: by reference and with spread. Mutate each copy
 *      and observe which one affects the original.
 * ------------------------------------------------------------------------ */
```

03 · OPERATORS

```
/* ============================================================================
 * 03 · OPERATORS
 * ============================================================================
 * Run:  node lessons/03-operators.js
 *
 * WHAT YOU'LL LEARN
 *   • Arithmetic, assignment, comparison, and logical operators
 *   • Strict (===) vs loose (==) equality — and why you should use ===
 *   • Modern operators: nullish (??) and optional chaining (?.)
 * ========================================================================== */

// ── 1. Arithmetic ─────────────────────────────────────────────────────
console.log(7 + 3);  // => 10
console.log(7 - 3);  // => 4
console.log(7 * 3);  // => 21
console.log(7 / 3);  // => 2.333...
console.log(7 % 3);  // => 1   (modulo / remainder — great for "is it even?")
console.log(2 ** 3); // => 8   (exponentiation: 2 to the power of 3)

// ── 2. Assignment shorthands ──────────────────────────────────────────
let n = 10;
n += 5;  console.log(n); // => 15   (same as n = n + 5)
n -= 3;  console.log(n); // => 12
n *= 2;  console.log(n); // => 24
n /= 4;  console.log(n); // => 6

// Increment / decrement
let i = 0;
console.log(i++); // => 0  (post: returns THEN increments → i is now 1)
```

```
console.log(++i); // => 2  (pre: increments THEN returns)

// ── 3. Comparison ──────────────────────────────────────────────────────
console.log(5 > 3);   // => true
console.log(5 <= 5);  // => true

// === is STRICT equality: compares value AND type, no conversion. Prefer it.
console.log(5 === 5);    // => true
console.log(5 === '5');  // => false  (number vs string)

// == is LOOSE equality: converts types first → surprising results. Avoid it.
console.log(5 == '5');   // => true   (string '5' gets converted to number 5)
console.log(0 == false); // => true   (😱 this is why we avoid ==)
console.log(0 === false);// => false

// ── 4. Logical operators ───────────────────────────────────────────────
console.log(true && false); // => false  (AND — both must be true)
console.log(true || false); // => true   (OR — at least one true)
console.log(!true);         // => false  (NOT — flips the boolean)

// && and || actually return one of the OPERANDS, not just true/false:
console.log('' || 'fallback');     // => fallback  (first truthy value)
console.log('hi' && 'world');      // => world     (last value if all truthy)

// ── 5. Nullish coalescing (??) ─────────────────────────────────────────
// ?? returns the right side ONLY when the left is null or undefined.
// Unlike ||, it treats 0 and '' as valid values.
const volume = 0;
console.log(volume || 5);  // => 5   (|| sees 0 as falsy — wrong here!)
console.log(volume ?? 5);  // => 0   (?? keeps 0 — correct)

// ── 6. Optional chaining (?.) ──────────────────────────────────────────
// Safely read deep properties without crashing on null/undefined.
const profile = { name: 'Sam', address: { city: 'Pune' } };
console.log(profile.address?.city);   // => Pune
console.log(profile.contact?.email);  // => undefined  (no crash!)
// console.log(profile.contact.email); // ❌ TypeError without the ?.



/* PRACTICE ------------------------------------------------------------
 *   1. Use % to check whether 17 is even or odd.
 *   2. Predict then verify: 0 == '', 0 === '', null == undefined.
 *   3. Give a setting a default with ?? where 0 should be a valid value.
 * ------------------------------------------------------------------- */
```

04 · TYPE CONVERSION & COERCION

```
/* ============================================================================
 * 04 · TYPE CONVERSION & COERCION
 * ============================================================================
 * Run:  node lessons/04-type-conversion.js
 *
 * WHAT YOU'LL LEARN
 *   • Explicit conversion (you do it) vs implicit coercion (JS does it)
 *   • Truthy and falsy values
 *   • The classic gotchas that confuse every beginner
 * ============================================================================ */

// ── 1. Explicit conversion (recommended) ──────────────────────────────────
// You clearly state the target type. Predictable and readable.

console.log(Number('42'));     // => 42
console.log(Number('42px'));   // => NaN   ("Not a Number" — conversion failed)
console.log(parseInt('42px')); // => 42    (parseInt grabs the leading number)
console.log(parseFloat('3.14rad')); // => 3.14

console.log(String(42));       // => "42"
console.log(String(true));     // => "true"
console.log((42).toString());  // => "42"

console.log(Boolean(1));       // => true
console.log(Boolean(0));       // => false



// ── 2. Truthy & Falsy ─────────────────────────────────────────────────────
// Every value is either "truthy" or "falsy" in a boolean context (if, &&, etc).
// There are exactly 8 FALSY values — memorize these; everything else is truthy.

const falsyValues = [false, 0, -0, 0n, '', null, undefined, NaN];
falsyValues.forEach(v => console.log(Boolean(v))); // => all false

// Common surprises — these are all TRUTHY:
console.log(Boolean('0'));     // => true  (non-empty string!)
console.log(Boolean('false')); // => true  (non-empty string!)
console.log(Boolean([]));      // => true  (empty array is truthy)
console.log(Boolean({}));      // => true  (empty object is truthy)



// ── 3. Implicit coercion (happens automatically) ──────────────────────────
// The + operator: if EITHER side is a string, it concatenates.
console.log(1 + '2');   // => "12"  (number 1 becomes string "1")
console.log('3' + 4);   // => "34"
console.log(1 + 2 + '3'); // => "33" (left-to-right: 1+2=3, then 3+'3'="33")
```

```
// Other math operators (- * / %) convert strings TO numbers:
console.log('6' - 2);   // => 4
console.log('6' * '2'); // => 12
console.log('a' - 2);   // => NaN


// ── 4. The == vs === reminder ──────────────────────────────────────────
// == coerces types before comparing → unpredictable. Always prefer ===.
console.log('' == 0);          // => true   😱
console.log('0' == 0);         // => true   😱
console.log(null == undefined);// => true   (the one == case that's handy)
console.log(NaN === NaN);      // => false  (NaN is never equal to anything!)
console.log(Number.isNaN(NaN));// => true   (the correct way to check for NaN)


/* PRACTICE ----------------------------------------------------------------
 *   1. Convert the string "100" to a number three different ways.
 *   2. Predict the output of: [] + [], [] + {}, true + 1.  Then run them.
 *   3. List all 8 falsy values from memory, then check yourself above.
 * ------------------------------------------------------------------------ */
```

05 · STRINGS

```
/* ============================================================================
 * 05 · STRINGS
 * ============================================================================
 * Run:  node lessons/05-strings.js
 *
 * WHAT YOU'LL LEARN
 *   • Creating strings and template literals
 *   • The most-used string methods
 *   • Strings are immutable — methods return NEW strings
 * ========================================================================== */

// ── 1. Creating strings ────────────────────────────────────────────────
const single = 'single quotes';
const double = "double quotes";        // identical to single quotes
const name = 'Sam';
const age = 25;

// Template literals (backticks): allow ${expressions} and multi-line text.
// PREFER these — they're cleaner than + concatenation.
const greeting = `Hi, I'm ${name} and I'm ${age} years old.`;
console.log(greeting); // => Hi, I'm Sam and I'm 25 years old.

const multiline = `Line one
```

```
Line two`;
console.log(multiline);



// ── 2. Length & accessing characters ──────────────────────────────────────
const text = 'JavaScript';
console.log(text.length);    // => 10
console.log(text[0]);        // => J     (zero-indexed)
console.log(text.at(-1));    // => t     (.at() supports negative indexes)



// ── 3. Searching ──────────────────────────────────────────────────────────
console.log(text.includes('Script')); // => true
console.log(text.startsWith('Java'));  // => true
console.log(text.endsWith('pt'));      // => true
console.log(text.indexOf('S'));        // => 4   (-1 if not found)



// ── 4. Transforming (returns a NEW string — originals never change) ──────
console.log(text.toUpperCase());       // => JAVASCRIPT
console.log(text.toLowerCase());       // => javascript
console.log('  trim me  '.trim());     // => "trim me"
console.log(text.replace('Java', 'Type')); // => TypeScript
console.log('a-b-c'.replaceAll('-', '/')); // => a/b/c
console.log('ab'.repeat(3));           // => abababab

// slice(start, end) — extract a portion (end not included)
console.log(text.slice(0, 4));   // => Java
console.log(text.slice(4));      // => Script
console.log(text.slice(-6));     // => Script  (negative = from the end)

// Immutability proof: text itself is unchanged
console.log(text); // => JavaScript



// ── 5. Splitting & joining ────────────────────────────────────────────────
const csv = 'red,green,blue';
const colors = csv.split(',');     // string → array
console.log(colors);               // => [ 'red', 'green', 'blue' ]
console.log(colors.join(' | '));   // array → string  => red | green | blue



// ── 6. Padding & numbers-as-strings ───────────────────────────────────────
console.log('5'.padStart(3, '0'));  // => 005   (handy for time/IDs)
console.log('5'.padEnd(3, '.'));    // => 5..



/* PRACTICE ------------------------------------------------------------------
 *   1. Build a sentence with a template literal using two variables.
```

```
 *   2. Take "  Hello World  ", trim it, lowercase it, and replace the space.
 *   3. Split "2024-06-04" by "-" and log the year, month, day separately.
 * ------------------------------------------------------------------------ */
```

06 · NUMBERS & MATH

```
/* ============================================================================
 * 06 · NUMBERS & MATH
 * ============================================================================
 * Run:  node lessons/06-numbers-math.js
 *
 * WHAT YOU'LL LEARN
 *   • How JS represents numbers (one type for ints and decimals)
 *   • Rounding and formatting
 *   • The Math object
 *   • The floating-point precision gotcha
 * ========================================================================== */

// ── 1. One number type ──────────────────────────────────────────────────────
console.log(10);     // integer
console.log(10.5);   // decimal — both are the same `number` type
console.log(1e6);    // => 1000000  (scientific notation)
console.log(0xff);   // => 255      (hexadecimal)



// ── 2. Formatting & rounding ────────────────────────────────────────────────
const price = 19.99876;
console.log(price.toFixed(2));   // => "19.99"  (string, 2 decimal places)
console.log(Math.round(2.5));    // => 3   (nearest integer)
console.log(Math.floor(2.9));    // => 2   (round DOWN)
console.log(Math.ceil(2.1));     // => 3   (round UP)
console.log(Math.trunc(-2.7));   // => -2  (drop the decimal, no rounding)

// Format with thousands separators:
console.log((1234567.89).toLocaleString('en-US')); // => 1,234,567.89

// Intl.NumberFormat — the full-control formatter (currency, percent, locales).
// Build it once and reuse it; it handles the currency symbol and rounding.
const usd = new Intl.NumberFormat('en-US', { style: 'currency', currency: 'USD' });
console.log(usd.format(1234.5));  // => $1,234.50

const inr = new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' });
console.log(inr.format(1234567)); // => ₹12,34,567.00  (note Indian digit grouping)

const pct = new Intl.NumberFormat('en-US', { style: 'percent', maximumFractionDigits: 1 });
console.log(pct.format(0.1487));  // => 14.9%
```

```
// ── 3. The Math object ──────────────────────────────────────────────
console.log(Math.max(3, 7, 2));  // => 7
console.log(Math.min(3, 7, 2));  // => 3
console.log(Math.abs(-9));       // => 9
console.log(Math.sqrt(81));      // => 9
console.log(Math.pow(2, 10));    // => 1024  (or use 2 ** 10)
console.log(Math.PI);            // => 3.14159...

// Random number between 0 (inclusive) and 1 (exclusive):
// (commented out so the lesson's output stays stable when you re-run it)
// console.log(Math.random());

// Random integer between min and max (inclusive) — a reusable pattern:
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
// console.log(randomInt(1, 6)); // a dice roll: 1–6


// ── 4. Parsing & validating ─────────────────────────────────────────
console.log(parseInt('42px', 10)); // => 42   (always pass the radix 10)
console.log(parseFloat('3.14m'));  // => 3.14
console.log(Number.isInteger(5));  // => true
console.log(Number.isInteger(5.1));// => false
console.log(Number.isNaN(0 / 0));  // => true  (0/0 is NaN)
console.log(Number.isFinite(1/0)); // => false (1/0 is Infinity)


// ── 5. ⚠ The floating-point gotcha ──────────────────────────────────
// Computers store decimals in binary, which can't represent some values exactly.
console.log(0.1 + 0.2);            // => 0.30000000000000004  (not 0.3!)
console.log(0.1 + 0.2 === 0.3);    // => false

// Fixes:
console.log((0.1 + 0.2).toFixed(2));              // => "0.30"
console.log(Math.abs(0.1 + 0.2 - 0.3) < 1e-9);    // => true (compare with tolerance)
// For money, work in the smallest unit (cents) as integers to avoid this entirely.


/* PRACTICE ------------------------------------------------------------
 *   1. Round 7.456 to 1 decimal place as a string.
 *   2. Find the largest of [12, 7, 25, 3] using Math.max with the spread (...).
 *   3. Write a function that returns a random integer between 1 and 100.
 * ------------------------------------------------------------------- */
```

07 · CONDITIONALS

```
/* ============================================================================
 * 07 · CONDITIONALS
 * ============================================================================
 * Run:  node lessons/07-conditionals.js
 *
 * WHAT YOU'LL LEARN
 *   • if / else if / else
 *   • The ternary operator (? :)
 *   • switch statements
 *   • Guard clauses for cleaner code
 * ============================================================================ */

// ── 1. if / else if / else ──────────────────────────────────────────────────
const score = 82;

if (score >= 90) {
  console.log('Grade: A');
} else if (score >= 80) {
  console.log('Grade: B'); // => Grade: B
} else if (score >= 70) {
  console.log('Grade: C');
} else {
  console.log('Grade: F');
}



// ── 2. Ternary operator — a one-line if/else that RETURNS a value ──────────
const age = 20;
const status = age >= 18 ? 'adult' : 'minor';
console.log(status); // => adult

// Great for assigning a value; avoid nesting them deeply (hard to read).
const fee = age < 12 ? 0 : age < 65 ? 100 : 50;
console.log(fee); // => 100



// ── 3. Logical operators as conditionals ────────────────────────────────────
// && runs the right side only if the left is truthy:
const isLoggedIn = true;
isLoggedIn && console.log('Welcome back!'); // => Welcome back!

// || provides a fallback value:
const username = '' || 'Guest';
console.log(username); // => Guest



// ── 4. switch — clean alternative for many discrete cases ──────────────────
```

```
const day = 'SAT';
let kind;

switch (day) {
  case 'SAT':
  case 'SUN':           // cases can share a body (no break between them)
    kind = 'weekend';
    break;              // ⚠ without break, execution "falls through" to the next case
  case 'MON':
    kind = 'start of week';
    break;
  default:              // runs if nothing matched
    kind = 'weekday';
}
console.log(kind); // => weekend



// ── 5. Guard clauses — return early to avoid deep nesting ──────────────────
// ❌ Nested and hard to read:
function checkoutNested(cart) {
  if (cart) {
    if (cart.items.length > 0) {
      return 'Proceeding to payment';
    } else {
      return 'Cart is empty';
    }
  } else {
    return 'No cart';
  }
}

// ✅ Flat and readable — handle the exits first:
function checkout(cart) {
  if (!cart) return 'No cart';
  if (cart.items.length === 0) return 'Cart is empty';
  return 'Proceeding to payment';
}
console.log(checkout({ items: ['book'] })); // => Proceeding to payment
console.log(checkout({ items: [] }));       // => Cart is empty
console.log(checkout(null));                // => No cart



/* PRACTICE ------------------------------------------------------------------
 *   1. Write an if/else that logs "even" or "odd" for a number (use % 2).
 *   2. Rewrite it as a ternary.
 *   3. Use a switch to map 1–7 to weekday names.
 * ---------------------------------------------------------------------- */
```

08 · LOOPS

```
/* ============================================================================
 * 08 · LOOPS
 * ============================================================================
 * Run:  node lessons/08-loops.js
 *
 * WHAT YOU'LL LEARN
 *   • for, while, do...while
 *   • for...of (values) vs for...in (keys)
 *   • break and continue
 *   • Which loop to reach for
 * ============================================================================ */

// ── 1. Classic for loop — when you need an index/counter ──────────────────
for (let i = 0; i < 3; i++) {
  console.log('for', i); // => 0, 1, 2
}
// Anatomy:  for (initialize; condition; afterEachLoop) { body }



// ── 2. while — loop until a condition becomes false ───────────────────────
let count = 3;
while (count > 0) {
  console.log('while', count); // => 3, 2, 1
  count--;
}

// ── 3. do...while — runs the body AT LEAST once, then checks ──────────────
let n = 0;
do {
  console.log('do-while runs once even though condition is false');
} while (n > 0);



// ── 4. for...of — iterate over VALUES (arrays, strings, Sets, Maps) ───────
// This is the modern go-to for looping over a collection.
const fruits = ['apple', 'banana', 'cherry'];
for (const fruit of fruits) {
  console.log(fruit); // => apple, banana, cherry
}

// Need the index too? Use .entries():
for (const [index, fruit] of fruits.entries()) {
  console.log(index, fruit); // => 0 apple, 1 banana, 2 cherry
}

// Strings are iterable too:
for (const char of 'hi') {
```

```
  console.log(char); // => h, i
}



// ── 5. for...in — iterate over KEYS of an object ──────────────────────
// Use this for objects. (Avoid it for arrays — for...of is better there.)
const user = { name: 'Sam', age: 25, city: 'Pune' };
for (const key in user) {
  console.log(`${key}: ${user[key]}`); // => name: Sam, age: 25, city: Pune
}



// ── 6. break & continue ───────────────────────────────────────────────
for (let i = 1; i <= 5; i++) {
  if (i === 4) break;       // stop the loop entirely
  if (i === 2) continue;    // skip the rest of THIS iteration only
  console.log('flow', i);   // => 1, 3
}



// ── 7. Array helper loops (preview of lesson 13) ──────────────────────
// forEach runs a function for each element — no manual index needed.
fruits.forEach((fruit, i) => console.log(i, fruit));



/* WHICH LOOP? ------------------------------------------------------------
 *   • Looping over array/string values .......... for...of
 *   • Looping over object keys .................. for...in
 *   • Need a numeric counter / step ............. classic for
 *   • Repeat until a condition .................. while
 *   • Transforming an array into a new one ...... .map() (lesson 13)
 * ------------------------------------------------------------------------ */

/* PRACTICE ---------------------------------------------------------------
 *   1. Print 1–10 with a for loop, but skip multiples of 3 (continue).
 *   2. Sum all numbers in [4, 8, 15, 16, 23, 42] with for...of.
 *   3. Loop over an object's keys and values with for...in.
 * ------------------------------------------------------------------------ */
```

09 · FUNCTIONS

```
/* ========================================================================
 * 09 · FUNCTIONS
 * ========================================================================
 * Run:  node lessons/09-functions.js
 *
 * WHAT YOU'LL LEARN
```

```
 *   • Function declarations vs expressions vs arrow functions
 *   • Parameters: default values, rest parameters
 *   • Returning values
 *   • Functions are "first-class" — they're values you can pass around
 *   • Recursion — a function that calls itself
 * ========================================================================== */

// ── 1. Function declaration ──────────────────────────────────────────────
// Hoisted — you can call it BEFORE it's defined in the file.
console.log(add(2, 3)); // => 5  (works even though add is defined below)

function add(a, b) {
  return a + b; // `return` sends a value back to the caller
}
// A function with no `return` returns undefined.



// ── 2. Function expression ───────────────────────────────────────────────
// Stored in a variable. NOT hoisted — must be defined before use.
const multiply = function (a, b) {
  return a * b;
};
console.log(multiply(4, 5)); // => 20



// ── 3. Arrow functions — shorter syntax ──────────────────────────────────
const square = (x) => x * x;          // single expression → implicit return
console.log(square(6));               // => 36

const subtract = (a, b) => a - b;     // multiple params need parentheses
console.log(subtract(10, 4));         // => 6

const greet = () => 'Hello!';         // no params → empty parentheses
console.log(greet());                 // => Hello!

// Need multiple lines? Use a block { } and an explicit return:
const describe = (name, age) => {
  const label = `${name} is ${age}`;
  return label;
};
console.log(describe('Sam', 25));     // => Sam is 25
// NOTE: arrows handle `this` differently — covered in lesson 11.



// ── 4. Default parameters ────────────────────────────────────────────────
function greetUser(name = 'Guest') {
  return `Welcome, ${name}!`;
}
console.log(greetUser('Alex')); // => Welcome, Alex!
```

```
console.log(greetUser());       // => Welcome, Guest!  (default kicks in)


// ── 5. Rest parameters — collect "the rest" into an array ──────────────
function sum(...numbers) {        // numbers is a real array: [1, 2, 3, 4]
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4));   // => 10
console.log(sum(5, 5));         // => 10


// ── 6. Functions are values (first-class) ──────────────────────────────
// You can store them, pass them as arguments, and return them.
function applyTwice(fn, value) {
  return fn(fn(value));         // call the passed-in function twice
}
console.log(applyTwice(square, 2)); // => 16  (square(square(2)) = square(4))

// A function returned by another function (a "factory"):
function makeMultiplier(factor) {
  return (x) => x * factor;
}
const triple = makeMultiplier(3);
console.log(triple(5)); // => 15


// ── 7. Recursion — a function that calls itself ────────────────────────
// Every recursion needs TWO things:
//   1. a BASE CASE that stops it (no base case → "Maximum call stack" crash)
//   2. a step that moves CLOSER to the base case on each call
function factorial(n) {
  if (n <= 1) return 1;          // base case — stop here
  return n * factorial(n - 1);   // recursive step — n gets smaller each time
}
console.log(factorial(5)); // => 120   (5 * 4 * 3 * 2 * 1)

// Anything you can do with a loop, you can do with recursion:
function sumTo(n) {
  if (n === 0) return 0;         // base case
  return n + sumTo(n - 1);
}
console.log(sumTo(4)); // => 10  (4 + 3 + 2 + 1 + 0)

// Recursion SHINES on nested/tree-shaped data, where loops get awkward.
// Here we add up every number in an arbitrarily deep nested array:
function deepSum(arr) {
  let total = 0;
  for (const item of arr) {
    // if it's an array, recurse into it; otherwise add the number
```

```
    total += Array.isArray(item) ? deepSum(item) : item;
  }
  return total;
}
console.log(deepSum([1, [2, [3, 4], 5], [6]])); // => 21

// Mental model: each call waits ("on the call stack") for the deeper call to
// finish, then combines the result — like opening boxes inside boxes, then
// re-stacking them on the way back out.



/* PRACTICE ------------------------------------------------------------------
 *   1. Write isEven(n) three ways: declaration, expression, arrow.
 *   2. Write a function with a default greeting parameter.
 *   3. Write average(...nums) that returns the mean of any count of numbers.
 *   4. Write a recursive countdown(n) that logs n, n-1, ... down to 0.
 *   5. Write a recursive power(base, exp) — no Math.pow, no ** operator.
 * ---------------------------------------------------------------------- */
```

10 · SCOPE & CLOSURES

```
/* ============================================================================
 * 10 · SCOPE & CLOSURES
 * ============================================================================
 * Run:  node lessons/10-scope-closures.js
 *
 * WHAT YOU'LL LEARN
 *   • Global, function, and block scope
 *   • Lexical scope (inner functions see outer variables)
 *   • Closures — the single most important "aha" concept in JS
 * ========================================================================== */

// ── 1. The three scopes ──────────────────────────────────────────────────
const globalVar = 'I am global';   // visible everywhere

function outer() {
  const functionVar = 'I am in outer()'; // visible inside outer() only
  console.log(globalVar);    // ✅ inner code can see outer scopes
  console.log(functionVar);  // ✅

  if (true) {
    const blockVar = 'I am in this block'; // visible in this { } only
    console.log(blockVar);   // ✅
  }
  // console.log(blockVar);   // ❌ ReferenceError — block-scoped
}
outer();
```

```
// console.log(functionVar);  // ❌ ReferenceError — not visible outside


// ── 2. Lexical scope ──────────────────────────────────────────────────
// "Lexical" = scope is determined by WHERE code is written, not where it runs.
// Inner functions always have access to the variables of their parents.
function grandparent() {
  const family = 'Smith';
  function parent() {
    function child() {
      return family; // reaches all the way up to grandparent's variable
    }
    return child();
  }
  return parent();
}
console.log(grandparent()); // => Smith


// ── 3. Closures ───────────────────────────────────────────────────────
// A closure = a function that "remembers" the variables from where it was
// created, EVEN AFTER that outer function has finished running.

function makeCounter() {
  let count = 0;             // this variable is "enclosed" by the inner function
  return function () {
    count++;                 // it keeps living between calls
    return count;
  };
}

const counter = makeCounter();
console.log(counter()); // => 1
console.log(counter()); // => 2
console.log(counter()); // => 3  (count persists — that's the closure!)

// Each closure gets its OWN private copy:
const counterA = makeCounter();
const counterB = makeCounter();
console.log(counterA()); // => 1
console.log(counterA()); // => 2
console.log(counterB()); // => 1  (independent from counterA)


// ── 4. Practical use: data privacy ────────────────────────────────────
// Closures let you create "private" variables that can't be touched directly.
function createBankAccount(initial) {
  let balance = initial;     // private — no outside code can read/write it
  return {
```

```
    deposit(amount) { balance += amount; return balance; },
    withdraw(amount) {
      if (amount > balance) return 'Insufficient funds';
      balance -= amount;
      return balance;
    },
    getBalance() { return balance; },
  };
}

const account = createBankAccount(100);
console.log(account.deposit(50));  // => 150
console.log(account.withdraw(30)); // => 120
console.log(account.getBalance()); // => 120
// account.balance is undefined — it's truly private.
console.log(account.balance);      // => undefined



/* PRACTICE ------------------------------------------------------------------
 *   1. Write makeAdder(x) that returns a function adding x to its argument.
 *      const add5 = makeAdder(5); add5(10) === 15
 *   2. Build a once(fn) wrapper that runs fn only the first time it's called.
 *   3. Explain in your own words why `count` survives between counter() calls.
 * ---------------------------------------------------------------------------- */
```

11 · THE `this` KEYWORD

```
/* ============================================================================
 * 11 · THE `this` KEYWORD
 * ============================================================================
 * Run:  node lessons/11-this-keyword.js
 *
 * WHAT YOU'LL LEARN
 *   • What `this` refers to (it depends on HOW a function is called)
 *   • Why arrow functions don't have their own `this`
 *   • call, apply, and bind
 *
 * BIG IDEA: `this` is NOT decided where a function is written — it's decided
 *           by HOW the function is called.
 * ============================================================================ */

// ── 1. `this` inside an object method = the object ──────────────────────────
const user = {
  name: 'Sam',
  greet() {
    return `Hi, I'm ${this.name}`; // `this` = the object the method is called on
  },
};
```

```
console.log(user.greet()); // => Hi, I'm Sam



// ── 2. The classic gotcha: a regular function inside a method ──────────────
const team = {
  name: 'Rockets',
  members: ['Ana', 'Bob'],
  listBroken() {
    // ❌ A normal function gets its OWN `this` (undefined in strict mode),
    //    so `this.name` is lost here.
    this.members.forEach(function (m) {
      // console.log(`${m} is on ${this.name}`); // would throw / be undefined
    });
  },
  listFixed() {
    // ✅ Arrow functions DON'T have their own `this` — they reuse the
    //    surrounding `this` (the team object). This is why arrows shine here.
    this.members.forEach((m) => {
      console.log(`${m} is on ${this.name}`);
    });
  },
};
team.listFixed(); // => Ana is on Rockets / Bob is on Rockets



// ── 3. Arrow functions inherit `this` ──────────────────────────────────────
const counter = {
  count: 0,
  // ❌ Don't use an arrow for the METHOD itself — it would grab the outer
  //    (module/global) `this`, not the object.
  increment() {
    const helper = () => ++this.count; // arrow keeps `this` = counter
    return helper();
  },
};
console.log(counter.increment()); // => 1
console.log(counter.increment()); // => 2



// ── 4. call / apply / bind — manually set `this` ───────────────────────────
function introduce(greeting) {
  return `${greeting}, I'm ${this.name}`;
}
const person = { name: 'Alex' };

// call: run now, pass `this` then arguments one by one
console.log(introduce.call(person, 'Hello'));  // => Hello, I'm Alex

// apply: like call, but arguments come as an array
```

```
console.log(introduce.apply(person, ['Hi']));  // => Hi, I'm Alex

// bind: DON'T run now — return a NEW function with `this` permanently fixed
const boundIntro = introduce.bind(person);
console.log(boundIntro('Hey')); // => Hey, I'm Alex



/* QUICK REFERENCE ---------------------------------------------------------
 *   obj.method()        → this = obj
 *   standalone()        → this = undefined (strict) or globalThis
 *   arrow function      → this = whatever it was in the enclosing scope
 *   fn.call/apply/bind  → this = the object you provide
 * ------------------------------------------------------------------------- */

/* PRACTICE ----------------------------------------------------------------
 *   1. Make an object `dog` with a name and a bark() method using `this`.
 *   2. Break it by extracting the method: const b = dog.bark; b(); — see why.
 *   3. Fix it with .bind(dog).
 * ------------------------------------------------------------------------- */
```

12 · ARRAYS — basics

```
/* ============================================================================
 * 12 · ARRAYS — basics
 * ============================================================================
 * Run:  node lessons/12-arrays.js
 *
 * WHAT YOU'LL LEARN
 *   • Creating and accessing arrays
 *   • Adding / removing elements (the mutating methods)
 *   • Mutating vs non-mutating operations
 * (Transform methods like map/filter/reduce get their own lesson — 13.)
 * ============================================================================ */

// ── 1. Creating & accessing ──────────────────────────────────────────────
const fruits = ['apple', 'banana', 'cherry'];
console.log(fruits[0]);        // => apple   (zero-indexed)
console.log(fruits.at(-1));    // => cherry  (.at supports negative indexes)
console.log(fruits.length);    // => 3

// Arrays can hold any mix of types (but usually keep them consistent):
const mixed = [1, 'two', true, null, { id: 1 }];
console.log(mixed.length);     // => 5



// ── 2. Add / remove at the ENDS (these MUTATE the array) ─────────────────
const stack = ['a', 'b'];
```

```
stack.push('c');         // add to end      → ['a','b','c']
console.log(stack.pop());// remove from end → returns 'c', array is ['a','b']
stack.unshift('z');      // add to start    → ['z','a','b']
console.log(stack.shift());// remove from start → returns 'z', array is ['a','b']
console.log(stack);      // => [ 'a', 'b' ]


// ── 3. splice — remove/insert ANYWHERE (mutates) ──────────────────────────
const nums = [1, 2, 3, 4, 5];
// splice(startIndex, deleteCount, ...itemsToInsert)
nums.splice(1, 2);          // remove 2 items starting at index 1
console.log(nums);          // => [ 1, 4, 5 ]
nums.splice(1, 0, 'x', 'y');// insert without deleting
console.log(nums);          // => [ 1, 'x', 'y', 4, 5 ]


// ── 4. slice — COPY a portion (does NOT mutate) ───────────────────────────
const letters = ['a', 'b', 'c', 'd'];
console.log(letters.slice(1, 3)); // => [ 'b', 'c' ]  (end not included)
console.log(letters.slice(-2));   // => [ 'c', 'd' ]
console.log(letters);             // => unchanged: [ 'a','b','c','d' ]


// ── 5. Searching ──────────────────────────────────────────────────────────
const colors = ['red', 'green', 'blue'];
console.log(colors.includes('green')); // => true
console.log(colors.indexOf('blue'));   // => 2   (-1 if not found)


// ── 6. Combining & converting ─────────────────────────────────────────────
console.log([1, 2].concat([3, 4]));    // => [1,2,3,4]  (non-mutating join)
console.log([1, 2, 3].join('-'));      // => "1-2-3"    (array → string)
console.log([3, 1, 2].sort());         // => [1,2,3]    (⚠ MUTATES, sorts as text)
console.log([1, 2, 3].reverse());      // => [3,2,1]    (⚠ MUTATES)

// ⚠ Default sort is alphabetical! For numbers, pass a comparator:
console.log([10, 2, 1].sort());            // => [1, 10, 2]  (wrong!)
console.log([10, 2, 1].sort((a, b) => a - b)); // => [1, 2, 10]  (correct)


// ── 7. Copying safely (avoid accidental shared references) ────────────────
const source = [1, 2, 3];
const copy1 = [...source];        // spread copy
const copy2 = source.slice();     // slice copy
copy1.push(4);
console.log(source, copy1);       // => [1,2,3] [1,2,3,4]  (source untouched)


/* MUTATING vs NON-MUTATING ------------------------------------------------
```

```
 *   MUTATES:     push pop shift unshift splice sort reverse fill
 *   RETURNS NEW: slice concat map filter [...spread] join
 *   When in doubt, prefer non-mutating — it prevents sneaky bugs.
 * ------------------------------------------------------------------------ */

/* PRACTICE ---------------------------------------------------------------
 *   1. Start with [], push 3 items, then remove the first one.
 *   2. Sort [42, 7, 100, 1] numerically ascending and descending.
 *   3. Copy an array, mutate the copy, and prove the original is unchanged.
 * ------------------------------------------------------------------------ */
```

13 · ARRAY METHODS — map, filter, reduce & friends

```
/* ========================================================================
 * 13 · ARRAY METHODS — map, filter, reduce & friends
 * ========================================================================
 * Run:  node lessons/13-array-methods.js
 *
 * WHAT YOU'LL LEARN
 *   • The "iteration" methods every JS developer uses daily
 *   • How they take a CALLBACK function
 *   • Chaining them together for clean data transformations
 *
 * These are NON-MUTATING — they return new arrays/values and leave the
 * original alone. This is the heart of modern, readable JavaScript.
 * ======================================================================== */

const numbers = [1, 2, 3, 4, 5, 6];

// ── 1. forEach — do something for each item (returns nothing) ──────────
numbers.forEach((n) => console.log('item:', n)); // side effects only



// ── 2. map — transform EACH item into a new array (same length) ────────
const doubled = numbers.map((n) => n * 2);
console.log(doubled); // => [ 2, 4, 6, 8, 10, 12 ]

const names = ['ana', 'bob'];
console.log(names.map((name) => name.toUpperCase())); // => [ 'ANA', 'BOB' ]



// ── 3. filter — keep only items that pass a test (length may shrink) ───
const evens = numbers.filter((n) => n % 2 === 0);
console.log(evens); // => [ 2, 4, 6 ]



// ── 4. reduce — boil an array down to a SINGLE value ──────────────────
```

```
// reduce((accumulator, current) => ..., startingValue)
const total = numbers.reduce((sum, n) => sum + n, 0);
console.log(total); // => 21

// reduce is powerful — e.g. find the max, or count occurrences:
const max = numbers.reduce((biggest, n) => (n > biggest ? n : biggest));
console.log(max); // => 6

const letters = ['a', 'b', 'a', 'c', 'b', 'a'];
const counts = letters.reduce((tally, letter) => {
  tally[letter] = (tally[letter] || 0) + 1;
  return tally;
}, {});
console.log(counts); // => { a: 3, b: 2, c: 1 }



// ── 5. find / findIndex — get the FIRST match ──────────────────────────
const users = [
  { id: 1, name: 'Ana' },
  { id: 2, name: 'Bob' },
];
console.log(users.find((u) => u.id === 2));      // => { id: 2, name: 'Bob' }
console.log(users.findIndex((u) => u.id === 2)); // => 1
// (find returns undefined and findIndex returns -1 when nothing matches)



// ── 6. some / every — boolean tests across the array ───────────────────
console.log(numbers.some((n) => n > 5));   // => true  (at least one matches)
console.log(numbers.every((n) => n > 0));  // => true  (all match)



// ── 7. sort (non-mutating with a copy) ─────────────────────────────────
const scores = [42, 7, 100, 1];
const sorted = [...scores].sort((a, b) => a - b); // copy first to avoid mutation
console.log(sorted);  // => [ 1, 7, 42, 100 ]
console.log(scores);  // => [ 42, 7, 100, 1 ]  (original preserved)



// ── 8. flat / flatMap ──────────────────────────────────────────────────
console.log([1, [2, [3]]].flat());      // => [ 1, 2, [3] ]   (one level)
console.log([1, [2, [3]]].flat(Infinity)); // => [ 1, 2, 3 ]  (all levels)



// ── 9. Chaining — the real power ───────────────────────────────────────
// "From these orders, get the total of completed orders over $50."
const orders = [
  { item: 'pen',    price: 5,   done: true },
  { item: 'laptop', price: 900, done: true },
  { item: 'mouse',  price: 60,  done: false },
```

```
  { item: 'monitor',price: 200, done: true },
];

const revenue = orders
  .filter((o) => o.done)            // keep completed
  .filter((o) => o.price > 50)      // keep > $50
  .map((o) => o.price)              // get just the prices
  .reduce((sum, p) => sum + p, 0);  // add them up
console.log(revenue); // => 1100



/* PRACTICE ------------------------------------------------------------------
 *   1. From [1..10], map to squares, then filter to keep only even squares.
 *   2. Use reduce to find the longest word in ['hi','hello','hey','howdy'].
 *   3. From a list of {name, age} people, get the names of everyone 18+.
 * ---------------------------------------------------------------------------- */
```

14 · OBJECTS

```
/* ============================================================================
 * 14 · OBJECTS
 * ============================================================================
 * Run:  node lessons/14-objects.js
 *
 * WHAT YOU'LL LEARN
 *   • Creating objects, reading/writing/deleting properties
 *   • Methods and shorthand syntax
 *   • Iterating with Object.keys / values / entries
 *   • Copying and merging objects
 *   • Getters/setters, property descriptors, and freezing
 *   • JSON — turning objects into text and back
 * ============================================================================ */

// ── 1. Creating & accessing ─────────────────────────────────────────────────
const user = {
  name: 'Sam',
  age: 25,
  'favorite color': 'blue', // keys with spaces need quotes
};

console.log(user.name);              // => Sam   (dot notation — preferred)
console.log(user['age']);            // => 25    (bracket notation)
console.log(user['favorite color']); // => blue  (brackets required for odd keys)

// Bracket notation is essential when the key is in a variable:
const key = 'name';
console.log(user[key]);              // => Sam
```

```
// ── 2. Adding, updating, deleting ──────────────────────────────────────
user.email = 'sam@example.com'; // add
user.age = 26;                  // update
delete user['favorite color'];  // delete
console.log(user); // => { name: 'Sam', age: 26, email: 'sam@example.com' }

// Check if a key exists:
console.log('email' in user);          // => true
console.log(user.phone === undefined); // => true (phone doesn't exist)


// ── 3. Methods & `this` ────────────────────────────────────────────────
const account = {
  owner: 'Alex',
  balance: 100,
  // method shorthand (no `function` keyword needed):
  describe() {
    return `${this.owner} has $${this.balance}`;
  },
};
console.log(account.describe()); // => Alex has $100


// ── 4. Shorthand property names ────────────────────────────────────────
const name = 'Dana';
const age = 30;
// Instead of { name: name, age: age }:
const person = { name, age };
console.log(person); // => { name: 'Dana', age: 30 }


// ── 5. Computed property names ─────────────────────────────────────────
const field = 'score';
const result = { [field]: 95 }; // the key comes from a variable
console.log(result); // => { score: 95 }


// ── 6. Iterating over an object ────────────────────────────────────────
const car = { make: 'Toyota', model: 'Corolla', year: 2020 };

console.log(Object.keys(car));   // => [ 'make', 'model', 'year' ]
console.log(Object.values(car)); // => [ 'Toyota', 'Corolla', 2020 ]
console.log(Object.entries(car));// => [ ['make','Toyota'], ['model','Corolla'], ['year',2020] ]

// entries() pairs nicely with for...of and destructuring:
for (const [k, v] of Object.entries(car)) {
  console.log(`${k} = ${v}`); // => make = Toyota, etc.
```

```
}


// ── 7. Copying & merging (shallow) ──────────────────────────────────────
const defaults = { theme: 'light', fontSize: 14 };
const overrides = { fontSize: 18 };

const settings = { ...defaults, ...overrides }; // later keys win
console.log(settings); // => { theme: 'light', fontSize: 18 }

// Object.assign does the same (mutates the first argument):
const merged = Object.assign({}, defaults, overrides);
console.log(merged); // => { theme: 'light', fontSize: 18 }

// ⚠ Spread is a SHALLOW copy — nested objects are still shared:
const original = { nested: { a: 1 } };
const shallow = { ...original };
shallow.nested.a = 99;
console.log(original.nested.a); // => 99 (the nested object is shared!)
// For a deep copy of plain data: structuredClone(original)
const deep = structuredClone(original);
deep.nested.a = 1;
console.log(original.nested.a, deep.nested.a); // => 99 1


// ── 8. Nested objects & optional chaining ───────────────────────────────
const company = { ceo: { name: 'Lee', contact: { city: 'Pune' } } };
console.log(company.ceo.contact.city);      // => Pune
console.log(company.cfo?.contact?.city);    // => undefined (no crash)


// ── 9. Getters & setters on plain objects ───────────────────────────────
// A getter/setter looks like a normal property from the outside, but runs a
// function. Great for computed/derived values. (Classes do this too — lesson 17.)
const temperature = {
  celsius: 25,
  get fahrenheit() {            // read like a property: temperature.fahrenheit
    return this.celsius * 1.8 + 32;
  },
  set fahrenheit(f) {           // write like a property: temperature.fahrenheit = 212
    this.celsius = (f - 32) / 1.8;
  },
};
console.log(temperature.fahrenheit); // => 77   (computed, no parentheses)
temperature.fahrenheit = 212;
console.log(temperature.celsius);    // => 100  (the setter ran)


// ── 10. Property descriptors & Object.defineProperty ────────────────────
```

```
// Every property secretly has flags: writable, enumerable, configurable.
// Normal assignment makes all three true. defineProperty lets you control them.
const config = {};
Object.defineProperty(config, 'VERSION', {
  value: '1.0',
  writable: false,    // can't be reassigned
  enumerable: false,  // hidden from Object.keys / for...in / JSON
  configurable: false,// can't be deleted or redefined
});
config.VERSION = '2.0';                // silently ignored (throws in strict mode)
console.log(config.VERSION);           // => 1.0
console.log(Object.keys(config));      // => []  (it's non-enumerable)
console.log(Object.getOwnPropertyDescriptor(config, 'VERSION'));
// => { value: '1.0', writable: false, enumerable: false, configurable: false }


// ── 11. Locking objects: freeze & seal ──────────────────────────────────
// freeze: no add, no remove, no change — fully read-only (shallow).
const settingsFrozen = Object.freeze({ theme: 'dark', fontSize: 14 });
settingsFrozen.fontSize = 99;          // ignored (throws in strict mode)
settingsFrozen.newKey = 1;             // ignored — can't add either
console.log(settingsFrozen, Object.isFrozen(settingsFrozen)); // => {...unchanged}, true

// seal: can CHANGE existing values, but can't ADD or REMOVE keys.
const sealed = Object.seal({ count: 0 });
sealed.count = 5;                      // ✅ allowed (existing key)
sealed.extra = 1;                      // ❌ ignored (can't add)
console.log(sealed, Object.isSealed(sealed)); // => { count: 5 }, true

// ⚠ freeze is SHALLOW — nested objects can still change.
// For deep immutability, freeze recursively or keep data flat.


// ── 12. JSON — objects as text ──────────────────────────────────────────
// JSON (JavaScript Object Notation) is how objects travel: saved to a file,
// stored in localStorage, or sent over the network. It is plain TEXT, not an
// object. Two functions convert back and forth:
const profile = { name: 'Sam', age: 25, tags: ['js', 'css'], active: true };

const text = JSON.stringify(profile); // object → JSON string
console.log(text); // => {"name":"Sam","age":25,"tags":["js","css"],"active":true}

const back = JSON.parse(text);        // JSON string → object (a NEW object)
console.log(back.name, back.tags[0]); // => Sam js

// Pretty-print: pass a "space" argument (great for logs and files):
console.log(JSON.stringify(profile, null, 2)); // 2-space indented, multi-line

// Pick/transform fields with a replacer (array = allowlist, or a function):
```

```
console.log(JSON.stringify(profile, ['name', 'age'])); // => {"name":"Sam","age":25}

// Revive values while parsing with a reviver (e.g. rebuild Dates):
const order = JSON.parse('{"id":1,"when":"2026-06-05T00:00:00.000Z"}', (k, v) =>
  k === 'when' ? new Date(v) : v
);
console.log(order.when instanceof Date); // => true

// ⚠ WHAT JSON DROPS — it only understands plain data:
//   • functions, undefined, and Symbols are OMITTED from objects
//   • Date becomes a STRING (use a reviver to turn it back, as above)
//   • Map/Set become {} ; NaN and Infinity become null
const messy = { fn() {}, skip: undefined, when: new Date(0), n: NaN };
console.log(JSON.stringify(messy)); // => {"when":"1970-01-01T00:00:00.000Z","n":null}

// ⚠ ALWAYS guard JSON.parse — bad text throws:
try {
  JSON.parse('{ not valid }');
} catch (err) {
  console.log('parse failed:', err.name); // => SyntaxError
}

// The "JSON deep-copy trick" — quick, but only for plain JSON-safe data
// (loses the same things listed above). Prefer structuredClone for general use.
const copy = JSON.parse(JSON.stringify(profile));
copy.tags.push('html');
console.log(profile.tags.length, copy.tags.length); // => 2 3  (independent)



/* PRACTICE ------------------------------------------------------------------
 *   1. Build a `book` object with title, author, and a summary() method.
 *   2. Loop over its entries and print "key: value" lines.
 *   3. Merge two settings objects so the second overrides the first.
 *   4. JSON.stringify the book, then JSON.parse it back — notice the
 *      summary() method is gone. Why? (JSON only stores plain data.)
 * ---------------------------------------------------------------------------- */
```

15 · DESTRUCTURING & SPREAD / REST

```
/* ============================================================================
 * 15 · DESTRUCTURING & SPREAD / REST
 * ============================================================================
 * Run:  node lessons/15-destructuring-spread.js
 *
 * WHAT YOU'LL LEARN
 *   • Pulling values out of arrays and objects in one line (destructuring)
 *   • Spreading (...) to expand arrays/objects
```

```
 *   • Rest (...) to collect leftovers
 * These show up EVERYWHERE in modern JS — learn them well.
 * ========================================================================== */

// ── 1. Array destructuring ──────────────────────────────────────────────────
const rgb = [255, 128, 0];
const [red, green, blue] = rgb; // unpack by position
console.log(red, green, blue);  // => 255 128 0

// Skip items with empty commas:
const [first, , third] = ['a', 'b', 'c'];
console.log(first, third); // => a c

// Default values for missing items:
const [x = 0, y = 0, z = 0] = [10, 20];
console.log(x, y, z); // => 10 20 0

// Swap variables without a temp variable:
let a = 1, b = 2;
[a, b] = [b, a];
console.log(a, b); // => 2 1



// ── 2. Object destructuring ─────────────────────────────────────────────────
const user = { name: 'Sam', age: 25, city: 'Pune' };
const { name, age } = user; // unpack by KEY name (order doesn't matter)
console.log(name, age);     // => Sam 25

// Rename while unpacking:
const { name: fullName } = user;
console.log(fullName);      // => Sam

// Defaults for missing keys:
const { country = 'India' } = user;
console.log(country);       // => India

// Nested destructuring:
const order = { id: 7, customer: { email: 'a@b.com' } };
const { customer: { email } } = order;
console.log(email);         // => a@b.com



// ── 3. Destructuring in function parameters (very common) ──────────────────
// Instead of accepting an object and reading props inside:
function greet({ name, greeting = 'Hello' }) {
  return `${greeting}, ${name}!`;
}
console.log(greet({ name: 'Ana' }));                    // => Hello, Ana!
console.log(greet({ name: 'Bob', greeting: 'Hey' }));   // => Hey, Bob!
```

```
// ── 4. Spread (...) — expand an iterable into individual elements ──────────
// Combine arrays:
const lows = [1, 2];
const highs = [3, 4];
console.log([...lows, ...highs]);   // => [ 1, 2, 3, 4 ]

// Copy an array (shallow):
const copy = [...lows];

// Pass array items as separate arguments:
console.log(Math.max(...[5, 9, 2])); // => 9

// Merge / clone objects (later keys win):
const base = { theme: 'light', size: 14 };
const custom = { ...base, size: 18 }; // => { theme: 'light', size: 18 }
console.log(custom);

// Spread a string into characters:
console.log([...'abc']); // => [ 'a', 'b', 'c' ]



// ── 5. Rest (...) — collect "the rest" into an array/object ────────────────
// In array destructuring:
const [winner, ...runnersUp] = ['gold', 'silver', 'bronze'];
console.log(winner);     // => gold
console.log(runnersUp);  // => [ 'silver', 'bronze' ]

// In object destructuring:
const { id, ...details } = { id: 1, name: 'Sam', age: 25 };
console.log(id);      // => 1
console.log(details); // => { name: 'Sam', age: 25 }

// In function parameters (gather all arguments):
function sum(...nums) {
  return nums.reduce((t, n) => t + n, 0);
}
console.log(sum(1, 2, 3, 4)); // => 10

// SPREAD vs REST: same `...` symbol, opposite jobs.
//   Spread → EXPANDS something out (used in calls/literals).
//   Rest   → COLLECTS many into one (used in parameters/patterns).



/* PRACTICE ------------------------------------------------------------------
 *   1. Destructure { title, year } from a movie object, defaulting year to 2024.
 *   2. Merge two arrays and dedupe them with [...new Set([...a, ...b])].
 *   3. Write a function printFirstAndRest(first, ...rest) and test it.
```

```
 * ------------------------------------------------------------------------ */
```

16 · SETS & MAPS

```
/* ============================================================================
 * 16 · SETS & MAPS
 * ============================================================================
 * Run:  node lessons/16-sets-maps.js
 *
 * WHAT YOU'LL LEARN
 *   • Set — a collection of UNIQUE values
 *   • Map — key/value pairs where keys can be ANY type
 *   • When to reach for these instead of arrays/objects
 * ============================================================================ */

// ── 1. Set — unique values, fast membership checks ──────────────────────────
const ids = new Set();
ids.add(1);
ids.add(2);
ids.add(2);   // ignored — already present
ids.add(3);
console.log(ids);        // => Set(3) { 1, 2, 3 }
console.log(ids.size);   // => 3
console.log(ids.has(2)); // => true   (fast lookup, unlike array.includes)
ids.delete(1);
console.log([...ids]);   // => [ 2, 3 ]  (spread to convert back to an array)

// ⭐ Most common use: remove duplicates from an array in one line.
const withDupes = [1, 2, 2, 3, 3, 3, 4];
const unique = [...new Set(withDupes)];
console.log(unique); // => [ 1, 2, 3, 4 ]

// Iterating a Set:
for (const id of ids) console.log('id:', id);



// ── 2. Map — key/value pairs with ANY key type ──────────────────────────────
// Unlike objects (whose keys are always strings), Map keys can be numbers,
// objects, functions — anything. And it remembers insertion order.
const scores = new Map();
scores.set('Ana', 90);
scores.set('Bob', 75);
console.log(scores.get('Ana')); // => 90
console.log(scores.has('Bob')); // => true
console.log(scores.size);       // => 2
scores.delete('Bob');
```

```
// Object keys (impossible with a plain object):
const userA = { id: 1 };
const lastSeen = new Map();
lastSeen.set(userA, '2026-06-04');
console.log(lastSeen.get(userA)); // => 2026-06-04

// Iterating a Map (entries are [key, value] pairs):
const colors = new Map([
  ['red', '#f00'],
  ['green', '#0f0'],
]);
for (const [name, hex] of colors) {
  console.log(`${name} → ${hex}`); // => red → #f00, green → #0f0
}
console.log([...colors.keys()]);   // => [ 'red', 'green' ]
console.log([...colors.values()]); // => [ '#f00', '#0f0' ]



// ── 3. Object vs Map — which to use? ──────────────────────────────────────
//   Use a plain OBJECT when:
//     • keys are fixed, known strings ("structured record")
//     • you want JSON serialization (JSON.stringify works on objects)
//   Use a MAP when:
//     • keys are dynamic, or not strings
//     • you frequently add/remove entries
//     • you need a reliable .size and guaranteed insertion order

// Convert between them:
const obj = Object.fromEntries(colors); // Map → object
console.log(obj); // => { red: '#f00', green: '#0f0' }
const backToMap = new Map(Object.entries(obj)); // object → Map
console.log(backToMap.size); // => 2



/* PRACTICE ------------------------------------------------------------------
 *   1. Count unique words in "the cat the dog the bird" using a Set.
 *   2. Build a Map of country → capital and print each pair.
 *   3. Use a Map to tally how many times each letter appears in "banana".
 * ---------------------------------------------------------------------------- */
```

17 · CLASSES & OBJECT-ORIENTED PROGRAMMING

```
/* ============================================================================
 * 17 · CLASSES & OBJECT-ORIENTED PROGRAMMING
 * ============================================================================
 * Run:  node lessons/17-classes-oop.js
 *
```

```
 * WHAT YOU'LL LEARN
 *   • Defining a class with a constructor and methods
 *   • Creating instances with `new`
 *   • Inheritance with `extends` and `super`
 *   • Getters/setters, static members, and private fields (#)
 * ========================================================================== */

// ── 1. A basic class ──────────────────────────────────────────────────────
class Animal {
  // The constructor runs when you do `new Animal(...)`.
  constructor(name, sound) {
    this.name = name;   // instance properties
    this.sound = sound;
  }

  // A method (shared by all instances):
  speak() {
    return `${this.name} says ${this.sound}`;
  }
}

const dog = new Animal('Rex', 'Woof');
console.log(dog.speak());       // => Rex says Woof
console.log(dog instanceof Animal); // => true



// ── 2. Inheritance with extends / super ──────────────────────────────────
class Dog extends Animal {
  constructor(name, breed) {
    super(name, 'Woof');   // call the parent constructor FIRST
    this.breed = breed;
  }

  // Add new behavior:
  fetch() {
    return `${this.name} fetches the ball`;
  }

  // Override a parent method (and optionally reuse it via super):
  speak() {
    return `${super.speak()} (a ${this.breed})`;
  }
}

const buddy = new Dog('Buddy', 'Labrador');
console.log(buddy.speak()); // => Buddy says Woof (a Labrador)
console.log(buddy.fetch()); // => Buddy fetches the ball
```

```
// ── 3. Getters & setters — computed/guarded properties ──────────────────
class Temperature {
  constructor(celsius) {
    this.celsius = celsius;
  }
  // Accessed like a property (no parentheses): temp.fahrenheit
  get fahrenheit() {
    return this.celsius * 1.8 + 32;
  }
  set fahrenheit(value) {
    this.celsius = (value - 32) / 1.8;
  }
}

const temp = new Temperature(25);
console.log(temp.fahrenheit); // => 77   (getter — note: no () )
temp.fahrenheit = 212;        // setter
console.log(temp.celsius);    // => 100



// ── 4. Static members — belong to the class, not instances ──────────────
class MathUtils {
  static PI = 3.14159;
  static double(n) {
    return n * 2;
  }
}
console.log(MathUtils.PI);        // => 3.14159   (called on the class itself)
console.log(MathUtils.double(8)); // => 16
// const m = new MathUtils(); m.double(8) // ❌ static methods aren't on instances



// ── 5. Private fields (#) — true encapsulation ──────────────────────────
class BankAccount {
  #balance = 0;            // private — only accessible inside this class

  constructor(initial) {
    this.#balance = initial;
  }
  deposit(amount) {
    this.#balance += amount;
    return this.#balance;
  }
  get balance() {
    return this.#balance;
  }
}

const acct = new BankAccount(100);
console.log(acct.deposit(50)); // => 150
```

```
console.log(acct.balance);     // => 150
// console.log(acct.#balance); // ❌ SyntaxError — private outside the class



/* OOP VOCAB ------------------------------------------------------------------
 *   class       → a blueprint for objects
 *   instance    → an object made from a class (via `new`)
 *   constructor → sets up a new instance
 *   inheritance → a class building on another (extends/super)
 *   encapsulation → hiding internal state (#private fields)
 * ---------------------------------------------------------------------------- */

/* PRACTICE -------------------------------------------------------------------
 *   1. Make a class Rectangle(width, height) with an area() method.
 *   2. Extend it as Square(side) using super.
 *   3. Add a static method Rectangle.fromObject({width, height}).
 * ---------------------------------------------------------------------------- */
```

18 · PROTOTYPES — how inheritance REALLY works

```
/* ============================================================================
 * 18 · PROTOTYPES — how inheritance REALLY works
 * ============================================================================
 * Run:  node lessons/18-prototypes.js
 *
 * WHAT YOU'LL LEARN
 *   • The prototype chain — JS's actual inheritance mechanism
 *   • That `class` (lesson 17) is "syntactic sugar" over prototypes
 *   • How property lookup walks up the chain
 *
 * This is more advanced/theoretical. You can write JS for a long time using
 * only `class`, but understanding prototypes explains WHY things work.
 * ============================================================================ */

// ── 1. Every object has a hidden link to a "prototype" ──────────────────────
// When you read obj.something, JS looks on obj first; if not found, it follows
// the link to obj's prototype, then ITS prototype, and so on — the "chain".

const animal = {
  eats: true,
  walk() {
    return 'walking...';
  },
};

// Create `rabbit` whose prototype is `animal`:
const rabbit = Object.create(animal);
```

```
rabbit.jumps = true;

console.log(rabbit.jumps); // => true  (own property)
console.log(rabbit.eats);  // => true  (inherited from animal via the chain)
console.log(rabbit.walk());// => walking...  (method found on the prototype)

// Where did each property come from?
console.log(rabbit.hasOwnProperty('jumps')); // => true  (its own)
console.log(rabbit.hasOwnProperty('eats'));  // => false (inherited)



// ── 2. The chain ends at null ──────────────────────────────────────────
// rabbit → animal → Object.prototype → null
console.log(Object.getPrototypeOf(rabbit) === animal);            // => true
console.log(Object.getPrototypeOf(animal) === Object.prototype);  // => true
console.log(Object.getPrototypeOf(Object.prototype));             // => null



// ── 3. Built-in methods live on prototypes ─────────────────────────────
// When you call [1,2,3].map(...), `map` isn't on your array — it's on
// Array.prototype, found by walking the chain.
const arr = [1, 2, 3];
console.log(Object.getPrototypeOf(arr) === Array.prototype); // => true
console.log(arr.map === Array.prototype.map);                // => true



// ── 4. Constructor functions (the pre-`class` style) ───────────────────
// Before ES6 classes, this is how "classes" were written. Methods go on
// the constructor's .prototype so all instances SHARE one copy (memory-efficient).
function Person(name) {
  this.name = name;            // instance-specific data
}
Person.prototype.greet = function () {  // shared method
  return `Hi, I'm ${this.name}`;
};

const sam = new Person('Sam');
console.log(sam.greet());                     // => Hi, I'm Sam
console.log(sam.greet === new Person('X').greet); // => true (same shared function)



// ── 5. `class` is sugar over all of this ───────────────────────────────
// These two are essentially equivalent:
class PersonClass {
  constructor(name) { this.name = name; }
  greet() { return `Hi, I'm ${this.name}`; }
}
const sam2 = new PersonClass('Sam');
console.log(typeof PersonClass);                      // => function (!)
```

```
console.log(PersonClass.prototype.greet === Object.getPrototypeOf(sam2).greet); // => true
// The class syntax just put `greet` on the prototype for you, like §4.



/* MENTAL MODEL ------------------------------------------------------------
 *   • Reading a property → JS walks UP the prototype chain until it finds it.
 *   • Methods are stored ONCE on the prototype and shared by all instances.
 *   • `class` is friendlier syntax; under the hood it's prototypes all the way.
 * ------------------------------------------------------------------------ */

/* PRACTICE ---------------------------------------------------------------
 *   1. Use Object.create to make an object that inherits a `describe` method.
 *   2. Check hasOwnProperty for an own vs inherited property.
 *   3. Log the prototype chain of [] until you reach null.
 * ------------------------------------------------------------------------ */
```

19 · CALLBACKS & PROMISES (asynchronous JavaScript)

```
/* ========================================================================
 * 19 · CALLBACKS & PROMISES (asynchronous JavaScript)
 * ========================================================================
 * Run:  node lessons/19-callbacks-promises.js
 *
 * WHAT YOU'LL LEARN
 *   • Why JS needs async (it's single-threaded)
 *   • Callbacks and "callback hell"
 *   • Promises: then / catch / finally
 *   • Promise.all / race / allSettled
 *
 * Watch the OUTPUT ORDER when you run this — it teaches the event loop.
 * ======================================================================== */

// ── 1. JS is single-threaded — it can't "wait" and block ──────────────────
// Slow things (timers, network, files) are handed off and finish LATER.
console.log('1: start');

setTimeout(() => console.log('3: timeout finished (later)'), 0);

console.log('2: end');
// Output order: 1, 2, THEN 3 — even though the timeout was 0ms. The synchronous
// code runs first; the callback waits in a queue until the stack is clear.



// ── 2. Callbacks — "call this function when you're done" ──────────────────
function fetchUser(id, callback) {
  setTimeout(() => {
    callback({ id, name: 'Sam' }); // simulate a 200ms network delay
```

```
  }, 200);
}
fetchUser(1, (user) => console.log('callback got:', user.name)); // => Sam (after 200ms)

// ⚠ CALLBACK HELL: nesting callbacks for sequential async steps gets ugly:
//   fetchUser(1, (user) => {
//     fetchPosts(user, (posts) => {
//       fetchComments(posts[0], (comments) => { ... }); // deeply nested 😖
//     });
//   });
// Promises were invented to flatten this.


// ── 3. Promises — an object representing a future value ──────────────────
// A Promise is in one of three states: pending → fulfilled (resolve) or rejected.
const promise = new Promise((resolve, reject) => {
  const success = true;
  setTimeout(() => {
    if (success) resolve('✅ data loaded');
    else reject(new Error('❌ failed'));
  }, 100);
});

promise
  .then((result) => console.log('then:', result))   // runs on resolve
  .catch((err) => console.log('catch:', err.message)) // runs on reject
  .finally(() => console.log('finally: always runs'));


// ── 4. Chaining — each .then returns a new promise (no more nesting!) ─────
function getUser(id) {
  return new Promise((resolve) =>
    setTimeout(() => resolve({ id, name: 'Ana' }), 50)
  );
}
function getPosts(user) {
  return new Promise((resolve) =>
    setTimeout(() => resolve([`${user.name}'s post`]), 50)
  );
}

getUser(1)
  .then((user) => getPosts(user))   // return a promise → the chain waits for it
  .then((posts) => console.log('chained:', posts)) // => [ "Ana's post" ]
  .catch((err) => console.log('error:', err));


// ── 5. Running promises together ──────────────────────────────────────────
const p1 = Promise.resolve('a');
```

```
const p2 = new Promise((r) => setTimeout(() => r('b'), 80));
const p3 = Promise.resolve('c');

// Promise.all — wait for ALL to finish (rejects if ANY one rejects):
Promise.all([p1, p2, p3]).then((values) => console.log('all:', values)); // => ['a','b','c']

// Promise.race — settle as soon as the FIRST one settles:
Promise.race([p2, Promise.resolve('fast')]).then((v) => console.log('race:', v)); // => fast

// Promise.allSettled — wait for all, never rejects; reports each outcome:
Promise.allSettled([Promise.resolve('ok'), Promise.reject('bad')]).then((results) =>
  console.log('allSettled:', results)
); // => [{status:'fulfilled',value:'ok'}, {status:'rejected',reason:'bad'}]



/* KEY TAKEAWAYS ------------------------------------------------------------
 *   • Synchronous code always runs before any async callback/promise.
 *   • .then chains replace nested callbacks.
 *   • Always end a chain with .catch to handle errors.
 *   • Next lesson (async/await) makes promises read like normal code.
 * ------------------------------------------------------------------------- */

/* PRACTICE -----------------------------------------------------------------
 *   1. Wrap setTimeout in a `delay(ms)` function that returns a Promise.
 *   2. Chain delay(100) → log "done after 100ms".
 *   3. Use Promise.all to wait for delay(50) and delay(150) together.
 * ------------------------------------------------------------------------- */
```

20 · ASYNC / AWAIT

```
/* ============================================================================
 * 20 · ASYNC / AWAIT
 * ============================================================================
 * Run:  node lessons/20-async-await.js
 *
 * WHAT YOU'LL LEARN
 *   • async/await — write asynchronous code that READS like synchronous code
 *   • Error handling with try/catch
 *   • Running tasks in sequence vs in parallel (a common performance trap)
 *
 * async/await is just nicer syntax over Promises (lesson 19). Under the hood,
 * an async function always returns a Promise.
 * ========================================================================== */

// A tiny helper used throughout: resolves after `ms` milliseconds.
function delay(ms, value) {
  return new Promise((resolve) => setTimeout(() => resolve(value), ms));
```

```
}


// ── 1. The basics: async marks a function, await pauses for a Promise ──────
async function loadData() {
  console.log('loading...');
  const result = await delay(100, '✅ data'); // pause here until it resolves
  console.log('got:', result);                // runs AFTER the await
  return result;                              // async functions return a Promise
}

// Compare to the promise version — same thing, fewer .then() callbacks:
//   loadData() === getData().then(result => ...)


// ── 2. Error handling with try/catch (clean!) ──────────────────────────────
async function riskyLoad() {
  try {
    const data = await Promise.reject(new Error('network down'));
    console.log(data); // skipped — the await threw
  } catch (err) {
    console.log('caught:', err.message); // => caught: network down
  } finally {
    console.log('cleanup runs either way');
  }
}


// ── 3. Sequential vs parallel — a critical distinction ─────────────────────
async function sequential() {
  // ❌ SLOW: each await waits for the previous one. Total ≈ 100+100+100 = 300ms.
  const a = await delay(100, 'a');
  const b = await delay(100, 'b');
  const c = await delay(100, 'c');
  return [a, b, c];
}

async function parallel() {
  // ✅ FAST: start all three FIRST, then await them together. Total ≈ 100ms.
  const pA = delay(100, 'a'); // started now
  const pB = delay(100, 'b'); // started now
  const pC = delay(100, 'c'); // started now
  return Promise.all([pA, pB, pC]); // wait for all in parallel
}
// Rule: if tasks DON'T depend on each other, run them in parallel.


// ── 4. await in a loop ─────────────────────────────────────────────────────
async function processInOrder(items) {
```

```
  const results = [];
  for (const item of items) {
    const r = await delay(30, item.toUpperCase()); // one at a time, in order
    results.push(r);
  }
  return results;
}
// For independent items, prefer: await Promise.all(items.map(processOne))


// ── 5. Run everything (top-level) ──────────────────────────────────────
// We use an async IIFE so we can `await` at the top level in any environment.
(async () => {
  await loadData();                            // => loading... / got: ✅ data
  await riskyLoad();                           // => caught: network down / cleanup...
  console.log('sequential:', await sequential());// => [ 'a', 'b', 'c' ]  (~300ms)
  console.log('parallel:', await parallel());    // => [ 'a', 'b', 'c' ]  (~100ms)
  console.log('inOrder:', await processInOrder(['x', 'y', 'z'])); // => ['X','Y','Z']
})();


/* MENTAL MODEL ------------------------------------------------------------
 *   • `await x` means "pause this function until promise x settles, then
 *      give me its value (or throw its error)".
 *   • The rest of your program keeps running while an async function awaits.
 *   • await ONLY works inside an async function.
 * ------------------------------------------------------------------------- */

/* PRACTICE ----------------------------------------------------------------
 *   1. Rewrite a .then() chain from lesson 19 using async/await.
 *   2. Fetch two things in parallel with Promise.all and time the difference.
 *   3. Wrap a failing await in try/catch and log a friendly error message.
 * ------------------------------------------------------------------------- */
```

21 · ERROR HANDLING

```
/* ============================================================================
 * 21 · ERROR HANDLING
 * ============================================================================
 * Run:  node lessons/21-error-handling.js
 *
 * WHAT YOU'LL LEARN
 *   • try / catch / finally
 *   • throw and the Error object
 *   • Custom error classes
 *   • Handling errors in async code
 *   • Global "last resort" handlers (browser & Node)
```

```
 * ========================================================================== */

// ── 1. try / catch / finally ──────────────────────────────────────────────
try {
  console.log('trying...');
  const data = JSON.parse('{ invalid json }'); // this throws
  console.log(data); // skipped
} catch (err) {
  console.log('caught:', err.message); // => caught: ... (parse error)
} finally {
  console.log('finally always runs (cleanup, closing files, etc.)');
}



// ── 2. Throwing your own errors ───────────────────────────────────────────
// `throw` stops the function and jumps to the nearest catch.
function divide(a, b) {
  if (b === 0) {
    throw new Error('Cannot divide by zero');
  }
  return a / b;
}

try {
  console.log(divide(10, 2)); // => 5
  console.log(divide(10, 0)); // throws
} catch (err) {
  console.log('error:', err.message); // => error: Cannot divide by zero
}
// Always throw an Error OBJECT (not a string) — it carries a message + stack.



// ── 3. The Error object & built-in error types ────────────────────────────
const err = new Error('something broke');
console.log(err.name);    // => Error
console.log(err.message); // => something broke
// err.stack contains the file/line trace (useful when debugging/logging).

// Built-in subtypes JS throws automatically:
//   TypeError      → wrong type (e.g. null.foo)
//   ReferenceError → using an undeclared variable
//   SyntaxError    → invalid code / bad JSON
//   RangeError     → value out of range (e.g. new Array(-1))
try {
  null.foo;
} catch (e) {
  console.log(e.name); // => TypeError
}
```

```
// ── 4. Custom error classes — meaningful, catchable error types ──────────────
class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = 'ValidationError';
    this.field = field; // attach extra context
  }
}

function createUser(name) {
  if (!name) {
    throw new ValidationError('Name is required', 'name');
  }
  return { name };
}

try {
  createUser('');
} catch (err) {
  if (err instanceof ValidationError) {
    console.log(`[${err.field}] ${err.message}`); // => [name] Name is required
  } else {
    throw err; // not ours — re-throw so it isn't silently swallowed
  }
}



// ── 5. Errors in async code ──────────────────────────────────────────────────
async function fetchThing() {
  // In async functions, use plain try/catch around awaits:
  try {
    await Promise.reject(new Error('timeout'));
  } catch (err) {
    console.log('async caught:', err.message); // => async caught: timeout
  }
}
fetchThing();

// For raw promises, use .catch():
Promise.reject(new Error('boom')).catch((e) => console.log('promise caught:', e.message));



// ── 6. Global "last resort" handlers ─────────────────────────────────────────
// try/catch handles errors you EXPECT. But something always slips through — an
// unexpected throw, a promise with no .catch(). A global handler is your safety
// net: log it (to a monitoring service — lesson 47) so you find out in prod.
//
// IN THE BROWSER:
```

```
//   window.addEventListener('error', (e) => {
//     report({ msg: e.message, file: e.filename, line: e.lineno }); // log it
//   });
//   // A promise that rejects with nobody listening fires THIS instead:
//   window.addEventListener('unhandledrejection', (e) => {
//     report({ msg: 'unhandled rejection', reason: e.reason });
//     e.preventDefault(); // stops the default console error (optional)
//   });
//
// IN NODE (this part runs):
process.on('unhandledRejection', (reason) => {
  console.log('global: unhandled rejection ->', reason?.message ?? reason);
});
process.on('uncaughtException', (err) => {
  console.log('global: uncaught exception ->', err.message);
  // A truly uncaught error left the app in an UNKNOWN state. Best practice:
  // log, then exit and let a process manager restart you cleanly:
  //   process.exit(1);
});
// Trigger the rejection handler on purpose (no .catch on this one):
Promise.reject(new Error('nobody caught me'));
// ⚠ Global handlers are a NET, not a strategy — handle errors locally where you
// can actually recover; let the global handler only LOG the ones that escape.



/* BEST PRACTICES ------------------------------------------------------------
 *   • Throw Error objects, not strings.
 *   • Only catch errors you can meaningfully handle; re-throw the rest.
 *   • Don't leave an empty catch {} — at minimum, log it.
 *   • Use custom error classes so callers can react to specific failures.
 * ------------------------------------------------------------------------- */

/* PRACTICE -----------------------------------------------------------------
 *   1. Write parseAge(str) that throws a RangeError if the age is negative.
 *   2. Create a NotFoundError class and throw it from a lookup function.
 *   3. Wrap a JSON.parse in try/catch and return a default object on failure.
 *   4. Add a process.on('unhandledRejection', ...) and trigger it with a
 *      Promise.reject that has no .catch — confirm your handler logs it.
 * ------------------------------------------------------------------------- */
```

22 · MODULES — import / export

```
/* ============================================================================
 * 22 · MODULES — import / export
 * ============================================================================
 * Run:  node lessons/22-modules.js   (the runnable demo at the bottom)
 *
 * WHAT YOU'LL LEARN
```

```
 *   • Why we split code into modules (files)
 *   • ES Modules (import/export) — the modern standard
 *   • CommonJS (require/module.exports) — Node's older system
 *   • Dynamic import() — load a module on demand (code splitting)
 *
 * Modules need MULTIPLE files to truly demonstrate, so the import/export
 * examples below are shown as comments — but there is a REAL, runnable pair
 * of files in  lessons/modules-demo/  that you can run right now:
 *
 *     node lessons/modules-demo/app.mjs    # ES Modules (import/export)
 *     node lessons/modules-demo/app.cjs    # CommonJS  (require/exports)
 *
 * Open those four files (math.mjs/app.mjs and math.cjs/app.cjs) alongside this
 * lesson. The self-contained demo at the bottom of THIS file just lets it run
 * on its own with `node lessons/22-modules.js`.
 * ========================================================================== */

/* ── 1. WHY modules? ─────────────────────────────────────────────────────────
 * One giant file is hard to navigate and reuse. Modules let each file own one
 * job and explicitly share (export) only what others need (import). Anything
 * not exported stays private to the file.
 */



/* ── 2. ES MODULES (ESM) — the modern standard ──────────────────────────────
 * Enable in Node by adding  "type": "module"  to package.json,
 * or by naming files with the  .mjs  extension. In the browser, use
 * <script type="module" src="app.js"></script>.
 *
 * ---- file: math.js ----
 *   // Named exports — export many values by name:
 *   export const PI = 3.14159;
 *   export function add(a, b) { return a + b; }
 *
 *   // Default export — ONE main value per file:
 *   export default function multiply(a, b) { return a * b; }
 *
 * ---- file: app.js ----
 *   import multiply, { PI, add } from './math.js';
 *   //     ^default     ^named (names must match, braces required)
 *
 *   import { add as sum } from './math.js';   // rename on import
 *   import * as math from './math.js';        // grab everything as a namespace
 *   console.log(math.PI, math.add(2, 3));
 *
 * Notes:
 *   • import lines are hoisted and run first, before other code.
 *   • Paths are relative ('./') and usually need the .js extension in Node ESM.
 */
```

```
/* ── 3. COMMONJS (CJS) — Node's original system ──────────────────────────
 * The default in Node when there's no "type": "module". You'll see this a lot
 * in existing Node code.
 *
 * ---- file: math.cjs ----
 *   const PI = 3.14159;
 *   function add(a, b) { return a + b; }
 *   module.exports = { PI, add };     // export an object
 *
 * ---- file: app.cjs ----
 *   const { PI, add } = require('./math.cjs');
 *   const math = require('./math.cjs'); // or grab the whole object
 *
 * ESM vs CJS quick map:
 *   export default x   ↔  module.exports = x
 *   export { a, b }    ↔  module.exports = { a, b }
 *   import x from '…'  ↔  const x = require('…')
 */



/* ── 3b. DYNAMIC import() — load a module ON DEMAND ──────────────────────
 * The `import ... from` form is STATIC: it runs at the top, always, before
 * anything else. `import()` is a FUNCTION that returns a Promise — so you can
 * load a module later, conditionally, or only when it's actually needed.
 *
 *   button.addEventListener('click', async () => {
 *     const { renderChart } = await import('./chart.js'); // fetched only on click
 *     renderChart(data);
 *   });
 *
 * WHY IT MATTERS — CODE SPLITTING (lesson 41):
 *   Bundlers (lesson 44) split each dynamic import into its own file, so the
 *   initial page ships LESS JavaScript and the heavy bits load on demand.
 *
 * Other uses:
 *   • Conditional/lazy loading:  if (isAdmin) await import('./admin-panel.js');
 *   • Load a CommonJS module from ESM:  const cjs = await import('./old.cjs');
 *   • import() works even in plain <script> (no type="module" required).
 *
 * It returns the module's NAMESPACE object (default export is `.default`):
 *   const mod = await import('./math.js');
 *   mod.add(2, 3);     mod.default(4, 5);   // named + default
 */
// Runnable taste of it: import a built-in Node module dynamically.
import('node:os').then((os) => {
  console.log('dynamic import → platform:', os.platform()); // e.g. 'linux'
});
```

```
/* ── 3c. TOP-LEVEL await — `await` at module scope (ES2022) ──────────────
 * Inside an ES module you can use `await` at the TOP LEVEL — no `async`
 * function wrapper needed. The module's own evaluation pauses until it resolves:
 *
 *   // config.mjs
 *   const res = await fetch('https://api.example.com/config');
 *   export const config = await res.json();   // ← awaited at top level
 *
 *   // app.mjs — importing waits until config.mjs has finished awaiting:
 *   import { config } from './config.mjs';     // `config` is already resolved
 *
 * WHY IT MATTERS:
 *   • Initialise things that need async work (config, DB connection, a WASM
 *     module) before the module's exports are usable — no half-ready exports.
 *   • Combine with dynamic import() to pick a module at load time:
 *       const { strings } = await import(`./i18n/${navigator.language}.mjs`);
 *
 * CAVEATS:
 *   • ESM ONLY. It does NOT work in CommonJS (.cjs) or in classic <script>;
 *     needs type="module" / a .mjs file / "type":"module" in package.json.
 *   • A slow top-level await DELAYS every module that imports this one, so keep
 *     it for genuine startup work — not per-request logic.
 */
// Runnable taste of it: this file is CommonJS, so we show it via a dynamic
// import of an ESM data: URL (the imported module uses top-level await itself).
import('data:text/javascript,export default await Promise.resolve(42)')
  .then((m) => console.log('top-level await → default export:', m.default)); // => 42


// ── 4. Runnable demo (no real imports needed) ──────────────────────────────
// This simulates a "module" as a self-contained object so the file still runs.
const mathModule = (() => {
  const PI = 3.14159;                  // "private" until exported
  const add = (a, b) => a + b;
  const multiply = (a, b) => a * b;
  return { PI, add, multiply };        // the file's "exports"
})();

console.log('PI:', mathModule.PI);            // => 3.14159
console.log('add:', mathModule.add(2, 3));    // => 5
console.log('multiply:', mathModule.multiply(4, 5)); // => 20


/* PRACTICE ----------------------------------------------------------------
 *   First, RUN the real demo and read its files:
 *      node lessons/modules-demo/app.mjs
 *      node lessons/modules-demo/app.cjs
 *   Then try these:
 *   1. Add a `subtract` named export to modules-demo/math.mjs and import it
 *      in app.mjs. Re-run: node lessons/modules-demo/app.mjs
```

```
 *   2. Change which function is the `default` export and update app.mjs.
 *   3. Add a `multiply` export to math.cjs and use it from app.cjs.
 *   4. In app.mjs, load math.mjs with a dynamic `await import('./math.mjs')`
 *      instead of a top-of-file import — log the returned namespace object.
 *   5. Add a top-level `await` to math.mjs (e.g. `export const ready =
 *      await Promise.resolve(true);`), import `ready` in app.mjs, and confirm
 *      it's already resolved when app.mjs runs.
 * ------------------------------------------------------------------------ */
```

23 · THE DOM — Document Object Model ⚠️ BROWSER ONLY

```
/* ========================================================================
 * 23 · THE DOM — Document Object Model   ⚠ BROWSER ONLY
 * ========================================================================
 * The DOM is how JS reads and changes the web page. It DOES NOT exist in Node
 * (there's no page), so running this with `node` just prints a reminder.
 *
 * HOW TO RUN THIS LESSON:
 *   1. Open index.html in the JS-learn folder.
 *   2. Change its script tag to:  <script src="lessons/23-dom.js"></script>
 *   3. Open the browser, then open DevTools (F12) → Console tab.
 *
 * WHAT YOU'LL LEARN
 *   • Selecting elements
 *   • Reading/changing text, HTML, attributes, classes, and styles
 *   • Creating and removing elements
 * ======================================================================== */

// Guard so this file is harmless when run in Node:
if (typeof document === 'undefined') {
  console.log('⚠  This is a BROWSER lesson. Open it via index.html — see the header.');
} else {
  // ── 1. Selecting elements ──────────────────────────────────────────
  // querySelector uses CSS selectors and returns the FIRST match (or null).
  const title = document.querySelector('h1');          // by tag
  const box = document.querySelector('#box');          // by id (#)
  const items = document.querySelectorAll('.item');    // ALL matches (a NodeList)

  // Older, still-valid alternatives:
  // document.getElementById('box');
  // document.getElementsByClassName('item');

  // ── 2. Reading & changing content ──────────────────────────────────
  if (title) {
    console.log(title.textContent);        // read the text
    title.textContent = 'Updated by JS!';  // change the text (safe — no HTML)
    title.innerHTML = 'Now <em>italic</em>'; // parses HTML (⚠ never with user input — XSS risk)
  }
```

```
  // ── 3. Attributes ──────────────────────────────────────────────────────
  const link = document.querySelector('a');
  if (link) {
    link.getAttribute('href');
    link.setAttribute('target', '_blank');
    link.href = 'https://example.com'; // many attributes are also properties
  }

  // ── 4. Classes & styles ────────────────────────────────────────────────
  if (box) {
    box.classList.add('active');
    box.classList.remove('hidden');
    box.classList.toggle('open');     // add if absent, remove if present
    box.classList.contains('active'); // => true/false

    box.style.color = 'white';
    box.style.backgroundColor = 'navy'; // note: camelCase (background-color)
  }

  // ── 5. Creating & inserting elements ───────────────────────────────────
  const list = document.querySelector('ul');
  if (list) {
    const li = document.createElement('li'); // create (not yet on the page)
    li.textContent = 'New item';
    li.classList.add('item');
    list.appendChild(li);            // add as the last child
    // list.prepend(li);             // add as the first child
    // li.remove();                  // remove an element
  }

  // ── 6. Looping over a NodeList ─────────────────────────────────────────
  items.forEach((item, i) => {
    item.textContent = `Item ${i + 1}`;
  });
}

/* PRACTICE (in the browser) ------------------------------------------------
 *   1. Add a <ul id="list"></ul> to index.html, then append 3 <li>s with JS.
 *   2. Select the <h1> and change its color and text.
 *   3. Toggle a "dark" class on the <body> from the console.
 * ------------------------------------------------------------------------ */
```

24 · EVENTS ⚠️ BROWSER ONLY

```
/* ============================================================================
 * 24 · EVENTS   ⚠ BROWSER ONLY
```

```
 * ============================================================================
 * Events are how your page REACTS to the user: clicks, typing, submitting forms,
 * scrolling, etc. Like the DOM lesson, this runs in a browser, not Node.
 *
 * HOW TO RUN: see lesson 23's header (open via index.html, check the console).
 *
 * WHAT YOU'LL LEARN
 *   • addEventListener
 *   • The event object (event.target, preventDefault)
 *   • Event bubbling and delegation
 *   • Common events (click, input, submit, keydown)
 * ============================================================================ */

if (typeof document === 'undefined') {
  console.log('⚠  This is a BROWSER lesson. Open it via index.html — see lesson 23.');
} else {
  // ── 1. addEventListener — the standard way to respond to events ─────────────
  const button = document.querySelector('#myButton');
  if (button) {
    button.addEventListener('click', (event) => {
      console.log('Button clicked!');
      console.log('clicked element:', event.target); // the element that fired it
    });
    // You can attach MANY listeners to the same element — they all run.
    // Remove one later with button.removeEventListener('click', namedHandler).
  }

  // ── 2. The event object ─────────────────────────────────────────────────────
  const input = document.querySelector('#nameInput');
  if (input) {
    // 'input' fires on every keystroke; event.target.value is the current text.
    input.addEventListener('input', (event) => {
      console.log('typing:', event.target.value);
    });
  }

  // ── 3. Forms — preventDefault stops the page reload ─────────────────────────
  const form = document.querySelector('#myForm');
  if (form) {
    form.addEventListener('submit', (event) => {
      event.preventDefault(); // stop the browser's default submit/reload
      const data = new FormData(form);
      console.log('form name field:', data.get('name'));
    });
  }

  // ── 4. Keyboard events ──────────────────────────────────────────────────────
  document.addEventListener('keydown', (event) => {
    console.log('key pressed:', event.key); // e.g. "Enter", "a", "Escape"
    if (event.key === 'Escape') console.log('Escape pressed!');
```

```
  });

  // ── 5. Bubbling & delegation ──────────────────────────────────────
  // When you click a child, the event "bubbles" UP through its ancestors.
  // DELEGATION uses this: attach ONE listener to a parent instead of many to
  // each child — great for lists that change over time.
  const list = document.querySelector('#todoList');
  if (list) {
    list.addEventListener('click', (event) => {
      // Did the click land on an <li>? (closest walks up to find a match)
      const li = event.target.closest('li');
      if (li) {
        li.classList.toggle('done');
        console.log('toggled:', li.textContent);
      }
    });
    // Now ANY <li> inside #todoList responds — even ones added later.
  }
}

/* COMMON EVENTS ------------------------------------------------------------
 *   click, dblclick     → mouse clicks
 *   input, change       → form field value changed
 *   submit              → form submitted (use preventDefault)
 *   keydown, keyup      → keyboard
 *   mouseover, mouseout → hover
 *   load, DOMContentLoaded → page/resources ready
 * ------------------------------------------------------------------------- */

/* PRACTICE (in the browser) ------------------------------------------------
 *   1. Add a button that increments a counter shown in a <span>.
 *   2. Add an input that live-updates an <h2> as you type.
 *   3. Use delegation: one listener on a <ul> that strikes through any <li>
 *      you click.
 * ------------------------------------------------------------------------- */
```

25 · FETCH & APIs — talking to the internet

```
/* ============================================================================
 * 25 · FETCH & APIs — talking to the internet
 * ============================================================================
 * Run:  node lessons/25-fetch-apis.js   (needs internet; Node 18+ has fetch)
 *
 * WHAT YOU'LL LEARN
 *   • HTTP fundamentals: methods, status codes, headers, REST
 *   • fetch() — making HTTP requests (GET and POST)
 *   • Working with JSON
 *   • Handling errors and HTTP status codes
```

```
 *   • This ties together Promises + async/await from lessons 19–20
 *
 * `fetch` is built into modern browsers AND Node 18+. It returns a Promise.
 * ========================================================================== */

/* ── 0. HTTP IN 60 SECONDS — what fetch is actually doing ──────────────────
 * Every request is: a METHOD + a URL + HEADERS (+ an optional BODY). The server
 * replies with a STATUS CODE + headers + a body. Know these and APIs stop being
 * mysterious.
 *
 * METHODS (the "verb" — what you want to do):
 *   GET     read data (no body)            POST    create something (has a body)
 *   PUT     replace a whole resource       PATCH   update part of a resource
 *   DELETE  remove a resource
 *
 * STATUS CODES (grouped by first digit):
 *   2xx success   → 200 OK, 201 Created, 204 No Content
 *   3xx redirect  → 301 Moved, 304 Not Modified (cached)
 *   4xx YOUR fault → 400 Bad Request, 401 Unauthorized, 403 Forbidden,
 *                    404 Not Found, 429 Too Many Requests
 *   5xx SERVER fault → 500 Internal Error, 502/503 upstream/unavailable
 *
 * COMMON HEADERS:
 *   Content-Type: application/json   → what the body IS
 *   Accept: application/json         → what you WANT back
 *   Authorization: Bearer <token>    → who you are (lesson 39)
 *
 * REST = a convention for designing URLs around "resources" (nouns) + methods:
 *   GET    /users        list users        POST   /users        create a user
 *   GET    /users/42     read user 42      PATCH  /users/42     update user 42
 *   DELETE /users/42     delete user 42
 * The same idea powers almost every API you'll call.
 */



// ── 1. JSON — the data format of the web ──────────────────────────────────
// APIs send/receive text in JSON format. Convert between JS and JSON like so:
const user = { name: 'Sam', age: 25, hobbies: ['code', 'chess'] };

const json = JSON.stringify(user);          // JS object → JSON string
console.log(json); // => {"name":"Sam","age":25,"hobbies":["code","chess"]}

const parsed = JSON.parse(json);            // JSON string → JS object
console.log(parsed.name); // => Sam

// Pretty-print with indentation (handy for logging/debugging):
console.log(JSON.stringify(user, null, 2));
```

```
// ── 1b. Building URLs with query strings (URLSearchParams) ──────────────────
// Many GET requests need a "?key=value&key=value" query string. Don't glue it
// by hand — URLSearchParams encodes spaces and special characters correctly.
const params = new URLSearchParams({ q: 'iced coffee', page: 2, sort: 'new' });
console.log(params.toString()); // => q=iced+coffee&page=2&sort=new

const searchUrl = `https://api.example.com/search?${params}`;
console.log(searchUrl); // => https://api.example.com/search?q=iced+coffee&page=2&sort=new
// You'd then:  await fetch(searchUrl)
// (URLSearchParams also PARSES query strings — see lesson 34.)



// ── 2. A GET request with async/await ───────────────────────────────────────
async function getPost(id) {
  // fetch returns a Response. .json() reads & parses the body (also a Promise).
  const response = await fetch(`https://jsonplaceholder.typicode.com/posts/${id}`);

  // ⚠ fetch does NOT throw on 404/500 — you must check response.ok yourself.
  if (!response.ok) {
    throw new Error(`HTTP ${response.status} ${response.statusText}`);
  }

  const data = await response.json();
  return data;
}



// ── 3. A POST request — sending data ────────────────────────────────────────
async function createPost(post) {
  const response = await fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(post), // body must be a string
  });
  return response.json();
}



// ── 4. Fetching several things in parallel ──────────────────────────────────
async function getManyPosts(ids) {
  // Start all requests at once, then await them together (fast — see lesson 20).
  const requests = ids.map((id) => getPost(id));
  return Promise.all(requests);
}



// ── 5. Run it (with proper error handling) ──────────────────────────────────
(async () => {
  try {
```

```
    const post = await getPost(1);
    console.log('GET post 1 title:', post.title);

    const created = await createPost({ title: 'Hello', body: 'World', userId: 1 });
    console.log('POST created id:', created.id); // => 101 (this fake API echoes back)

    const many = await getManyPosts([1, 2, 3]);
    console.log('parallel fetched titles:', many.map((p) => p.title.slice(0, 15)));
  } catch (err) {
    // Network failure (e.g. offline) OR a thrown HTTP error lands here.
    console.log('Request failed:', err.message);
    console.log('(If you are offline, that is expected — the code is correct.)');
  }
})();



/* THE PATTERN TO REMEMBER ------------------------------------------------
 *   const res = await fetch(url, options);
 *   if (!res.ok) throw new Error(res.status);   // fetch won't throw for you
 *   const data = await res.json();              // parse the body
 *   ... wrap the whole thing in try/catch.
 * ------------------------------------------------------------------------ */

/* PRACTICE ---------------------------------------------------------------
 *   1. Fetch https://jsonplaceholder.typicode.com/users and log every name.
 *   2. Fetch a single todo and log whether it's completed.
 *   3. Add a check that logs a friendly message when response.ok is false
 *      (try a URL ending in /posts/99999999).
 *   4. Log response.status and response.headers.get('content-type') for a GET.
 *   5. Send a DELETE to /posts/1 and log the status code you get back.
 * ------------------------------------------------------------------------ */
```

26 · REGULAR EXPRESSIONS (Regex)

```
/* ============================================================================
 * 26 · REGULAR EXPRESSIONS (Regex)
 * ============================================================================
 * Run:  node lessons/26-regular-expressions.js
 *
 * WHAT YOU'LL LEARN
 *   • Creating patterns and the common flags
 *   • Character classes, quantifiers, anchors, groups
 *   • Lookahead & lookbehind (match based on what's around)
 *   • The modern flags: u (unicode), s (dotAll), y (sticky), v (unicodeSets)
 *   • test / match / matchAll / replace / split with regex
 *
 * A regex is a tiny pattern language for finding/validating/replacing text.
```

```
 * Read each pattern's comment slowly — that's how regex "clicks".
 * ========================================================================== */

// ── 1. Creating a regex ──────────────────────────────────────────────────
const literal = /cat/;                 // literal notation (preferred)
const fromString = new RegExp('cat');  // when the pattern is dynamic/in a variable
console.log(literal.test('a cat'));    // => true   (.test → does it match? boolean)
console.log(fromString.test('dog'));   // => false



// ── 2. Flags (after the closing slash) ──────────────────────────────────
//   g → global (find ALL matches, not just the first)
//   i → case-insensitive
//   m → multiline (^ and $ match per line)
console.log('Cat cat CAT'.match(/cat/gi)); // => [ 'Cat', 'cat', 'CAT' ]



// ── 3. Character classes — what kind of character ──────────────────────
//   \d digit      \w word char (a-z A-Z 0-9 _)     \s whitespace
//   \D not-digit  \W not-word                       \S not-space
//   .  any char (except newline)
//   [abc] one of a/b/c     [a-z] a range     [^abc] NOT a/b/c
console.log('Order #42 ok'.match(/\d+/));   // => [ '42', ... ]  (\d+ = one or more digits)
console.log('a1 b2 c3'.match(/[a-c]\d/g));  // => [ 'a1', 'b2', 'c3' ]



// ── 4. Quantifiers — how many ──────────────────────────────────────────
//   *  zero or more     +  one or more     ?  zero or one (optional)
//   {3}  exactly 3      {2,4} between 2-4   {2,} 2 or more
console.log(/colou?r/.test('color'));  // => true  (the u is optional)
console.log(/colou?r/.test('colour')); // => true
console.log('aaa'.match(/a{2}/));      // => [ 'aa', ... ]



// ── 5. Anchors — position ──────────────────────────────────────────────
//   ^ start of string      $ end of string      \b word boundary
console.log(/^hello/.test('hello world')); // => true  (starts with hello)
console.log(/world$/.test('hello world')); // => true  (ends with world)
console.log(/\bcat\b/.test('the cat sat')); // => true (whole word "cat")
console.log(/\bcat\b/.test('category'));    // => false (cat is part of a word)



// ── 6. Groups & capturing ──────────────────────────────────────────────
// ( ) captures part of the match so you can reuse it.
const date = '2026-06-04';
const match = date.match(/(\d{4})-(\d{2})-(\d{2})/);
console.log(match[0]); // => 2026-06-04  (whole match)
console.log(match[1]); // => 2026        (first group)
console.log(match[2]); // => 06          (second group)
```

```
// Named groups are clearer:
const named = date.match(/(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/);
console.log(named.groups.year, named.groups.month); // => 2026 06

// Non-capturing group (?:...) — group for structure WITHOUT creating a $1 slot:
console.log('abcabc'.match(/(?:abc)+/)[0]); // => abcabc  (grouped, not captured)



// ── 6b. Lookahead & lookbehind — match by CONTEXT (not consumed) ──────────────
// These assert what comes BEFORE/AFTER without including it in the match.
//   (?=...)  positive lookahead   — followed by ...
//   (?!...)  negative lookahead   — NOT followed by ...
//   (?<=...) positive lookbehind  — preceded by ...
//   (?<!...) negative lookbehind  — NOT preceded by ...

// Get the number, but only when it's followed by "px" (the px isn't captured):
console.log('width: 42px'.match(/\d+(?=px)/)[0]); // => 42

// Get the amount AFTER a "$" (the $ isn't part of the match):
console.log('Total $59 now'.match(/(?<=\$)\d+/)[0]); // => 59

// Negative lookahead — a word NOT followed by "ing":
console.log('run runs running'.match(/run(?!ning)/g)); // => [ 'run', 'run' ]

// Classic use: add thousands separators by inserting commas at the right spots.
console.log('1234567'.replace(/\B(?=(\d{3})+(?!\d))/g, ',')); // => 1,234,567



// ── 6c. Modern flags: u (unicode), s (dotAll), y (sticky) ─────────────────────
//   u → full Unicode mode. Required for \p{...} property escapes & astral chars.
console.log(/\p{Emoji}/u.test('hi 👋'));     // => true  (match by Unicode property)
console.log([...'a😀b'].length);             // => 3  (spread is unicode-aware)
//   s → "dotAll": let . match newlines too (by default it doesn't).
console.log(/a.b/s.test('a\nb'));            // => true  (without /s → false)
//   y → "sticky": match ONLY at regex.lastIndex, then advance it. Great for
//       tokenizers/parsers that scan a string piece by piece.
const sticky = /\d+/y;
sticky.lastIndex = 5;
console.log(sticky.exec('abc  123')[0]);     // => 123  (matches starting at idx 5)
//   v → "unicodeSets" (ES2024): a smarter, stricter superset of u. It adds set
//       operations inside character classes and matches multi-codepoint graphemes.
//   • Intersection [A&&B] — match chars in BOTH sets:
console.log('café'.replace(/[\p{L}&&\p{ASCII}]/gu, '*')); // => "****"  (u: can't AND; && is
literal → letters OR ascii)
console.log('café'.replace(/[\p{L}&&\p{ASCII}]/gv, '*')); // => "***é"  (v: letters AND ascii → é
kept, it's non-ASCII)
//   • Subtraction [A--B] — match chars in A but NOT B:
console.log('café 123!'.replace(/[\p{L}--[aeiou]]/gv, '*')); // => "*a** 123!"  (letters except
```

```
vowels)
//   • Match a whole multi-codepoint grapheme via \q{} string literals:
console.log(/[\q{👍🏽}]/v.test('👍🏽'));       // => true  (one class entry = a string)
//   Rule of thumb: prefer /v over /u in new code when you use \p{...} or classes.



// ── 7. The string methods that take a regex ──────────────────────────────
// match — first match (or with /g, all matches as an array)
console.log('cat bat hat'.match(/[cbh]at/g)); // => [ 'cat', 'bat', 'hat' ]

// matchAll — all matches WITH their groups (returns an iterator)
for (const m of 'a1 b2'.matchAll(/(\w)(\d)/g)) {
  console.log(m[1], m[2]); // => a 1 / b 2
}

// replace — find & replace ($1 refers to a captured group)
console.log('2026-06-04'.replace(/(\d{4})-(\d{2})-(\d{2})/, '$3/$2/$1')); // => 04/06/2026
console.log('a b  c   d'.replace(/\s+/g, ' ')); // => "a b c d" (collapse spaces)

// split — split on a pattern
console.log('a, b ,c ,  d'.split(/\s*,\s*/)); // => [ 'a', 'b', 'c', 'd' ]



// ── 8. Practical validators ──────────────────────────────────────────────
const isEmail = (s) => /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(s);
console.log(isEmail('sam@example.com')); // => true
console.log(isEmail('not-an-email'));    // => false

// ⚠ Don't over-trust regex for emails/URLs in production — but it's great for
// quick format checks and pulling data out of text.



/* CHEAT SHEET ---------------------------------------------------------------
 *   \d \w \s   digit / word / space        ^ $ \b   start / end / boundary
 *   *  +  ?    many / 1+ / optional         {n} {n,m} exact / range counts
 *   [abc] [^abc] one-of / none-of          ( ) capture   (?<name> ) named
 *   (?:..) non-capturing group             (?=) (?!) lookahead pos/neg
 *   (?<=) (?<!) lookbehind pos/neg         \p{...} unicode property (needs u)
 *   flags: g (all)  i (ignore case)  m (multiline)  s (dot matches \n)
 *          u (unicode)  y (sticky — match at lastIndex only)  d (match indices)
 *          v (unicodeSets — superset of u: set ops [A&&B]/[A--B] & \q{} strings)
 *   Tip: build & test patterns at regex101.com — it explains every token.
 * ------------------------------------------------------------------------- */

/* PRACTICE ------------------------------------------------------------------
 *   1. Write a regex that matches a 10-digit phone number.
 *   2. Extract all hashtags from "love #js and #coding" using matchAll.
 *   3. Replace every vowel in "hello world" with "*".
 *   4. Use a lookbehind to grab the number after "id=" in "?id=42&x=1".
```

```
 *   5. Use a negative lookahead to match "cat" only when NOT followed by "alog".
 * ------------------------------------------------------------------------ */
```

27 · GENERATORS & ITERATORS

```
/* ========================================================================
 * 27 · GENERATORS & ITERATORS
 * ========================================================================
 * Run:  node lessons/27-generators-iterators.js
 *
 * WHAT YOU'LL LEARN
 *   • The iterator protocol (what makes something "for...of"-able)
 *   • Generator functions (function*) and yield
 *   • Lazy / infinite sequences
 *   • Making your own objects iterable with Symbol.iterator
 * ======================================================================== */

// ── 1. The iterator protocol ──────────────────────────────────────────
// An "iterator" is any object with a .next() that returns { value, done }.
// Arrays, strings, Sets, Maps are "iterable" because they can produce one.
const arr = [10, 20];
const it = arr[Symbol.iterator](); // get the array's iterator
console.log(it.next()); // => { value: 10, done: false }
console.log(it.next()); // => { value: 20, done: false }
console.log(it.next()); // => { value: undefined, done: true }



// ── 2. Generator functions — the easy way to make iterators ───────────
// function* + yield. Calling it does NOT run the body — it returns a generator.
// Each .next() runs UNTIL the next yield, then PAUSES there.
function* countToThree() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = countToThree();
console.log(gen.next()); // => { value: 1, done: false }
console.log(gen.next()); // => { value: 2, done: false }
console.log(gen.next()); // => { value: 3, done: false }
console.log(gen.next()); // => { value: undefined, done: true }

// Generators are iterable, so for...of and spread just work:
console.log([...countToThree()]);          // => [ 1, 2, 3 ]
for (const n of countToThree()) console.log('got', n); // => 1, 2, 3
```

```
// ── 3. Generators keep state between yields ──────────────────────────────
function* idMaker() {
  let id = 1;
  while (true) {          // an "infinite" generator — safe because it's LAZY
    yield id++;
  }
}
const ids = idMaker();
console.log(ids.next().value); // => 1
console.log(ids.next().value); // => 2
console.log(ids.next().value); // => 3
// It only computes the next value when you ask — never runs forever.


// ── 4. Lazy sequences — compute only what you need ──────────────────────
function* take(iterable, n) {
  let count = 0;
  for (const item of iterable) {
    if (count++ >= n) return;
    yield item;
  }
}
function* naturals() {
  let i = 1;
  while (true) yield i++;
}
console.log([...take(naturals(), 5)]); // => [ 1, 2, 3, 4, 5 ]  (from an infinite source!)


// ── 5. yield* — delegate to another generator/iterable ──────────────────
function* letters() { yield 'a'; yield 'b'; }
function* combined() {
  yield* letters();   // yield everything from letters()
  yield* [1, 2];      // yield* works on any iterable
}
console.log([...combined()]); // => [ 'a', 'b', 1, 2 ]


// ── 6. Make your OWN object iterable ────────────────────────────────────
// Add a [Symbol.iterator] method (a generator is the simplest way).
const range = {
  start: 1,
  end: 5,
  *[Symbol.iterator]() {
    for (let i = this.start; i <= this.end; i++) yield i;
  },
};
console.log([...range]);            // => [ 1, 2, 3, 4, 5 ]
for (const n of range) process.stdout.write(n + ' '); // => 1 2 3 4 5
console.log();
```

```
/* WHY USE GENERATORS? ------------------------------------------------------
 *   • Represent infinite or huge sequences without storing them all in memory.
 *   • Produce values lazily, on demand.
 *   • Write custom iteration for your own data structures cleanly.
 * ------------------------------------------------------------------------- */

/* PRACTICE -----------------------------------------------------------------
 *   1. Write a generator fibonacci() and take the first 10 numbers.
 *   2. Make an object { from, to } iterable so [...it] counts from→to.
 *   3. Write a generator that yields only the even numbers of an array.
 * ------------------------------------------------------------------------- */
```

28 · SYMBOLS, PROXY & REFLECT (metaprogramming)

```
/* ============================================================================
 * 28 · SYMBOLS, PROXY & REFLECT (metaprogramming)
 * ============================================================================
 * Run:  node lessons/28-symbols-proxy-reflect.js
 *
 * WHAT YOU'LL LEARN
 *   • Symbol — unique, hidden-ish keys
 *   • Proxy — intercept operations on an object (get/set/has/delete...)
 *   • Reflect — the standard helpers Proxy traps delegate to
 *
 * This is "metaprogramming": code that customizes how objects behave.
 * You won't use it daily, but libraries (Vue, MobX) are built on it.
 * ========================================================================== */

// ── 1. Symbols — guaranteed-unique values ──────────────────────────────────
const a = Symbol('id');   // the 'id' is just a description (for debugging)
const b = Symbol('id');
console.log(a === b);     // => false  (every Symbol is unique, even same desc)
console.log(a.toString());// => Symbol(id)

// Use as object keys that won't collide with normal string keys:
const ID = Symbol('id');
const user = { name: 'Sam', [ID]: 12345 };
console.log(user[ID]);            // => 12345
console.log(Object.keys(user));  // => [ 'name' ]  (symbol keys are skipped here)
console.log(user.name);          // normal keys still work as usual

// Well-known symbols let you hook into language behavior (lesson 27 used
// Symbol.iterator). Another example — customize toString tagging:
const widget = { [Symbol.toStringTag]: 'Widget' };
console.log(Object.prototype.toString.call(widget)); // => [object Widget]
```

```
// ── 2. Proxy — wrap an object and intercept operations ──────────────────────
// A Proxy sits in front of a "target" and runs your "traps" on access.
const target = { message: 'hello' };

const logged = new Proxy(target, {
  get(obj, prop) {
    console.log(`(read ${String(prop)})`);
    return obj[prop];
  },
  set(obj, prop, value) {
    console.log(`(write ${String(prop)} = ${value})`);
    obj[prop] = value;
    return true; // set traps MUST return true on success
  },
});

logged.message;          // => (read message)
logged.message = 'hi';   // => (write message = hi)
console.log(target.message); // => hi   (the real object was updated)


// ── 3. Proxy use-case: validation ──────────────────────────────────────────
function createValidatedUser() {
  return new Proxy({}, {
    set(obj, prop, value) {
      if (prop === 'age' && (typeof value !== 'number' || value < 0)) {
        throw new TypeError('age must be a non-negative number');
      }
      obj[prop] = value;
      return true;
    },
  });
}
const person = createValidatedUser();
person.age = 30;
console.log(person.age); // => 30
try {
  person.age = -5;       // blocked by the trap
} catch (e) {
  console.log('blocked:', e.message); // => blocked: age must be a non-negative number
}


// ── 4. Proxy use-case: default values for missing keys ──────────────────────
const withDefault = new Proxy({}, {
  get: (obj, prop) => (prop in obj ? obj[prop] : `<no ${String(prop)}>`),
});
```

```
withDefault.name = 'Ana';
console.log(withDefault.name); // => Ana
console.log(withDefault.email);// => <no email>  (instead of undefined)



// ── 5. Reflect — the "default behaviors" as functions ──────────────────────
// Reflect mirrors the proxy traps. Inside a trap, delegate to Reflect so you
// keep the standard behavior while adding your own logic on top.
const obj = { x: 1 };
console.log(Reflect.get(obj, 'x'));        // => 1   (same as obj.x)
Reflect.set(obj, 'y', 2);                  // same as obj.y = 2
console.log(Reflect.has(obj, 'y'));        // => true  (same as 'y' in obj)
console.log(Reflect.ownKeys(obj));         // => [ 'x', 'y' ]

// Idiomatic proxy using Reflect (forwards everything, logs reads):
const clean = new Proxy({ score: 9 }, {
  get(t, p, r) {
    console.log(`accessed ${String(p)}`);
    return Reflect.get(t, p, r); // default get, no manual t[p]
  },
});
console.log(clean.score); // => accessed score / 9



/* WHEN DO YOU USE THIS? ------------------------------------------------------
 *   • Symbol: unique keys, hooking into iteration/string-tagging.
 *   • Proxy + Reflect: reactive state (Vue 3), observable objects, validation,
 *     logging, virtualized/lazy objects, API mocking.
 *   • For everyday app code you rarely need these — but knowing they exist
 *     explains how "magic" libraries work.
 * ------------------------------------------------------------------------- */

/* PRACTICE -------------------------------------------------------------------
 *   1. Create two Symbols with the same description and prove they differ.
 *   2. Build a Proxy that makes an object read-only (set throws).
 *   3. Use Reflect.ownKeys to list every key (including symbols) of an object.
 * ------------------------------------------------------------------------- */
```

29 · ADVANCED ASYNC

```
/* ============================================================================
 * 29 · ADVANCED ASYNC
 * ============================================================================
 * Run:  node lessons/29-advanced-async.js
 *
 * WHAT YOU'LL LEARN
 *   • Microtasks vs macrotasks (the real event-loop order)
```

```
 *   • async iteration: for await...of and async generators
 *   • AbortController — cancelling async work
 *   • debounce & throttle — controlling how often a function runs
 *   • retry with backoff, and concurrency limiting (a "pool")
 *
 * Builds on lessons 19–20. These are the patterns senior devs reach for.
 * ========================================================================== */

const delay = (ms, v) => new Promise((r) => setTimeout(() => r(v), ms));


// ── 1. Microtasks vs macrotasks (event-loop ordering) ──────────────────────
// After the current synchronous code finishes, the engine drains ALL
// microtasks (Promise callbacks) BEFORE the next macrotask (setTimeout).
console.log('A: sync');
setTimeout(() => console.log('D: setTimeout (macrotask)'), 0);
Promise.resolve().then(() => console.log('C: promise (microtask)'));
console.log('B: sync');
// Order: A, B, C, D  → promises always jump ahead of setTimeout.


// ── 2. Async generators + for await...of ───────────────────────────────────
// A generator that yields PROMISES. for await...of awaits each one in turn.
async function* fetchPages() {
  for (let page = 1; page <= 3; page++) {
    const data = await delay(20, `page ${page} data`); // pretend network call
    yield data;
  }
}

async function readPages() {
  for await (const page of fetchPages()) {
    console.log('received:', page); // => received: page 1 data, ... 2, ... 3
  }
}


// ── 3. AbortController — cancel async work ─────────────────────────────────
// The standard way to cancel fetches, timers, or any awaitable.
function cancellableDelay(ms, signal) {
  return new Promise((resolve, reject) => {
    const id = setTimeout(resolve, ms);
    signal.addEventListener('abort', () => {
      clearTimeout(id);
      reject(new Error('aborted'));
    });
  });
}
```

```
async function abortDemo() {
  const controller = new AbortController();
  setTimeout(() => controller.abort(), 30); // cancel after 30ms
  try {
    await cancellableDelay(1000, controller.signal); // would take 1s
    console.log('completed'); // never reached
  } catch (err) {
    console.log('abort demo:', err.message); // => abort demo: aborted
  }
  // Real fetch supports this directly: fetch(url, { signal: controller.signal })
}



// ── 4. debounce — run only AFTER activity stops ──────────────────────────
// "Wait until the user stops typing for 300ms, THEN search." Resets on each call.
function debounce(fn, wait) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), wait);
  };
}



// ── 5. throttle — run at most ONCE per interval ──────────────────────────
// "No matter how often this fires (scroll/resize), run at most once per 200ms."
function throttle(fn, interval) {
  let last = 0;
  let lastArgs = null;
  return function (...args) {
    const now = Date.now();
    if (now - last >= interval) {
      last = now;
      fn.apply(this, args);
    } else {
      lastArgs = args; // remember the most recent call (optional trailing run)
    }
  };
}
// debounce → "settle then act" (search box). throttle → "steady rate" (scroll).



// ── 6. Retry with backoff — a real-world async pattern ──────────────────
async function withRetry(task, attempts = 3) {
  for (let i = 1; i <= attempts; i++) {
    try {
      return await task();
    } catch (err) {
      if (i === attempts) throw err;        // out of tries → give up
      await delay(10 * i);                   // wait longer each time (backoff)
```

```
      console.log(`retry ${i} after failure: ${err.message}`);
    }
  }
}


// ── 7. Concurrency limiting — a "pool" of N at a time ──────────────────────
// Promise.all (lesson 20) starts EVERYTHING at once. With 1,000 URLs that can
// overwhelm the server (or your rate limit). A pool runs at most `limit` tasks
// concurrently: as each finishes, the next starts. Keeps results in order.
async function mapLimit(items, limit, task) {
  const results = new Array(items.length);
  let next = 0;
  async function worker() {
    while (next < items.length) {
      const i = next++;                 // claim the next index
      results[i] = await task(items[i], i);
    }
  }
  // start `limit` workers; they pull from the shared queue until it's empty
  const pool = Array.from({ length: Math.min(limit, items.length) }, worker);
  await Promise.all(pool);
  return results;
}


// ── 8. Run the demos ──────────────────────────────────────────────────────
(async () => {
  await readPages();
  await abortDemo();

  let flaky = 0;
  const result = await withRetry(async () => {
    flaky++;
    if (flaky < 3) throw new Error(`fail #${flaky}`);
    return 'success on attempt 3';
  });
  console.log(result); // => success on attempt 3

  // concurrency demo: 6 tasks, only 2 run at a time → finishes in ~3 "waves".
  let active = 0, peak = 0;
  const out = await mapLimit([1, 2, 3, 4, 5, 6], 2, async (n) => {
    active++; peak = Math.max(peak, active);
    await delay(20);
    active--;
    return n * 10;
  });
  console.log('mapLimit result:', out);          // => [10,20,30,40,50,60] (in order)
  console.log('mapLimit peak concurrency:', peak); // => 2 (never more than the limit)
```

```
  // debounce demo: 5 rapid calls → fn runs once, with the LAST argument.
  const search = debounce((q) => console.log('searching for:', q), 50);
  ['a', 'ap', 'app', 'appl', 'apple'].forEach((q) => search(q));
  // => searching for: apple   (only once, after the burst settles)
})();



/* PRACTICE ------------------------------------------------------------------
 *   1. Predict the log order of a sync log, a setTimeout(0), and a Promise.then.
 *   2. Write an async generator that yields 1..n with a small delay each.
 *   3. Wrap a debounce around a function and call it rapidly — watch it fire once.
 *   4. Use mapLimit to "fetch" 10 fake URLs 3 at a time; log when each starts.
 * ---------------------------------------------------------------------------- */
```

30 · FUNCTIONAL PROGRAMMING

```
/* ============================================================================
 * 30 · FUNCTIONAL PROGRAMMING
 * ============================================================================
 * Run:  node lessons/30-functional-programming.js
 *
 * WHAT YOU'LL LEARN
 *   • Pure functions & immutability
 *   • Higher-order functions
 *   • Currying & partial application
 *   • Function composition (pipe / compose)
 *
 * FP is a STYLE: build programs from small, predictable functions that don't
 * change shared state. It makes code easier to test and reason about.
 * ============================================================================ */

// ── 1. Pure functions ──────────────────────────────────────────────────────
// A pure function: (1) same input → same output, (2) no side effects.
const addPure = (a, b) => a + b;        // ✅ pure
console.log(addPure(2, 3));             // => 5  (always)

let total = 0;
const addImpure = (n) => (total += n);  // ❌ impure — mutates outside state
addImpure(5);
console.log(total);                     // => 5  (depends on history; harder to test)



// ── 2. Immutability — don't mutate, return new data ────────────────────────
const nums = [1, 2, 3];

// ❌ mutating:
// nums.push(4);  nums.sort();
```

```
// ✅ immutable: produce NEW arrays/objects instead
const added = [...nums, 4];                 // => [1,2,3,4], nums untouched
const doubled = nums.map((n) => n * 2);     // => [2,4,6]
const sorted = [...nums].sort((a, b) => b - a); // copy before sort
console.log(nums, added, doubled, sorted);  // => [1,2,3] [1,2,3,4] [2,4,6] [3,2,1]

const user = { name: 'Sam', age: 25 };
const older = { ...user, age: 26 };         // new object, original unchanged
console.log(user.age, older.age);           // => 25 26



// ── 3. Higher-order functions (take or return functions) ──────────────────
// map/filter/reduce are HOFs (lesson 13). You can write your own:
const withLogging = (fn) => (...args) => {
  console.log(`calling with ${args}`);
  return fn(...args);
};
const loggedAdd = withLogging(addPure);
console.log(loggedAdd(4, 5)); // => calling with 4,5 / 9



// ── 4. Currying — turn f(a, b, c) into f(a)(b)(c) ─────────────────────────
// Lets you "fix" arguments one at a time and build specialized functions.
const multiply = (a) => (b) => a * b;
console.log(multiply(3)(4));   // => 12
const triple = multiply(3);    // pre-fill the first argument
console.log(triple(10));       // => 30

// A generic curry helper:
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) return fn(...args);
    return (...more) => curried(...args, ...more);
  };
}
const sum3 = curry((a, b, c) => a + b + c);
console.log(sum3(1)(2)(3));   // => 6
console.log(sum3(1, 2)(3));   // => 6
console.log(sum3(1)(2, 3));   // => 6



// ── 5. Partial application — pre-fill SOME arguments ──────────────────────
const greet = (greeting, name) => `${greeting}, ${name}!`;
const sayHello = greet.bind(null, 'Hello'); // fix the first arg
console.log(sayHello('Ana')); // => Hello, Ana!



// ── 6. Composition — pipe data through a series of functions ──────────────
```

```
// pipe: left → right (read top to bottom). compose: right → left (math style).
const pipe = (...fns) => (input) => fns.reduce((acc, fn) => fn(acc), input);
const compose = (...fns) => (input) => fns.reduceRight((acc, fn) => fn(acc), input);

const inc = (n) => n + 1;
const double = (n) => n * 2;
const square = (n) => n * n;

const transform = pipe(inc, double, square); // ((n+1)*2)^2
console.log(transform(3)); // => 64   (3→4→8→64)

const transform2 = compose(square, double, inc); // same result, reversed order
console.log(transform2(3)); // => 64



// ── 7. Putting it together (declarative data pipeline) ──────────────────────
const orders = [
  { item: 'pen', price: 5, qty: 3 },
  { item: 'pad', price: 8, qty: 2 },
  { item: 'ink', price: 12, qty: 1 },
];
const grandTotal = orders
  .map((o) => o.price * o.qty)   // line totals: [15, 16, 12]
  .filter((t) => t >= 12)        // keep big lines: [15, 16, 12]
  .reduce((sum, t) => sum + t, 0);
console.log(grandTotal); // => 43



/* WHY FP? ------------------------------------------------------------------
 *   • Pure + immutable = predictable, easy to test, fewer "spooky" bugs.
 *   • Small composable functions read like a recipe.
 *   • You don't have to go 100% FP — adopting purity & immutability where it
 *     helps already makes most codebases cleaner.
 * ------------------------------------------------------------------------ */

/* PRACTICE -----------------------------------------------------------------
 *   1. Rewrite a loop that mutates an array as a pure map/filter chain.
 *   2. Curry a function divide(a)(b) and make a halve = divide-by-2 helper.
 *   3. Use pipe to: trim → lowercase → replace spaces with "-" on a string.
 * ------------------------------------------------------------------------ */
```

31 · TRICKY CONCEPTS & INTERVIEW GOTCHAS

```
/* ============================================================================
 * 31 · TRICKY CONCEPTS & INTERVIEW GOTCHAS
 * ============================================================================
 * Run:  node lessons/31-tricky-concepts.js
```

```
 *
 * WHAT YOU'LL LEARN
 *   • The classic puzzles that trip up everyone (and show up in interviews)
 *   • WHY each one behaves the way it does
 *
 * For each: read the setup, GUESS the output, then check the // => comment.
 * Understanding the "why" means you've truly understood the language.
 * ========================================================================== */

// ── 1. var in a loop with setTimeout (the #1 classic) ──────────────────────
// var is function-scoped: all callbacks share ONE i, which is 3 by the time
// they run. let is block-scoped: each iteration gets its OWN i.
console.log('--- var vs let in loops ---');
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log('var i =', i), 0); // => 3, 3, 3
}
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log('let j =', j), 10); // => 0, 1, 2
}



// ── 2. Hoisting: function declarations vs expressions ──────────────────────
console.log('--- hoisting ---');
console.log(declared());        // => "I work!"  (declarations hoist fully)
function declared() { return 'I work!'; }
try {
  expressed();                  // ❌ TypeError: expressed is not a function
} catch (e) {
  console.log('expression:', e.constructor.name); // => TypeError
}
var expressed = function () { return 'nope'; }; // var = undefined when called above



// ── 3. Reference equality ──────────────────────────────────────────────────
console.log('--- reference vs value ---');
console.log({} === {});           // => false (two different objects)
console.log([] === []);           // => false
const x = {}; const y = x;
console.log(x === y);             // => true  (same reference)
console.log(0.1 + 0.2 === 0.3);   // => false (floating point — lesson 06)



// ── 4. == coercion landmines (why we use ===) ──────────────────────────────
console.log('--- loose equality ---');
console.log([] == ![]);   // => true  😱  (![] is false → [] == false → '' == 0 → true)
console.log(null == undefined); // => true
console.log(null == 0);         // => false (null only loosely equals undefined)
console.log(NaN === NaN);       // => false (use Number.isNaN instead)
```

```
// ── 5. this depends on the CALL SITE ──────────────────────────────────────
console.log('--- this binding ---');
const obj = {
  name: 'Sam',
  regular() { return this?.name; },
  arrow: () => (typeof globalThis.name === 'string' ? globalThis.name : '<no this>'),
};
console.log(obj.regular());        // => Sam  (called as obj.method → this = obj)
const detached = obj.regular;
console.log(detached());           // => undefined (called standalone → this lost)
console.log(obj.arrow());          // => <no this> (arrow has no own this)


// ── 6. Array holes & common surprises ─────────────────────────────────────
console.log('--- array quirks ---');
console.log(typeof NaN);                  // => number  (NaN is a number!)
console.log([1, 2, 3].map(String));       // => [ '1', '2', '3' ]
console.log(['1', '7', '11'].sort());     // => [ '1', '11', '7' ]  (string sort!)
console.log([1, 7, 11].sort((a, b) => a - b)); // => [ 1, 7, 11 ]  (numeric sort)
console.log(parseInt('08'));              // => 8 (modern; older engines: beware!)


// ── 7. Closures capturing a shared variable ───────────────────────────────
console.log('--- closure capture ---');
function makeFns() {
  const fns = [];
  for (let k = 0; k < 3; k++) fns.push(() => k); // let → each captures its own k
  return fns;
}
console.log(makeFns().map((f) => f())); // => [ 0, 1, 2 ]
// (With var k, this would be [3, 3, 3] — same trap as #1.)


// ── 8. Object key coercion ────────────────────────────────────────────────
console.log('--- object keys are strings ---');
const o = {};
o[1] = 'number one';
o['1'] = 'string one';      // SAME key — keys are coerced to strings
console.log(o[1]);          // => string one  (the second write won)
console.log(Object.keys(o));// => [ '1' ]     (only one key)


// ── 9. Truthy/falsy in conditions ─────────────────────────────────────────
console.log('--- truthy/falsy ---');
if ([]) console.log('empty array is truthy');   // prints (empty array → truthy)
if ('0') console.log('"0" string is truthy');    // prints (non-empty string)
if (!0) console.log('0 is falsy');               // prints
// The 8 falsy values: false, 0, -0, 0n, "", null, undefined, NaN. Else → truthy.
```

```
// ── 10. The mutation-by-reference bug ──────────────────────────────────
console.log('--- shared reference bug ---');
function addItem(item, list = []) { list.push(item); return list; }
console.log(addItem('a')); // => [ 'a' ]
console.log(addItem('b')); // => [ 'b' ]  (default param is fresh each call — safe)
// ⚠ But a SHARED outer array would accumulate:
const shared = [];
const buggy = (item) => { shared.push(item); return shared; };
buggy('x'); buggy('y');
console.log(shared); // => [ 'x', 'y' ]  (mutated across calls — often a bug)



/* INTERVIEW TIPS ------------------------------------------------------------
 *   • Always prefer let/const over var (kills the loop-closure trap).
 *   • Always use === ; reserve == only for `== null` (catches null+undefined).
 *   • Remember: this is set by HOW a function is called, not where it's defined.
 *   • Objects/arrays are passed by REFERENCE — copy before mutating.
 *   • Know the 8 falsy values cold.
 * ---------------------------------------------------------------------------- */

/* PRACTICE ------------------------------------------------------------------
 *   1. Fix a "3,3,3" var loop two ways: with let, and with an IIFE closure.
 *   2. Explain out loud why [] == ![] is true, token by token.
 *   3. Write a function that safely copies before mutating its array argument.
 * ---------------------------------------------------------------------------- */
```

32 · DATES & TIME

```
/* ============================================================================
 * 32 · DATES & TIME
 * ============================================================================
 * Run:  node lessons/32-dates-time.js
 *
 * WHAT YOU'LL LEARN
 *   • Creating Date objects
 *   • Reading & changing parts of a date
 *   • Timestamps and date math (durations)
 *   • Formatting dates for humans with Intl.DateTimeFormat
 *
 * NOTE: Date is quirky (months are 0-based!). Read the comments carefully.
 * ========================================================================== */

// ── 1. Creating dates ──────────────────────────────────────────────────
const now = new Date();                 // current date & time
const fromISO = new Date('2026-06-04'); // from an ISO string (recommended format)
```

```
const fromParts = new Date(2026, 5, 4); // ⚠ month is 0-based! 5 = JUNE
const fromStamp = new Date(0);          // from a timestamp (ms since 1970-01-01 UTC)

console.log(fromISO.toISOString());     // => 2026-06-04T00:00:00.000Z
console.log(fromParts.getFullYear(), fromParts.getMonth()); // => 2026 5 (June)
console.log(fromStamp.toISOString());   // => 1970-01-01T00:00:00.000Z



// ── 2. Reading parts of a date ──────────────────────────────────────
const d = new Date('2026-06-04T09:30:00');
console.log(d.getFullYear());  // => 2026
console.log(d.getMonth());     // => 5    (0=Jan … 5=Jun — add 1 for humans)
console.log(d.getDate());      // => 4    (day of month, 1-based)
console.log(d.getDay());       // => 4    (day of week, 0=Sun … 4=Thu)
console.log(d.getHours(), d.getMinutes()); // => 9 30
// UTC variants exist too: getUTCFullYear(), getUTCHours(), ...



// ── 3. Timestamps & "now" ───────────────────────────────────────────
const ms = Date.now();          // milliseconds since 1970 (an integer)
console.log(typeof ms);         // => number
console.log(d.getTime());       // => the timestamp of date d
// Timestamps make dates easy to compare and subtract.



// ── 4. Date math — durations ────────────────────────────────────────
const start = new Date('2026-06-01');
const end = new Date('2026-06-04');
const diffMs = end - start;                 // subtracting dates → milliseconds
const diffDays = diffMs / (1000 * 60 * 60 * 24);
console.log(diffDays);          // => 3

// Add time by working in ms (or with setDate):
const tomorrow = new Date(start);
tomorrow.setDate(start.getDate() + 1);  // setDate handles month rollovers
console.log(tomorrow.toISOString().slice(0, 10)); // => 2026-06-02

// A reusable "days between" helper:
const daysBetween = (a, b) => Math.round((b - a) / 86_400_000);
console.log(daysBetween(start, end)); // => 3



// ── 5. Formatting for humans — Intl.DateTimeFormat (the modern way) ──
const date = new Date('2026-06-04T15:05:00');

console.log(new Intl.DateTimeFormat('en-US').format(date));      // => 6/4/2026
console.log(new Intl.DateTimeFormat('en-GB').format(date));      // => 04/06/2026

const long = new Intl.DateTimeFormat('en-US', {
```

```
  weekday: 'long', year: 'numeric', month: 'long', day: 'numeric',
});
console.log(long.format(date)); // => Thursday, June 4, 2026

const withTime = new Intl.DateTimeFormat('en-US', {
  hour: '2-digit', minute: '2-digit',
});
console.log(withTime.format(date)); // => 03:05 PM

// Quick built-ins (less control, but easy):
console.log(date.toDateString());     // => Thu Jun 04 2026
console.log(date.toLocaleDateString());// => locale-specific short date



// ── 6. "Time ago" — a common real-world function ──────────────────────
function timeAgo(date, from) {
  const seconds = Math.floor((from - date) / 1000);
  const units = [
    ['year', 31536000], ['month', 2592000], ['day', 86400],
    ['hour', 3600], ['minute', 60], ['second', 1],
  ];
  for (const [name, secs] of units) {
    const value = Math.floor(seconds / secs);
    if (value >= 1) return `${value} ${name}${value > 1 ? 's' : ''} ago`;
  }
  return 'just now';
}
const past = new Date('2026-06-01T12:00:00');
const reference = new Date('2026-06-04T12:00:00');
console.log(timeAgo(past, reference)); // => 3 days ago



/* GOTCHAS ------------------------------------------------------------------
 *   • Months are 0-based (January = 0). Days of month are 1-based. Confusing!
 *   • Always prefer ISO strings ('2026-06-04') — other formats are ambiguous.
 *   • Dates carry a time zone; '2026-06-04' is parsed as UTC midnight.
 *   • For heavy date work, libraries like date-fns or Day.js save headaches.
 * ------------------------------------------------------------------------- */

/* PRACTICE -----------------------------------------------------------------
 *   1. Log today's date as "Weekday, Month Day, Year" using Intl.
 *   2. Compute how many days until a target date.
 *   3. Build a function addDays(date, n) that returns a new date n days later.
 * ------------------------------------------------------------------------- */
```

33 · BROWSER STORAGE ⚠️ MOSTLY BROWSER

```
/* ============================================================================
 * 33 · BROWSER STORAGE   ⚠ MOSTLY BROWSER
 * ============================================================================
 * Run:  node lessons/33-browser-storage.js  (runs a Node-safe simulation)
 *       OR load via index.html to use REAL localStorage in the browser.
 *
 * WHAT YOU'LL LEARN
 *   • localStorage vs sessionStorage vs cookies — when to use each
 *   • Storing & reading data (it's always strings → use JSON)
 *   • A typical "save/load app state" pattern
 *
 * localStorage doesn't exist in Node, so this file runs a tiny in-memory
 * stand-in when needed, and shows the REAL browser API in comments.
 * ============================================================================ */

// ── 0. Get a storage object (real in browser, simulated in Node) ──────────
const storage =
  typeof localStorage !== 'undefined'
    ? localStorage
    : (() => {
        // Minimal in-memory mock so the lesson runs in Node:
        const map = new Map();
        return {
          setItem: (k, v) => map.set(k, String(v)),
          getItem: (k) => (map.has(k) ? map.get(k) : null),
          removeItem: (k) => map.delete(k),
          clear: () => map.clear(),
        };
      })();



// ── 1. The basics: setItem / getItem / removeItem ─────────────────────────
// ⚠ Storage only holds STRINGS. Numbers/booleans get converted to strings.
storage.setItem('username', 'Sam');
console.log(storage.getItem('username')); // => Sam
console.log(storage.getItem('missing'));  // => null  (not undefined!)

storage.setItem('count', 5);
console.log(typeof storage.getItem('count')); // => string ("5", not 5!)



// ── 2. Storing objects/arrays → JSON.stringify on the way in, parse on out ──
const user = { name: 'Ana', theme: 'dark', favorites: ['js', 'css'] };

storage.setItem('user', JSON.stringify(user));        // object → string
const loaded = JSON.parse(storage.getItem('user'));   // string → object
console.log(loaded.favorites); // => [ 'js', 'css' ]
```

```
// ── 3. A safe save/load helper (handles missing keys & bad JSON) ──────────
function save(key, value) {
  storage.setItem(key, JSON.stringify(value));
}
function load(key, fallback = null) {
  const raw = storage.getItem(key);
  if (raw === null) return fallback;
  try {
    return JSON.parse(raw);
  } catch {
    return fallback; // corrupted data → return the default instead of crashing
  }
}

save('settings', { volume: 0.8, muted: false });
console.log(load('settings'));            // => { volume: 0.8, muted: false }
console.log(load('nothing', 'default'));  // => default


// ── 4. Removing & clearing ─────────────────────────────────────────────────
storage.removeItem('count'); // delete one key
console.log(storage.getItem('count')); // => null
// storage.clear();          // ⚠ wipes EVERYTHING for this site


/* ── 5. localStorage vs sessionStorage vs cookies ──────────────────────────
 *
 *   localStorage   → persists FOREVER (until cleared). Same API as above.
 *                    Use for: theme, settings, draft content, cached data.
 *
 *   sessionStorage → cleared when the TAB closes. Identical API:
 *                    sessionStorage.setItem('k', 'v')
 *                    Use for: one-visit state, multi-step form progress.
 *
 *   cookies        → small (~4KB), sent to the SERVER on every request.
 *                    Read/write the raw string:
 *                      document.cookie = 'token=abc; max-age=3600; path=/';
 *                      document.cookie  // "token=abc; other=..."
 *                    Use for: auth tokens / things the server must see.
 *                    (For real apps, set auth cookies from the server as
 *                     HttpOnly so JS can't read them — safer.)
 *
 *   LIMITS: localStorage/sessionStorage ~5MB, synchronous, strings only,
 *           per-origin (one site can't read another's storage).
 */
```

```
/* ── 6. Real browser example (uncomment when loaded via index.html) ──────────
 *   // Persist a counter across page reloads:
 *   const n = Number(localStorage.getItem('visits') || 0) + 1;
 *   localStorage.setItem('visits', n);
 *   console.log(`You've visited ${n} times`);
 */



/* PRACTICE ----------------------------------------------------------------
 *   1. Save a to-do array to storage, reload it, and add one item.
 *   2. Build loadTheme()/saveTheme() that default to 'light' when unset.
 *   3. In the browser, make a counter that survives page refreshes.
 * ------------------------------------------------------------------------ */
```

34 · BOM & TIMERS (the Browser Object Model)

```
/* ============================================================================
 * 34 · BOM & TIMERS (the Browser Object Model)
 * ============================================================================
 * Run:  node lessons/34-bom-timers.js   (timers run; BOM parts are shown safely)
 *
 * WHAT YOU'LL LEARN
 *   • setTimeout / setInterval and how to cancel them
 *   • The BOM: window, location, history, navigator, screen
 *   • A practical countdown timer pattern
 *
 * The DOM (lessons 23–24) is the PAGE. The BOM is the BROWSER around it —
 * the window, the URL, history, the device. Timers work everywhere.
 * ========================================================================== */

// ── 1. setTimeout — run once after a delay ──────────────────────────────────
const id = setTimeout(() => {
  console.log('runs after 50ms');
}, 50);
// Cancel it before it fires with clearTimeout(id):
// clearTimeout(id);



// ── 2. setInterval — run repeatedly every N ms ──────────────────────────────
let ticks = 0;
const intervalId = setInterval(() => {
  ticks++;
  console.log('tick', ticks);
  if (ticks >= 3) {
    clearInterval(intervalId); // ⚠ ALWAYS stop intervals or they run forever
    console.log('interval stopped');
  }
```

```
}, 20);


// ── 3. A practical countdown ─────────────────────────────────────────
function countdown(from, onTick, onDone) {
  let n = from;
  const t = setInterval(() => {
    onTick(n);
    if (n === 0) {
      clearInterval(t);
      onDone();
    }
    n--;
  }, 15);
  return t;
}
countdown(3, (n) => console.log(`T-minus ${n}`), () => console.log('🚀 liftoff'));


// ── 4. setTimeout(0) — defer work to "after the current code" ────────
// Not actually 0ms — it runs after the current synchronous code & microtasks.
console.log('sync 1');
setTimeout(() => console.log('deferred (macrotask)'), 0);
console.log('sync 2'); // logs before "deferred" — see lesson 29's event loop


/* ── 5. The BOM objects (browser only — shown as comments) ──────────
 *
 *  window — the global object in browsers. window.x === x at top level.
 *           window.innerWidth / innerHeight  → viewport size
 *           window.alert() / confirm() / prompt()  → dialog boxes
 *           window.scrollTo(0, 0)             → scroll the page
 *
 *  location — the current URL (read AND navigate):
 *           location.href      // "https://site.com/page?q=1#top"
 *           location.pathname  // "/page"
 *           location.search    // "?q=1"
 *           location.hash      // "#top"
 *           location.href = 'https://example.com'  // navigate to a new page
 *           location.reload()                      // refresh
 *
 *  history — the back/forward stack:
 *           history.back();  history.forward();  history.go(-2);
 *           history.pushState({}, '', '/new-url'); // change URL, no reload (SPAs)
 *
 *  navigator — info about the browser/device:
 *           navigator.language          // "en-US"
 *           navigator.onLine            // true / false
 *           navigator.clipboard.writeText('copied!')
 *           navigator.geolocation.getCurrentPosition(...)
```

```
 *
 *  screen — the physical display:  screen.width, screen.height
 */



// ── 6. Reading query params (works in browser; demo'd with URL here) ──────────
// In a browser you'd use: new URLSearchParams(location.search)
const params = new URLSearchParams('?user=sam&page=2');
console.log(params.get('user')); // => sam
console.log(params.get('page')); // => 2
console.log([...params]);        // => [ [ 'user', 'sam' ], [ 'page', '2' ] ]



/* globalThis — the global object EVERYWHERE ---------------------------------
 *   In browsers the global is `window`; in Node it's `global`. `globalThis`
 *   works in both, so portable code uses it.
 */
console.log(typeof globalThis); // => object



/* PRACTICE ------------------------------------------------------------------
 *   1. Build a clock that logs the time every second, then stops after 5.
 *   2. Parse "?theme=dark&lang=en" with URLSearchParams and read both values.
 *   3. (Browser) Log location.pathname and navigator.language in the console.
 * ------------------------------------------------------------------------- */
```

35 · DEBUGGING & THE CONSOLE

```
/* ============================================================================
 * 35 · DEBUGGING & THE CONSOLE
 * ============================================================================
 * Run:  node lessons/35-debugging-console.js
 *
 * WHAT YOU'LL LEARN
 *   • The console methods beyond console.log
 *   • Reading error messages & stack traces
 *   • The `debugger` statement and breakpoints
 *   • A practical debugging mindset
 *
 * Debugging is HALF of programming. Knowing your tools turns "why won't this
 * work?!" into a calm, 2-minute investigation.
 * ========================================================================== */

// ── 1. Beyond console.log — the full console toolkit ──────────────────────────
console.log('plain message');
console.info('ℹ️  informational');
console.warn('⚠  a warning (yellow in browsers)');
```

```
console.error('❌ an error (red — does NOT stop the program)');

// console.table — gorgeous for arrays of objects:
console.table([
  { name: 'Ana', age: 25 },
  { name: 'Bob', age: 30 },
]);

// Grouping related logs (indented; collapsible in the browser):
console.group('User load');
console.log('fetching...');
console.log('done');
console.groupEnd();

// Counting how many times a line runs:
console.count('loop'); // => loop: 1
console.count('loop'); // => loop: 2

// Timing how long something takes:
console.time('work');
let sum = 0;
for (let i = 0; i < 1e6; i++) sum += i;
console.timeEnd('work'); // => work: 1.2ms (your number will vary)

// Assertions — log ONLY if the condition is false (great for sanity checks):
console.assert(sum > 0, 'sum should be positive'); // (prints nothing — it passed)



// ── 2. The log-the-label trick (see WHICH variable is which) ──────────────
const x = 5, y = 10;
// Wrap variables in an object → labels come for free:
console.log({ x, y }); // => { x: 5, y: 10 }   (clearer than console.log(x, y))



// ── 3. Reading a stack trace ──────────────────────────────────────────────
function level3() {
  throw new Error('something failed deep down');
}
function level2() { level3(); }
function level1() { level2(); }

try {
  level1();
} catch (err) {
  console.log('message:', err.message); // => something failed deep down
  // err.stack shows the call path TOP-to-bottom (where it threw → who called it):
  console.log('first stack line:', err.stack.split('\n')[1].trim());
  // Read stacks from the TOP: that's where the error actually happened.
}
```

```
// ── 4. The `debugger` statement & breakpoints ──────────────────────────
// Drop `debugger;` into your code. When DevTools (browser) or an inspector
// (node --inspect) is open, execution PAUSES there so you can inspect variables
// step by step. It's ignored when no debugger is attached.
function buggyAverage(nums) {
  // debugger; // ← uncomment, run `node --inspect-brk` or use VS Code's debugger
  const total = nums.reduce((a, b) => a + b, 0);
  return total / nums.length;
}
console.log(buggyAverage([2, 4, 6])); // => 4


/* ── 5. A debugging MINDSET (more useful than any tool) ──────────────────
 *   1. Reproduce it reliably — know the exact steps that trigger the bug.
 *   2. Read the FULL error message + the FIRST line of the stack trace.
 *   3. Isolate: log values right BEFORE the failing line. What's unexpected?
 *   4. Check assumptions: log the TYPE too — console.log(typeof x, x).
 *      (Most bugs are "I thought this was a number but it's a string/undefined.")
 *   5. Narrow it down: comment out half, see if the bug remains. Repeat.
 *   6. Fix the CAUSE, not the symptom. Then re-run to confirm.
 *
 *   COMMON ERROR MESSAGES, DECODED:
 *     "x is not defined"            → typo, or used before declaring (TDZ).
 *     "Cannot read properties of    → you accessed .foo on null/undefined.
 *      undefined (reading 'foo')"      Log the object first; use ?. (lesson 03).
 *     "x is not a function"         → it's not what you think (typo, or undefined).
 *     "Unexpected token"            → a syntax error (missing }, ), or comma).
 */


/* PRACTICE ----------------------------------------------------------------
 *   1. Use console.table to print an array of 3 product objects.
 *   2. Trigger a "Cannot read properties of undefined" on purpose, then fix it
 *      with optional chaining.
 *   3. Wrap a slow loop in console.time/timeEnd and read the duration.
 * ------------------------------------------------------------------------ */
```

36 · WEAKMAP, MEMORY & FINAL LANGUAGE DETAILS

```
/* ============================================================================
 * 36 · WEAKMAP, MEMORY & FINAL LANGUAGE DETAILS
 * ============================================================================
 * Run:  node lessons/36-weakmap-memory.js
 *
 * WHAT YOU'LL LEARN
```

```
 *   • How JS memory & garbage collection work (the short version)
 *   • WeakMap / WeakSet and why "weak" matters
 *   • WeakRef & FinalizationRegistry (advanced GC hooks)
 *   • Strict mode
 *   • Tagged template literals
 *
 * The last few pieces to round out your JavaScript knowledge.
 * ========================================================================== */

// ── 1. Garbage collection in 60 seconds ──────────────────────────────────
// JS manages memory FOR you. An object stays in memory as long as something
// can still REACH it (a variable, an array, a property...). When nothing
// references it anymore, the garbage collector frees it automatically.
let data = { big: 'lots of stuff' };
data = null; // the object is now unreachable → eligible to be cleaned up
// You don't free memory manually — but you CAN cause "leaks" by holding
// references you forgot about (e.g. an ever-growing array, or listeners you
// never remove). The fix is usually: stop referencing what you're done with.



// ── 2. Map vs WeakMap ────────────────────────────────────────────────────
// A normal Map holds its keys STRONGLY — they can never be garbage-collected
// while the Map exists. A WeakMap holds keys WEAKLY: if the key object is no
// longer referenced anywhere else, it (and its value) can be cleaned up.

const weak = new WeakMap();
let user = { name: 'Sam' };
weak.set(user, { lastLogin: '2026-06-04' }); // associate metadata with an object
console.log(weak.get(user)); // => { lastLogin: '2026-06-04' }
console.log(weak.has(user)); // => true

user = null; // now nothing else references the user object →
             // the WeakMap entry becomes eligible for garbage collection.

// RESTRICTIONS (the price of being "weak"):
//   • Keys MUST be objects (not strings/numbers).
//   • NOT iterable — no .size, no for...of, no .keys(). You can't list entries.
// USE CASES: attach private/extra data to objects without preventing their
// cleanup — caches, metadata, "have I processed this object?" tracking.



// ── 3. WeakSet — same idea, for a set of objects ─────────────────────────
const seen = new WeakSet();
let task = { id: 1 };
seen.add(task);
console.log(seen.has(task)); // => true
task = null; // entry can be garbage-collected; you don't have to clean up
// Great for "have I already handled this object?" without leaking memory.
```

```
// ── 3b. WeakRef & FinalizationRegistry (advanced — use rarely) ──────────────
// A WeakRef holds a reference that does NOT keep its target alive: the GC may
// still collect the object. Call .deref() to get it back — or `undefined` if
// it's already been collected. Useful for memory-sensitive caches.
let cached = { data: 'expensive to compute' };
const ref = new WeakRef(cached);
console.log(ref.deref()?.data); // => expensive to compute  (still alive here)
// After `cached = null` AND a GC cycle, ref.deref() would return undefined.
// ⚠ You can't predict WHEN the GC runs, so never rely on timing.

// A FinalizationRegistry lets you register a cleanup callback that MAY run after
// an object is collected — e.g. to release an external resource tied to it.
const registry = new FinalizationRegistry((heldValue) => {
  console.log('cleanup ran for:', heldValue); // may fire later, or never
});
registry.register(cached, 'cache-entry-1'); // associate a label with the object
// ⚠ Finalizers are NOT guaranteed to run (e.g. on exit) and timing is
// unpredictable. Treat them as a best-effort safety net, never your main logic.
// RULE OF THUMB: prefer WeakMap/WeakSet; reach for WeakRef/FinalizationRegistry
// only for genuine memory caches or wrapping external (C/WASM) resources.


// ── 4. Strict mode ──────────────────────────────────────────────────────────
// 'use strict' opts into a safer JS: it turns silent mistakes into errors.
// ES modules (lesson 22) and class bodies are ALWAYS strict automatically.
function sloppyVsStrict() {
  'use strict';
  // mistyped = 5;  // ❌ in strict mode: ReferenceError (without it: silently
  //                //    creates a global — a classic hard-to-find bug!)
  return 'strict mode catches accidental globals & other footguns';
}
console.log(sloppyVsStrict());
// Strict mode also: forbids duplicate params, makes `this` undefined in plain
// function calls (lesson 11), and disallows deleting variables. Prefer it
// (you get it free with modules/classes).


// ── 5. Tagged template literals ─────────────────────────────────────────────
// A function placed before a template literal receives the string PIECES and
// the interpolated VALUES separately — letting you process them.
function highlight(strings, ...values) {
  // strings: the text chunks; values: the ${...} results
  return strings.reduce(
    (out, str, i) => out + str + (i < values.length ? `[${values[i]}]` : ''),
    ''
  );
}
const name = 'Sam';
const score = 95;
```

```
console.log(highlight`User ${name} scored ${score} points`);
// => User [Sam] scored [95] points

// Real-world tags: styled-components (CSS-in-JS), String.raw, safe HTML/SQL
// escaping, i18n. String.raw is a built-in tag that ignores escape sequences:
console.log(String.raw`C:\new\test`); // => C:\new\test  (\n NOT a newline)



/* MEMORY-LEAK AVOIDANCE TIPS ------------------------------------------------
 *   • Remove event listeners when you're done (removeEventListener).
 *   • Clear timers you no longer need (clearInterval / clearTimeout).
 *   • Don't let global arrays/objects grow forever — cap or clear them.
 *   • Use WeakMap/WeakSet when associating data with objects you don't own.
 * ------------------------------------------------------------------------- */

/* 🎉 THAT'S THE WHOLE CORE LANGUAGE — basics, intermediate, advanced, and the
 *    browser. From here the course shifts gears:
 *      • Part 10  (37)     — modern JS (ES2021–ES2026)
 *      • Part 11  (38–47)  — professional & production engineering
 *      • Part 12  (48–50)  — computer-science core (data structures,
 *                            algorithms) and binary data
 *    A great next step right now: build a small app (to-do, weather, quiz)
 *    using lessons 23–25 + 33. Building is what makes it all stick. */

/* PRACTICE ------------------------------------------------------------------
 *   1. Use a WeakMap to cache a computed result keyed by an object argument.
 *   2. Add 'use strict' to a function and trigger an accidental-global error.
 *   3. Write a tag function that uppercases every interpolated value.
 *   4. Wrap an object in a WeakRef, log .deref()?.x, then set the original to
 *      null and explain why .deref() MIGHT (eventually) return undefined.
 * ------------------------------------------------------------------------- */
```

37 · MODERN JAVASCRIPT (ES2021 → ES2026)

```
/* ============================================================================
 * 37 · MODERN JAVASCRIPT (ES2021 → ES2026)
 * ============================================================================
 * Run:  node lessons/37-modern-js.js   (needs Node 22+ ; you have v24 ✅)
 *
 * WHAT YOU'LL LEARN
 *   The newest, genuinely useful additions to JavaScript — grouped by year.
 *   These make a lot of older patterns cleaner. Several REPLACE workarounds
 *   you saw earlier in the course (noted with "↪ replaces ...").
 *
 * NOTE: very new features need an up-to-date runtime. Everything here runs in
 *       Node 22/24 and current browsers (2024+). Older environments may not
 *       support the ES2024/ES2025 items yet.
```

```
 * ========================================================================== */

console.log('===== ES2021 =====');

// ── Logical assignment: ??=  ||=  &&= ──────────────────────────────────────
// Combine a logical check with assignment. Super handy for defaults.
let config = { theme: null, retries: 0 };
config.theme ??= 'light';   // assign ONLY if null/undefined  → 'light'
config.retries ||= 3;       // assign if falsy (0 counts!)     → 3
config.theme &&= 'dark';    // assign only if already truthy   → 'dark'
console.log(config);        // => { theme: 'dark', retries: 3 }
// ↪ replaces:  if (config.theme == null) config.theme = 'light'

// ── Promise.any — first to SUCCEED wins ────────────────────────────────────
// (vs Promise.race which settles on the first to finish, success OR fail)
const fastest = await Promise.any([
  Promise.reject('server A down'),
  Promise.resolve('server B responded'),
]);
console.log(fastest); // => server B responded



console.log('===== ES2022 =====');

// ── Top-level await ────────────────────────────────────────────────────────
// In ES modules you can `await` at the top level — no async IIFE wrapper.
// (This file uses it; that's why awaits above work without an async function.)
const ready = await Promise.resolve('no IIFE needed');
console.log(ready); // => no IIFE needed

// ── Object.hasOwn(obj, key) — the safe "has own property" check ────────────
const obj = { a: 1 };
console.log(Object.hasOwn(obj, 'a'));        // => true
console.log(Object.hasOwn(obj, 'toString')); // => false (inherited, not own)
// ↪ replaces the clunky:  Object.prototype.hasOwnProperty.call(obj, 'a')

// ── Error cause — chain errors without losing the original ─────────────────
try {
  try {
    throw new Error('low-level network fail');
  } catch (original) {
    throw new Error('Could not load user', { cause: original });
  }
} catch (err) {
  console.log(err.message, '| caused by:', err.cause.message);
  // => Could not load user | caused by: low-level network fail
}
```

```
console.log('===== ES2023 =====');

// ── findLast / findLastIndex — search from the END ──────────────────────
const nums = [1, 2, 3, 4, 5];
console.log(nums.findLast((n) => n % 2 === 1));      // => 5 (last odd)
console.log(nums.findLastIndex((n) => n % 2 === 1)); // => 4

// ── Immutable array methods — return a NEW array, no mutation ⭐ ─────────
// These are the modern fix for the "copy-before-sort" dance from lessons 12/13.
const original = [3, 1, 2];
console.log(original.toSorted((a, b) => a - b)); // => [1, 2, 3]
console.log(original.toReversed());              // => [2, 1, 3]
console.log(original.with(0, 99));               // => [99, 1, 2]  (replace index 0)
console.log(original.toSpliced(1, 1));           // => [3, 2]      (remove 1 at idx 1)
console.log(original);                           // => [3, 1, 2]  UNCHANGED ✅
// ↪ replaces:  [...arr].sort(...)   and   [...arr].reverse()


console.log('===== ES2024 =====');

// ── Object.groupBy — group array items by a key ⭐ ──────────────────────
const people = [
  { name: 'Ana', dept: 'eng' },
  { name: 'Bob', dept: 'sales' },
  { name: 'Cy', dept: 'eng' },
];
const byDept = Object.groupBy(people, (p) => p.dept);
console.log(byDept.eng.map((p) => p.name)); // => [ 'Ana', 'Cy' ]
// ↪ replaces a whole reduce() into an object (you wrote one in lesson 13!)

// Map.groupBy is the same, but returns a Map (keys can be any type):
const byParity = Map.groupBy([1, 2, 3, 4], (n) => (n % 2 ? 'odd' : 'even'));
console.log(byParity.get('odd')); // => [ 1, 3 ]

// ── Promise.withResolvers — get a promise + its resolve/reject together ──
// Cleaner than capturing them inside the executor with outer `let`s.
const { promise, resolve } = Promise.withResolvers();
setTimeout(() => resolve('done!'), 10);
console.log(await promise); // => done!

// ── Array.fromAsync — build an array from an async iterable ──────────────
async function* drip() { yield 1; yield 2; yield 3; }
console.log(await Array.fromAsync(drip())); // => [ 1, 2, 3 ]


console.log('===== ES2025 (newest) =====');

// ── Iterator helpers — map/filter/take LAZILY, without arrays ⭐ ─────────
// You can now chain methods directly on iterators (e.g. from .values(),
```

```
// generators, Map/Set entries) — processed lazily, one item at a time.
function* naturals() { let i = 1; while (true) yield i++; }
const firstThreeEvens = naturals()
  .filter((n) => n % 2 === 0)
  .map((n) => n * 10)
  .take(3)                       // stop after 3 — works on an INFINITE source
  .toArray();
console.log(firstThreeEvens); // => [ 20, 40, 60 ]
// ↳ before, .map/.filter only existed on arrays (you had to materialize first)

// ── New Set methods — real set algebra ⭐ ──────────────────────────────
const a = new Set([1, 2, 3, 4]);
const b = new Set([3, 4, 5, 6]);
console.log([...a.union(b)]);              // => [ 1, 2, 3, 4, 5, 6 ]
console.log([...a.intersection(b)]);       // => [ 3, 4 ]
console.log([...a.difference(b)]);         // => [ 1, 2 ]
console.log([...a.symmetricDifference(b)]);// => [ 1, 2, 5, 6 ]
console.log(new Set([1, 2]).isSubsetOf(a));// => true

// ── Promise.try — run a function and ALWAYS get a promise back ─────────
// Wraps sync OR async functions uniformly; sync throws become rejections.
const result = await Promise.try(() => 42);
console.log(result); // => 42

// ── RegExp.escape — safely put arbitrary text into a regex ─────────────
const userInput = 'a.b*c';
const safe = new RegExp(RegExp.escape(userInput)); // dots/stars treated literally
console.log(safe.test('a.b*c')); // => true
console.log(safe.test('aXbYc')); // => false (the . and * are NOT wildcards now)



console.log('===== ES2026 & THE CUTTING EDGE =====');

// ── using / await using — automatic cleanup (explicit resource management) ──
// A `using` declaration auto-calls the resource's [Symbol.dispose]() when the
// block ends — like try/finally, but automatic. `await using` calls the async
// [Symbol.asyncDispose](). Perfect for files, DB connections, locks, sockets.
// (Stage 3 / ES2026; needs a very recent runtime. Shown so you recognize it.)
//
//   function openResource(name) {
//     return { [Symbol.dispose]() { console.log(`closing ${name}`); } };
//   }
//   {
//     using db = openResource('db');   // no manual close needed
//     using file = openResource('file');
//     // ...use them...
//   } // ← here both are disposed automatically, in REVERSE order
//   // ↳ replaces the try { } finally { db.close(); file.close(); } dance.

// ── Import attributes — import non-JS modules safely ───────────────────
```

```
// Tell the engine what kind of module you're importing (currently JSON):
//   import config from './config.json' with { type: 'json' };
//   const data = await import('./data.json', { with: { type: 'json' } });
// ↪ replaces reading + JSON.parse-ing a file by hand for static config.

// ── Decorators — annotate classes/methods with @reusable behavior ──────────
// A decorator is a function that wraps a class/method to add behavior (logging,
// validation, memoization) declaratively. (Stage 3; TypeScript/Angular already
// use them; frameworks lean on them heavily.)
//
//   function logged(originalMethod, ctx) {
//     return function (...args) {
//       console.log(`calling ${ctx.name}`);
//       return originalMethod.call(this, ...args);
//     };
//   }
//   class Api { @logged getUser(id) { /* ... */ } } // @logged wraps the method
// ↪ a standardized version of the "decorator" pattern from lesson 40.

// ── Float16Array & Math.f16round — half-precision numbers (ES2025/26) ───────
// 16-bit floats: half the memory, enough precision for ML weights, graphics,
// and GPU data. A typed array (lesson 50) for compact decimal data.
if (typeof Float16Array !== 'undefined') {
  const half = new Float16Array([1.5, 2.25]);
  console.log('Float16Array:', half[0]); // => 1.5
} else {
  console.log('Float16Array: not in this runtime yet (very new)');
}



console.log('===== ON THE HORIZON =====');

/* ── Temporal API — the modern replacement for Date (NOT shipped yet) ─────────
 * Date is famously awkward (0-based months, mutable, no time-zone clarity).
 * Temporal fixes all of it. It's at Stage 3 and not yet in Node/most browsers,
 * so this is shown as a comment. Use the @js-temporal/polyfill package to try it.
 *
 *   const today = Temporal.Now.plainDateISO();      // 2026-06-04 (immutable)
 *   const next = today.add({ days: 7 });            // clean date math
 *   const meeting = Temporal.ZonedDateTime.from('2026-06-04T15:00[Asia/Kolkata]');
 *   today.month            // 6  (1-based — finally sane!)
 *   today.until(next).days // 7
 *
 * Until it ships, use lesson 32's Date + Intl, or a library (date-fns, Day.js).
 */
console.log('Temporal: coming soon — see comments. For now use lesson 32.');



/* WHAT CHANGED FOR YOUR EVERYDAY CODE ----------------------------------------
 *   • Sorting without mutating ......... arr.toSorted()       (was [...arr].sort())
```

```
 *   • Grouping data ................... Object.groupBy(...)   (was a reduce)
 *   • Defaults ........................ x ??= value
 *   • Own-property check .............. Object.hasOwn(o, k)
 *   • Lazy pipelines .................. iterator.map().filter().take()
 *   • Set math ........................ a.union(b), a.intersection(b)
 *   These are all "nice to have" — the fundamentals you already learned still
 *   work everywhere. Reach for these when your runtime supports them.
 * ------------------------------------------------------------------------ */

/* PRACTICE ---------------------------------------------------------------
 *   1. Rewrite lesson 13's letter-counting reduce using Object.groupBy.
 *   2. Replace a [...arr].sort() somewhere in your code with arr.toSorted().
 *   3. Use Set.intersection to find common items in two arrays of tags.
 * ------------------------------------------------------------------------ */
```

38 · TESTING — proving your code works

```
/* ========================================================================
 * 38 · TESTING — proving your code works
 * ========================================================================
 * Run:  node --test lessons/38-testing.js   (or just: node lessons/38-testing.js)
 *
 * WHAT YOU'LL LEARN
 *   • Why we write automated tests
 *   • Node's BUILT-IN test runner (node:test) + assert — zero install
 *   • Unit tests, grouping, async tests, and mocking
 *   • The testing pyramid: unit → integration → end-to-end (E2E)
 *   • Code coverage
 *   • The TDD loop (red → green → refactor)
 *
 * Node 18+ ships a real test runner. The same ideas map 1:1 onto Jest/Vitest
 * (describe/it/expect) — only the import and matcher names differ.
 * ======================================================================== */

import { test, describe, mock, before, after } from 'node:test';
import assert from 'node:assert/strict'; // "strict" = uses === by default

// ── The code we want to test (normally this lives in another file) ──────────
function add(a, b) {
  return a + b;
}
function divide(a, b) {
  if (b === 0) throw new Error('cannot divide by zero');
  return a / b;
}
async function fetchUser(id) {
  // pretend this hits a network; resolves after a tick
  return { id, name: 'Sam' };
```

```
}


// ── 1. A single unit test ──────────────────────────────────────────
test('add() sums two numbers', () => {
  assert.equal(add(2, 3), 5);          // pass: 2 + 3 === 5
  assert.notEqual(add(2, 3), 6);
});


// ── 2. Grouping related tests with describe ────────────────────────
describe('divide()', () => {
  test('divides normally', () => {
    assert.equal(divide(10, 2), 5);
  });

  test('throws on divide-by-zero', () => {
    // assert.throws checks that the function DOES throw (and the message matches)
    assert.throws(() => divide(1, 0), /cannot divide by zero/);
  });
});


// ── 3. Common assertions ───────────────────────────────────────────
test('assertion styles', () => {
  assert.ok(1 < 2);                          // truthy check
  assert.equal('a', 'a');                    // === for primitives
  assert.deepEqual({ a: 1 }, { a: 1 });      // structural equality for objects
  assert.deepEqual([1, 2], [1, 2]);          // ...and arrays
});


// ── 4. Async tests — just make the test function async and await ───
test('fetchUser() resolves with a user', async () => {
  const user = await fetchUser(7);
  assert.equal(user.id, 7);
  assert.equal(user.name, 'Sam');
});


// ── 5. Mocking — replace a real dependency with a fake you control ─
describe('mocking', () => {
  test('mock.fn records calls and returns what you tell it', () => {
    const sendEmail = mock.fn(() => 'sent'); // fake function

    function notify(send, to) {
      return send(`hello ${to}`);
    }
```

```
    const result = notify(sendEmail, 'ana@example.com');
    assert.equal(result, 'sent');
    assert.equal(sendEmail.mock.callCount(), 1);                // called once
    assert.equal(sendEmail.mock.calls[0].arguments[0], 'hello ana@example.com');
  });
});


// ── 6. Setup / teardown hooks ──────────────────────────────────────────
describe('hooks (before/after)', () => {
  let db;
  before(() => { db = { rows: [] }; });   // runs once before the tests below
  after(() => { db = null; });            // cleanup after

  test('starts empty', () => {
    assert.equal(db.rows.length, 0);
  });
});


/* ── 7. THE TESTING PYRAMID — unit / integration / E2E ──────────────────
 * Different tests catch different bugs. The PYRAMID = lots of fast unit tests,
 * fewer integration tests, a handful of slow end-to-end tests.
 *
 *   UNIT (most)        — one function/module in isolation (everything above).
 *                        Fast, run on every save. Mock the outside world.
 *   INTEGRATION (some) — several pieces TOGETHER (e.g. a route + its database).
 *                        Catches "they each work but don't fit" bugs.
 *   E2E (few)          — drive the REAL app in a REAL browser like a user would:
 *                        click, type, assert what's on screen. Slow but proves
 *                        the whole thing actually works.
 *
 * E2E is done with a browser-automation tool — PLAYWRIGHT (or Cypress):
 *   // npm i -D @playwright/test  &&  npx playwright install
 *   import { test, expect } from '@playwright/test';
 *   test('user can log in', async ({ page }) => {
 *     await page.goto('https://localhost:3000');
 *     await page.fill('#email', 'ana@example.com');
 *     await page.fill('#password', 'secret');
 *     await page.click('button[type=submit]');
 *     await expect(page.getByText('Welcome, Ana')).toBeVisible(); // real UI check
 *   });
 * Playwright can also test across Chromium/Firefox/WebKit and take screenshots.
 */


/* ── 8. CODE COVERAGE — how much of your code the tests run ─────────────
 * Coverage shows which lines/branches your tests actually exercise. Node has it
 * built in — no install:
 *
```

```
 *   node --test --experimental-test-coverage lessons/38-testing.js
 *
 * It prints a table of % lines/branches/functions covered, and which lines were
 * MISSED. (Jest/Vitest: `--coverage`.) ⚠ 100% coverage ≠ bug-free — it means
 * "executed", not "asserted correctly". Use it to FIND untested code, not as a
 * trophy.
 */



/* THE TDD LOOP ------------------------------------------------------------
 *   1. RED   — write a failing test for the behavior you want.
 *   2. GREEN — write the SIMPLEST code that makes it pass.
 *   3. REFACTOR — clean up, tests still green. Repeat.
 *   Tests are a safety net: they let you change code fearlessly.
 *
 * JEST / VITEST EQUIVALENT (same idea, different names):
 *   import { describe, it, expect } from 'vitest';
 *   it('adds', () => { expect(add(2,3)).toBe(5); });
 * ------------------------------------------------------------------------ */

/* PRACTICE ----------------------------------------------------------------
 *   1. Write a test for a `capitalize(str)` function (first letter upper).
 *   2. Test that it throws (or returns '') on an empty string.
 *   3. Mock a `logger` and assert it was called with the right message.
 *   4. Run this file with --experimental-test-coverage and read the report.
 *   5. Sketch (in comments) one unit, one integration, and one E2E test for a
 *      login feature — note what each would and wouldn't catch.
 * ------------------------------------------------------------------------ */
```

39 · SECURITY ESSENTIALS — don't get hacked

```
/* ========================================================================
 * 39 · SECURITY ESSENTIALS — don't get hacked
 * ========================================================================
 * Run:  node lessons/39-security.js
 *
 * WHAT YOU'LL LEARN
 *   • XSS and how to render user input safely
 *   • CSRF, CORS, and CSP in plain English
 *   • Input validation, auth tokens, and secure cookies
 *   • Prototype pollution & dependency safety
 *
 * Security is about NOT TRUSTING input. Most web vulnerabilities come from
 * treating user-controlled data as if it were safe code or markup.
 * ======================================================================== */

// ── 1. XSS (Cross-Site Scripting) — the #1 frontend vulnerability ──────────
// XSS happens when user input is injected into the page AS HTML/script.
```

```
// ❌ DANGEROUS:  el.innerHTML = userInput   ← a <script> or onerror runs!
// ✅ SAFE:       el.textContent = userInput  ← rendered as plain text, never run.
const userInput = '<img src=x onerror="stealCookies()">';

// If you MUST build HTML, escape the dangerous characters first:
function escapeHTML(str) {
  return str
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
    .replaceAll('"', '&quot;')
    .replaceAll("'", '&#39;');
}
console.log(escapeHTML(userInput));
// => &lt;img src=x onerror=&quot;stealCookies()&quot;&gt;  (harmless text now)
// In real apps, prefer textContent, or a vetted sanitizer like DOMPurify for
// rich HTML. Frameworks (React/Vue) escape by default — don't bypass them.


// ── 2. CSRF (Cross-Site Request Forgery) ──────────────────────────────
// A malicious site tricks the user's browser into sending a request to YOUR
// site using their logged-in cookies (e.g. a hidden form that transfers money).
// Defenses:
//   • CSRF tokens — a secret per-session value the server checks on writes.
//   • SameSite cookies — `Set-Cookie: token=...; SameSite=Strict` (or Lax).
//   • Check the Origin/Referer header on state-changing requests.


// ── 3. CORS (Cross-Origin Resource Sharing) ───────────────────────────
// Browsers BLOCK JS from reading responses across origins UNLESS the server
// opts in with headers. CORS is the SERVER granting permission — it is NOT a
// way to "fix" a blocked request from the client side.
//   Server sends:  Access-Control-Allow-Origin: https://your-app.com
// "No 'Access-Control-Allow-Origin' header" = the API must allow your origin.


// ── 4. CSP (Content Security Policy) ──────────────────────────────────
// An HTTP header that whitelists where scripts/styles/images may load from.
// A strong CSP turns many XSS bugs into no-ops (inline scripts won't run).
//   Content-Security-Policy: default-src 'self'; script-src 'self'


// ── 5. Validate & sanitize ALL input ──────────────────────────────────
// Never trust the client — validate on the SERVER too (client checks are UX).
function validateSignup({ email, age }) {
  const errors = [];
  if (!/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(email)) errors.push('invalid email');
  if (!Number.isInteger(age) || age < 13 || age > 120) errors.push('invalid age');
  return { ok: errors.length === 0, errors };
```

```
}
console.log(validateSignup({ email: 'bad', age: 5 }));
// => { ok: false, errors: [ 'invalid email', 'invalid age' ] }
console.log(validateSignup({ email: 'a@b.co', age: 30 })); // => { ok: true, errors: [] }



// ── 6. Auth tokens & cookies — rules of thumb ──────────────────────────
// • NEVER store secrets/passwords in plain text — hash with bcrypt/argon2 (server).
// • A JWT is 3 base64 parts: header.payload.signature. The payload is NOT
//   encrypted — anyone can read it. Don't put secrets in it; verify the signature.
const fakeJwt = 'eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoiYW5hIn0.sig';
const [, payload] = fakeJwt.split('.');
console.log(JSON.parse(Buffer.from(payload, 'base64').toString())); // => { user: 'ana' }
// • Store session tokens in cookies set:  HttpOnly; Secure; SameSite=Strict
//   (HttpOnly = JS can't read it → safe from XSS theft; Secure = HTTPS only.)



// ── 7. Prototype pollution ─────────────────────────────────────────────
// Merging untrusted JSON that contains "__proto__" can poison EVERY object.
// Guard against keys named __proto__ / constructor / prototype when merging,
// or use Object.create(null) / Map for untrusted key-value data.
const safeMap = new Map();
safeMap.set('__proto__', 'harmless here'); // a Map key is just data, not the proto
console.log(safeMap.get('__proto__')); // => harmless here



// ── 8. Dependency & supply-chain safety ────────────────────────────────
// • Run `npm audit` and keep dependencies patched.
// • Pin versions (lockfile) and review what you install — packages run code.
// • Don't commit secrets; use environment variables (see lesson 44/45).



/* PRACTICE ------------------------------------------------------------------
 *   1. Write a safeRender(el, text) helper that uses textContent.
 *   2. Add password-strength validation (min length, has number) to validateSignup.
 *   3. Explain in a comment why CORS errors can't be fixed from the browser.
 * ---------------------------------------------------------------------------- */
```

40 · DESIGN PATTERNS — reusable solutions with shared names

```
/* ============================================================================
 * 40 · DESIGN PATTERNS — reusable solutions with shared names
 * ============================================================================
 * Run:  node lessons/40-design-patterns.js
 *
 * WHAT YOU'LL LEARN
 *   The classic patterns every team uses — knowing their NAMES lets you
```

```
 *   communicate designs in seconds ("let's make it an Observer").
 *
 *   • Module  • Singleton  • Factory  • Observer (Pub/Sub)  • Strategy
 *   • Decorator  • Facade  • Adapter  • Command  • Dependency Injection
 *   • A note on MVC / MVVM
 *
 * A pattern is just a well-known shape of code, not a library. Use them when
 * they fit — not everywhere.
 * ========================================================================== */

// ── 1. MODULE — group related code, hide the internals ──────────────────────
// Expose a small public API; keep everything else private (via closures).
const Counter = (() => {
  let count = 0;                // private — unreachable from outside
  return {
    increment() { count++; return count; },
    reset() { count = 0; },
    get value() { return count; },
  };
})();
Counter.increment();
Counter.increment();
console.log(Counter.value); // => 2
// (Modern ES modules — lesson 22 — are the file-level version of this.)


// ── 2. SINGLETON — exactly ONE shared instance ──────────────────────────────
// Useful for things there should only be one of: a config, a cache, a DB pool.
const Settings = (() => {
  let instance;
  function create() { return { theme: 'dark', loadedAt: 'once' }; }
  return {
    get() {
      if (!instance) instance = create(); // created lazily, only the first time
      return instance;
    },
  };
})();
console.log(Settings.get() === Settings.get()); // => true (same object every time)


// ── 3. FACTORY — a function that builds objects for you ─────────────────────
// Centralizes creation logic so callers don't use `new` or know the details.
function createUser(role) {
  const base = { role, createdAt: 'now' };
  const perks = {
    admin: { canDelete: true, canEdit: true },
    editor: { canDelete: false, canEdit: true },
    viewer: { canDelete: false, canEdit: false },
  };
```

```
  return { ...base, ...(perks[role] ?? perks.viewer) };
}
console.log(createUser('admin'));  // => { role: 'admin', createdAt: 'now', canDelete: true,
canEdit: true }
console.log(createUser('viewer')); // => { ..., canDelete: false, canEdit: false }



// ── 4. OBSERVER (Pub/Sub) — broadcast events to many listeners ──────────────
// Subscribers register callbacks; the subject notifies them all on change.
// This is how event systems, state stores, and reactive UIs work under the hood.
class EventBus {
  #listeners = {};
  on(event, fn) {
    (this.#listeners[event] ??= []).push(fn);
    return () => this.off(event, fn);     // return an unsubscribe function
  }
  off(event, fn) {
    this.#listeners[event] = (this.#listeners[event] ?? []).filter((f) => f !== fn);
  }
  emit(event, data) {
    (this.#listeners[event] ?? []).forEach((fn) => fn(data));
  }
}
const bus = new EventBus();
const unsub = bus.on('login', (user) => console.log('welcome', user));
bus.emit('login', 'Ana'); // => welcome Ana
unsub();                  // stop listening
bus.emit('login', 'Bob'); // (nothing — unsubscribed)



// ── 5. STRATEGY — swap an algorithm at runtime ──────────────────────────────
// Instead of a big if/else, store interchangeable behaviors and pick one.
const shipping = {
  standard: (w) => w * 1.0,
  express:  (w) => w * 2.5 + 5,
  pickup:   () => 0,
};
function quote(method, weight) {
  const strategy = shipping[method] ?? shipping.standard;
  return strategy(weight);
}
console.log(quote('express', 2)); // => 10
console.log(quote('pickup', 99));  // => 0



// ── 6. DECORATOR — wrap something to add behavior, same interface ───────────
// Take a function/object and return an enhanced version WITHOUT editing the
// original. (You've already met decorators: memoize and debounce wrap a fn.)
function withLogging(fn) {
  return (...args) => {
```

```
    console.log(`calling with ${JSON.stringify(args)}`);
    const result = fn(...args);
    console.log(`→ returned ${result}`);
    return result;
  };
}
const add = (a, b) => a + b;
const loudAdd = withLogging(add); // same (a, b) interface, extra behavior
loudAdd(2, 3); // logs the call + result, still returns 5


// ── 7. FACADE — a simple front over a complicated subsystem ──────────────
// Hide several messy steps behind ONE friendly method. (jQuery, axios, and most
// SDKs are facades over fetch/XHR/etc.)
const subsystemA = { connect: () => 'A ready' };
const subsystemB = { auth: () => 'B authed' };
const subsystemC = { load: () => 'C loaded' };
const app = {
  start() {                       // callers don't need to know the 3 steps
    return [subsystemA.connect(), subsystemB.auth(), subsystemC.load()].join(' | ');
  },
};
console.log('facade:', app.start()); // => A ready | B authed | C loaded


// ── 8. ADAPTER — make an incompatible interface fit ──────────────────────
// Wrap an object so it matches the shape YOUR code expects (e.g. integrating a
// third-party API whose field names differ from yours).
const oldApiUser = { user_name: 'ana', years_old: 30 };       // their shape
function userAdapter(raw) {                                     // → your shape
  return { name: raw.user_name, age: raw.years_old };
}
console.log('adapter:', userAdapter(oldApiUser)); // => { name: 'ana', age: 30 }


// ── 9. COMMAND — wrap an action as an object (enables undo/redo, queues) ──
// Each command knows how to do() and undo() itself. A history stack (lesson 48)
// then gives you undo for free — this is how editors implement Ctrl+Z.
class History {
  #done = [];
  run(command) { command.execute(); this.#done.push(command); }
  undo() { this.#done.pop()?.undo(); }
}
const doc = { text: '' };
const typeCmd = (s) => ({ execute: () => (doc.text += s), undo: () => (doc.text =
doc.text.slice(0, -s.length)) });
const history = new History();
history.run(typeCmd('Hello'));
history.run(typeCmd(' World'));
console.log('command:', doc.text); // => Hello World
```

```
history.undo();
console.log('after undo:', doc.text); // => Hello



// ── 10. DEPENDENCY INJECTION — pass collaborators IN, don't hard-code them ────
// Instead of a function reaching for a global, give it what it needs as an
// argument. Result: easy to swap a real logger for a fake one in tests (lesson 38).
function createOrderService(logger, payments) {  // dependencies injected here
  return {
    checkout(amount) {
      payments.charge(amount);
      logger.log(`charged ${amount}`);
    },
  };
}
const fakeLogger = { log: (m) => console.log('DI log:', m) };
const fakePayments = { charge: (n) => {} };
createOrderService(fakeLogger, fakePayments).checkout(50); // => DI log: charged 50



/* ── 11. MVC / MVVM (architectural patterns) — the big picture ──────────────
 * Bigger than the above; they organize a WHOLE app, not one object:
 *   MVC  — Model (data/rules) · View (UI) · Controller (handles input, updates
 *          the model, picks the view). Classic server frameworks use this.
 *   MVVM — Model · View · ViewModel (exposes data the View binds to). The
 *          ViewModel + data-binding is what React/Vue/Angular popularized.
 * Takeaway: separate WHAT your app knows (model) from HOW it's shown (view) so
 * each can change independently. Frameworks give you this structure for free.
 */



/* WHEN TO REACH FOR EACH ------------------------------------------------------
 *   Module ..... hide internals, expose a clean API
 *   Singleton .. one shared resource (config/cache) — use sparingly
 *   Factory .... complex/conditional object creation
 *   Observer ... "when X happens, tell everyone who cares" (events, state)
 *   Strategy ... many interchangeable algorithms behind one call
 *   Decorator .. add behavior by wrapping, keep the same interface
 *   Facade ..... one simple method over a complex subsystem
 *   Adapter .... make a foreign interface fit your code
 *   Command .... actions as objects → undo/redo, queues, logging
 *   Dep. Inject  pass collaborators in → testable, swappable code
 * ---------------------------------------------------------------------------- */

/* PRACTICE --------------------------------------------------------------------
 *   1. Build a Logger module with private log history and a public add()/last().
 *   2. Add a 'logout' event to the EventBus and subscribe two listeners.
 *   3. Add a 'discount' strategy set (none/student/senior) chosen at runtime.
 *   4. Write a withTiming(fn) DECORATOR that logs how long fn took (lesson 41).
 *   5. Add a 'delete line' command to the History demo and test undo on it.
```

```
 *   6. Refactor createOrderService to inject a THIRD dependency (an emailer).
 * ------------------------------------------------------------------------ */
```

41 · PERFORMANCE — make it fast (and know what "fast" means)

```
/* ========================================================================
 * 41 · PERFORMANCE — make it fast (and know what "fast" means)
 * ========================================================================
 * Run:  node lessons/41-performance.js
 *
 * WHAT YOU'LL LEARN
 *   • Big-O intuition — why algorithm choice beats micro-tweaks
 *   • Memoization & caching
 *   • Debounce/throttle recap (lesson 29)
 *   • Browser rendering: reflow/repaint, lazy loading, code splitting
 *   • Core Web Vitals — how performance is actually measured in 2026
 *   • Measuring & profiling: performance marks, PerformanceObserver
 *   • List virtualization (windowing) for huge lists
 *
 * Rule: measure before optimizing. "Premature optimization is the root of all
 * evil" — make it correct, measure, then fix the real bottleneck.
 * ======================================================================== */

// ── 1. Big-O intuition — how cost grows with input size ──────────────────
//   O(1)      constant ...... array[i], map.get(key)
//   O(log n)  logarithmic ... binary search
//   O(n)      linear ........ a single loop
//   O(n log n) ............... good sorts
//   O(n²)     quadratic ..... nested loop over the same data — avoid at scale
//
// Picking the right data structure changes the Big-O. "Is X in this list?":
const big = Array.from({ length: 100_000 }, (_, i) => i);
const asSet = new Set(big);
console.log(big.includes(99_999)); // O(n)  — scans up to 100k items
console.log(asSet.has(99_999));    // O(1)  — instant lookup ✅



// ── 2. Memoization — cache expensive results by their inputs ─────────────
function memoize(fn) {
  const cache = new Map();
  return (...args) => {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);  // cache HIT — skip the work
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}
```

```
let calls = 0;
const slowSquare = (n) => { calls++; return n * n; };
const fastSquare = memoize(slowSquare);
fastSquare(9);
fastSquare(9);            // served from cache — fn body NOT re-run
fastSquare(9);
console.log(fastSquare(9), 'computed', calls, 'time(s)'); // => 81 computed 1 time(s)


// ── 3. Debounce & throttle (limit how OFTEN code runs) — see lesson 29 ────────
// debounce: wait until activity STOPS (search-as-you-type, resize).
// throttle: run at most once per interval (scroll, mousemove).
// Both prevent doing heavy work on every single event.


// ── 4. Browser rendering performance (concept) ────────────────────────────────
// • REFLOW (layout): the browser recomputes geometry — expensive. Triggered by
//   changing size/position or reading layout props (offsetHeight) mid-mutation.
// • REPAINT: redrawing pixels (e.g. color change) — cheaper than reflow.
// Tips: batch DOM writes; build nodes off-DOM then insert once (lesson 23);
//       animate `transform`/`opacity` (GPU) instead of top/left/width.


// ── 5. Loading performance (concept) ──────────────────────────────────────────
// • LAZY LOADING: load things only when needed.
//     <img loading="lazy">           // images
//     const mod = await import('./chart.js');  // dynamic import → CODE SPLITTING
//   Code splitting ships a smaller initial bundle; the rest loads on demand.
// • Defer non-critical JS:  <script defer src="app.js"></script>
// • Compress & cache: gzip/brotli, HTTP caching headers, a CDN.


// ── 6. Core Web Vitals — the metrics that matter (2026) ───────────────────────
//   LCP (Largest Contentful Paint) — load speed   → aim < 2.5s
//   INP (Interaction to Next Paint) — responsiveness → aim < 200ms
//   CLS (Cumulative Layout Shift)   — visual stability → aim < 0.1
// Measure with Lighthouse, the Performance panel, or the web-vitals library.


// ── 7. Measuring in code ──────────────────────────────────────────────────────
const t0 = performance.now();
let sum = 0;
for (let i = 0; i < 1_000_000; i++) sum += i;
const ms = performance.now() - t0;
console.log(`loop took ${ms.toFixed(1)}ms`); // varies by machine
// (console.time/timeEnd from lesson 35 do the same thing more conveniently.)


// ── 8. Profiling with marks & measures (the Performance API) ──────────────────
```

```
// performance.now() is one stopwatch. The Performance API lets you drop named
// MARKS and MEASURE the span between them — those show up in browser DevTools'
// Performance panel too, so you can see them on a real timeline.
performance.mark('work-start');
let total = 0;
for (let i = 0; i < 500_000; i++) total += i;
performance.mark('work-end');
const measure = performance.measure('heavy-work', 'work-start', 'work-end');
console.log(`measured "${measure.name}": ${measure.duration.toFixed(1)}ms`);

// A PerformanceObserver reacts to entries as they're recorded — this is how the
// `web-vitals` library and monitoring tools collect LCP/INP/CLS in real users'
// browsers (then report them, lesson 47):
const obs = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log(`observed ${entry.entryType} "${entry.name}": ${entry.duration.toFixed(1)}ms`);
  }
});
obs.observe({ entryTypes: ['measure'] });
performance.measure('observed-span', 'work-start', 'work-end');
// In the browser you'd also observe: 'largest-contentful-paint', 'event'
// (for INP), 'layout-shift' (for CLS), 'longtask', 'resource', 'navigation'.
setTimeout(() => obs.disconnect(), 50); // stop observing (avoid leaks, lesson 36)



/* ── 9. List virtualization (windowing) — render only what's visible ──────────
 * Rendering 10,000 rows = 10,000 DOM nodes = a frozen page. VIRTUALIZATION
 * renders only the ~20 rows currently in view (a "window") and swaps their
 * contents as you scroll. The list FEELS infinite but the DOM stays tiny.
 *
 *   const ROW_H = 30, visible = Math.ceil(container.clientHeight / ROW_H);
 *   container.onscroll = () => {
 *     const start = Math.floor(container.scrollTop / ROW_H);
 *     render(items.slice(start, start + visible + 2)); // only the window
 *   };
 *
 * Libraries: TanStack Virtual, react-window. This is THE fix for slow long
 * lists/tables/feeds — it turns O(n) DOM nodes into O(window).
 */



/* PRACTICE ----------------------------------------------------------------
 *   1. Memoize a slow fibonacci(n) and compare call counts with/without it.
 *   2. Rewrite an O(n²) "find duplicates" using a Set to get O(n).
 *   3. Name which Web Vital each fixes: slow hero image, janky button, jumpy layout.
 *   4. Wrap two code blocks in performance.mark/measure and log which is slower.
 *   5. In words: why does virtualization keep a 100k-row list fast? (DOM size.)
 * ------------------------------------------------------------------------ */
```

42 · POLYFILLS — implement the built-ins yourself

```
/* ============================================================================
 * 42 · POLYFILLS — implement the built-ins yourself
 * ============================================================================
 * Run:  node lessons/42-polyfills.js
 *
 * WHAT YOU'LL LEARN
 *   Re-building the methods you use every day. This is the single best way to
 *   *truly* understand them — and it's the most common JS interview exercise.
 *
 *   We'll re-implement: map, filter, reduce, call/apply/bind, debounce,
 *   and a minimal Promise.
 *
 * (A real "polyfill" also checks `if (!Array.prototype.x)` before defining, so
 *  it only fills the gap in old engines. We focus on the implementations.)
 * ============================================================================ */

// ── 1. Array.prototype.map ──────────────────────────────────────────────────
function myMap(arr, fn) {
  const out = [];
  for (let i = 0; i < arr.length; i++) out.push(fn(arr[i], i, arr));
  return out;
}
console.log(myMap([1, 2, 3], (n) => n * 2)); // => [ 2, 4, 6 ]



// ── 2. Array.prototype.filter ───────────────────────────────────────────────
function myFilter(arr, predicate) {
  const out = [];
  for (let i = 0; i < arr.length; i++) {
    if (predicate(arr[i], i, arr)) out.push(arr[i]);
  }
  return out;
}
console.log(myFilter([1, 2, 3, 4], (n) => n % 2 === 0)); // => [ 2, 4 ]



// ── 3. Array.prototype.reduce ───────────────────────────────────────────────
function myReduce(arr, reducer, initial) {
  let acc = initial;
  let start = 0;
  if (acc === undefined) { acc = arr[0]; start = 1; } // no seed → use first item
  for (let i = start; i < arr.length; i++) acc = reducer(acc, arr[i], i, arr);
  return acc;
}
console.log(myReduce([1, 2, 3, 4], (a, b) => a + b, 0)); // => 10
```

```
// ── 4. Function.prototype.call / apply / bind ──────────────────────────
// The trick: put the function on the target object so `this` becomes that object.
function myCall(fn, context, ...args) {
  const key = Symbol('fn');
  context = context ?? globalThis;
  context[key] = fn;
  const result = context[key](...args);
  delete context[key];
  return result;
}
function myBind(fn, context, ...preset) {
  return (...later) => myCall(fn, context, ...preset, ...later);
}
const person = { name: 'Ana' };
function greet(greeting) { return `${greeting}, ${this.name}`; }
console.log(myCall(greet, person, 'Hi'));        // => Hi, Ana
const sayHello = myBind(greet, person, 'Hello');
console.log(sayHello());                          // => Hello, Ana


// ── 5. debounce — delay until calls stop (lesson 29, built from scratch) ────
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);                 // cancel the previous pending call
    timer = setTimeout(() => fn(...args), delay);
  };
}
const log = debounce((msg) => console.log('debounced:', msg), 50);
log('a'); log('b'); log('c'); // only 'c' fires, ~50ms later


// ── 6. A minimal Promise ──────────────────────────────────────────────────
// Simplified, but shows the core: state machine + queued callbacks + then().
class MyPromise {
  constructor(executor) {
    this.state = 'pending';
    this.value = undefined;
    this.callbacks = [];
    const resolve = (value) => this.#settle('fulfilled', value);
    const reject = (reason) => this.#settle('rejected', reason);
    try { executor(resolve, reject); } catch (e) { reject(e); }
  }
  #settle(state, value) {
    if (this.state !== 'pending') return;  // settle only once
    this.state = state;
    this.value = value;
    this.callbacks.forEach((cb) => cb());  // flush anyone waiting
  }
  then(onFulfilled, onRejected) {
```

```
    return new MyPromise((resolve, reject) => {
      const run = () => {
        try {
          if (this.state === 'fulfilled') resolve(onFulfilled ? onFulfilled(this.value) :
this.value);
          else reject(onRejected ? onRejected(this.value) : this.value);
        } catch (e) { reject(e); }
      };
      // if already settled, run on the microtask queue; else queue until settle
      this.state === 'pending' ? this.callbacks.push(run) : queueMicrotask(run);
    });
  }
}
new MyPromise((resolve) => setTimeout(() => resolve(21), 10))
  .then((n) => n * 2)
  .then((n) => console.log('MyPromise result:', n)); // => 42



/* PRACTICE ------------------------------------------------------------------
 *   1. Implement myForEach(arr, fn) (like map but returns nothing).
 *   2. Implement myFlat(arr, depth) using recursion (lesson 09).
 *   3. Add a .catch(fn) method to MyPromise (hint: then(undefined, fn)).
 * ---------------------------------------------------------------------------- */
```

43 · TYPESCRIPT ON-RAMP — JavaScript with types

```
/* ============================================================================
 * 43 · TYPESCRIPT ON-RAMP — JavaScript with types
 * ============================================================================
 * Run:  node lessons/43-typescript.js   (the runnable parts are plain JS)
 *
 * WHAT YOU'LL LEARN
 *   What TypeScript adds on top of the JS you already know, why teams use it,
 *   and the core syntax — so your first .ts file feels familiar.
 *
 * TypeScript = JavaScript + a TYPE LAYER checked BEFORE you run. The types are
 * erased at build time; the browser/Node runs plain JS. It catches a whole
 * class of bugs ("undefined is not a function") at your desk, not in production.
 *
 * Run real TS:  npm i -D typescript && npx tsc file.ts   (or use ts-node / Bun / Deno)
 * Below, the TS is shown in comments; the runnable code is the JS equivalent.
 * ============================================================================ */

console.log('=== TypeScript concepts (TS in comments, JS runs) ===');

/* ── 1. Basic types ──────────────────────────────────────────────────────
 *   let name: string = 'Ana';
 *   let age: number = 30;
```

```
 *   let active: boolean = true;
 *   let tags: string[] = ['js', 'ts'];
 *   let pair: [string, number] = ['x', 1];   // tuple
 * Type INFERENCE means you rarely annotate obvious things:
 *   let count = 0;          // inferred as number — reassigning to 'hi' errors
 */

/* ── 2. Function signatures ──────────────────────────────────────────── */
 *   function add(a: number, b: number): number { return a + b; }
 *   const greet = (name: string): string => `Hi ${name}`;
 *   add(2, 'x');   // ❌ compile error: 'x' is not a number  ← caught early!
 */
function add(a, b) { return a + b; } // the JS this compiles to
console.log('add(2,3):', add(2, 3)); // => 5

/* ── 3. Interfaces & type aliases — shapes of objects ────────────────── */
 *   interface User { id: number; name: string; email?: string }  // ? = optional
 *   type ID = number | string;                                    // union type
 *   function format(u: User): string { return `${u.id}:${u.name}`; }
 */
const user = { id: 1, name: 'Ana' }; // a value matching the User shape
console.log('user:', `${user.id}:${user.name}`); // => 1:Ana

/* ── 4. Union types & narrowing ──────────────────────────────────────── */
 *   function len(x: string | string[]): number {
 *     if (typeof x === 'string') return x.length;  // TS NARROWS x to string here
 *     return x.length;                             // here x is string[]
 *   }
 * Narrowing = TS uses your runtime checks (typeof, in, Array.isArray) to know
 * the precise type inside each branch. */
function len(x) {
  return typeof x === 'string' ? x.length : x.length;
}
console.log('len:', len('hello'), len(['a', 'b'])); // => 5 2

/* ── 5. Generics — reusable, type-safe containers/functions ──────────── */
 *   function first<T>(arr: T[]): T | undefined { return arr[0]; }
 *   first<number>([1, 2, 3]);   // returns number
 *   first(['a', 'b']);          // T inferred as string
 * Think "a type parameter" — like a function argument, but for types. */
function first(arr) { return arr[0]; }
console.log('first:', first([10, 20]), first(['x'])); // => 10 x

/* ── 6. Utility & helpers you'll meet ────────────────────────────────── */
 *   Partial<User>      // all fields optional
 *   Readonly<User>     // all fields immutable
 *   Pick<User,'id'>    // just some fields
 *   enum Role { Admin, Editor, Viewer }
 *   as const           // freeze a literal's type
 *   unknown vs any     // prefer `unknown` (forces you to check before use)
```

```
 */

/* ── 7. JSDoc — types WITHOUT TypeScript ──────────────────────────────────
 * You can get editor type-checking in plain .js files using JSDoc comments —
 * a nice on-ramp before adopting .ts. This is real and checkable: */
/** @param {number} n  @returns {number} */
function double(n) { return n * 2; }
console.log('double:', double(21)); // => 42

/* WHY TEAMS USE IT ----------------------------------------------------------
 *   • Bugs caught at compile time, not runtime.
 *   • Autocomplete & refactoring that actually understands your data.
 *   • Types document the code and survive refactors.
 *   Cost: a build step and a learning curve. For anything beyond a small
 *   script, most 2026 production codebases are TypeScript.
 * ------------------------------------------------------------------------- */

/* PRACTICE ------------------------------------------------------------------
 *   1. Write (in a comment) a `Product` interface: id:number, name:string,
 *      price:number, inStock?:boolean.
 *   2. Annotate a `total(items: Product[]): number` signature.
 *   3. Add JSDoc types to a real isEven(n) function in this file.
 * ------------------------------------------------------------------------- */
```

44 · TOOLING & BUILD SYSTEMS — the modern JS workflow

```
/* ============================================================================
 * 44 · TOOLING & BUILD SYSTEMS — the modern JS workflow
 * ============================================================================
 * Run:  node lessons/44-tooling.js   (a guided tour; prints the key concepts)
 *
 * WHAT YOU'LL LEARN
 *   The tools that turn your source code into a shippable app, and what each
 *   one is FOR. You don't memorize commands — you learn the moving parts.
 *
 *   npm/package.json · semver · bundlers · transpilers · linters · git · CI/CD
 * ============================================================================ */

console.log('=== The modern JavaScript toolchain ===\n');

// ── 1. npm & package.json — the project manifest ─────────────────────────
// `npm init -y` creates package.json. It records dependencies and scripts.
const packageJson = {
  name: 'my-app',
  version: '1.0.0',
  type: 'module',                       // use ES modules (lesson 22)
  scripts: {
```

```
    dev: 'vite',                        // run with: npm run dev
    build: 'vite build',
    test: 'vitest',
    lint: 'eslint .',
  },
  dependencies: { /* runtime deps:  npm install <pkg>       */ },
  devDependencies: { /* build/test:  npm install -D <pkg>   */ },
};
console.log('package.json scripts → run with `npm run <name>`:');
console.log(Object.keys(packageJson.scripts).join(', ')); // => dev, build, test, lint
// • node_modules/  — installed packages (never commit it; it's huge).
// • package-lock.json — pins EXACT versions so installs are reproducible.

// ── 2. Semantic versioning (semver):  MAJOR.MINOR.PATCH ──────────────────────
//   ^1.2.3  → allow 1.x.x (minor+patch updates)   ← npm default
//   ~1.2.3  → allow 1.2.x (patch only)
//   1.2.3   → exactly this version
console.log('semver: MAJOR(breaking).MINOR(features).PATCH(fixes)');

// ── 3. Bundlers — many files → optimized output ──────────────────────────────
// A bundler resolves your imports, bundles them, tree-shakes dead code, and
// minifies for production. Modern choices:
//   • Vite   — fast dev server + build (most common in 2026)
//   • esbuild / Rollup — the engines under many tools
//   • Webpack — older, powerful, lots of legacy projects
console.log('bundler job: resolve imports → tree-shake → minify → split');

// ── 4. Transpiling — new syntax → widely-supported JS ────────────────────────
// • Babel / esbuild convert modern JS (and JSX/TS) into code older browsers run.
// • A "browserslist" config decides how far back to compile.
// • This is why you can write ES2025 today and still support older targets.

// ── 5. Linting & formatting — consistency + catching mistakes ────────────────
// • ESLint  — flags bugs & bad patterns (unused vars, == misuse). `npx eslint .`
// • Prettier — auto-formats code so the team never argues about style.
// Run them on save and in CI so problems never reach main.

// ── 6. Git — version control basics (every project) ──────────────────────────
const gitFlow = ['git init', 'git add .', 'git commit -m "msg"', 'git push',
  'git branch feature', 'git merge', 'open a Pull Request → review → merge'];
console.log('\ngit flow:', gitFlow.join('  →  '));
// • .gitignore keeps node_modules/, .env, and build output out of the repo.

// ── 7. CI/CD — automate test & deploy ────────────────────────────────────────
// Continuous Integration: on every push, a server (GitHub Actions, etc.) runs
//   install → lint → test → build. Red build = don't merge.
// Continuous Deployment: green build auto-ships to staging/production.
console.log('CI on push: install → lint → test → build → deploy');

// ── 8. Source maps — debug the original code, not the bundle ─────────────────
```

```
// Build tools emit .map files linking minified output back to your source, so
// DevTools shows YOUR code in the debugger even in production builds.

/* THE BIG PICTURE ------------------------------------------------------------
 *   write code → lint/format → transpile → bundle → test → CI → deploy
 *   You rarely wire this by hand: `npm create vite@latest` scaffolds most of it.
 * ---------------------------------------------------------------------------- */

/* PRACTICE -------------------------------------------------------------------
 *   1. Run `npm init -y` in a folder and add a "start": "node index.js" script.
 *   2. Explain the difference between dependencies and devDependencies.
 *   3. Scaffold a real app: `npm create vite@latest` and run `npm run dev`.
 * ---------------------------------------------------------------------------- */
```

45 · NODE.JS — JavaScript on the server

```
/* ============================================================================
 * 45 · NODE.JS — JavaScript on the server
 * ============================================================================
 * Run:  node lessons/45-nodejs.js
 *
 * WHAT YOU'LL LEARN
 *   The Node-specific APIs that the browser doesn't have: process & env,
 *   the file system, paths, events, streams, and a real HTTP server.
 *
 * Node is JavaScript outside the browser — it powers backends, CLIs, and
 * tooling. Core modules live under the `node:` prefix.
 * ========================================================================== */

import process from 'node:process';
import path from 'node:path';
import os from 'node:os';
import fs from 'node:fs/promises';
import { EventEmitter } from 'node:events';
import http from 'node:http';

// ── 1. process — info about the running program ─────────────────────────
console.log('node version :', process.version);          // => v24.x
console.log('platform     :', process.platform);         // => linux / darwin / win32
console.log('cwd          :', process.cwd().split('/').pop()); // current folder name
// CLI args:  node script.js foo bar  → process.argv = [node, script, 'foo', 'bar']
console.log('args         :', process.argv.slice(2));     // => [] (none here)
// Environment variables — where secrets/config live (NEVER hard-code secrets):
//   run:  API_KEY=abc node lessons/45-nodejs.js
console.log('API_KEY      :', process.env.API_KEY ?? '(not set — see lesson 39)');
```

```
// ── 2. path — build file paths the cross-platform way ──────────────────────
console.log('join         :', path.join('src', 'utils', 'math.js')); // src/utils/math.js
console.log('extname      :', path.extname('report.pdf'));           // => .pdf
console.log('basename     :', path.basename('/a/b/notes.txt'));      // => notes.txt


// ── 3. fs — read & write files (async/await) ───────────────────────────────
const tmpFile = path.join(os.tmpdir(), 'js-learn-demo.txt');
await fs.writeFile(tmpFile, 'hello from Node\n');     // create/overwrite
const contents = await fs.readFile(tmpFile, 'utf8');  // read back as text
console.log('file says    :', contents.trim());       // => hello from Node
await fs.appendFile(tmpFile, 'second line\n');         // append
await fs.unlink(tmpFile);                              // delete (cleanup)
console.log('temp file cleaned up ✅');


// ── 4. EventEmitter — Node's built-in observer (lesson 40 pattern) ─────────
class Order extends EventEmitter {}
const order = new Order();
order.on('paid', (amount) => console.log(`order paid: $${amount}`)); // listener
order.emit('paid', 99); // => order paid: $99
// Many Node objects (streams, servers, sockets) ARE EventEmitters.


// ── 5. Streams — process data piece by piece, not all at once ──────────────
// Streams let you handle huge files/responses with low memory by working on
// CHUNKS. (Conceptual — fs.createReadStream(path).on('data', chunk => ...).)
// Examples: reading a 2GB log line-by-line, or piping a download to disk.


// ── 6. A real HTTP server ──────────────────────────────────────────────────
// http.createServer gives you a callback per request. We start it on a random
// free port (0), make one request to ourselves, print the reply, then close.
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ ok: true, path: req.url }));
});

await new Promise((resolve) => server.listen(0, resolve)); // 0 = pick a free port
const { port } = server.address();
const reply = await fetch(`http://localhost:${port}/hello`).then((r) => r.json());
console.log('server replied:', reply); // => { ok: true, path: '/hello' }
server.close(); // stop listening so the program can exit
// In real apps you'd use a framework (Express/Fastify/Hono) on top of this.


/* BROWSER vs NODE ------------------------------------------------------------
 *   Browser has: window, document, DOM, localStorage.
 *   Node has:    process, fs, path, http, os, Buffer, EventEmitter.
```

```
 *   Shared:      JS syntax, fetch, Promises, console, timers, JSON.
 * ------------------------------------------------------------------------ */

/* PRACTICE ----------------------------------------------------------------
 *   1. Read process.argv and greet a name passed on the command line.
 *   2. Write an object to a JSON file with fs.writeFile, then read it back.
 *   3. Add a '/time' route to the server that returns the current time.
 * ------------------------------------------------------------------------ */
```

46 · ADVANCED BROWSER APIs (browser)

```
/* ========================================================================
 * 46 · ADVANCED BROWSER APIs  (browser)
 * ========================================================================
 * HOW TO RUN: these are BROWSER features. Open index.html and try them in the
 * console/devtools — they don't exist in Node. This file runs harmlessly in
 * Node (it just prints a notice); the real examples are in the comments.
 *
 * WHAT YOU'LL LEARN
 *   The powerful platform APIs beyond DOM/events:
 *   Web Workers · Service Workers/PWA · IndexedDB · Observers ·
 *   Web Components · requestAnimationFrame · device APIs
 * ======================================================================== */

// Guard so the file is harmless in Node:
if (typeof window === 'undefined') {
  console.log('⚠  Lesson 46 is BROWSER-only. Open index.html and read the code in devtools.');
}

/* ── 1. Web Workers — run heavy JS off the main thread ──────────────────
 * The main thread renders the UI; long loops there FREEZE the page. A Worker
 * runs in parallel and talks via messages — no shared variables.
 *
 *   // main.js
 *   const worker = new Worker('worker.js');
 *   worker.postMessage({ n: 40 });
 *   worker.onmessage = (e) => console.log('result', e.data);
 *
 *   // worker.js
 *   onmessage = (e) => postMessage(fib(e.data.n));  // CPU work, UI stays smooth
 */

/* ── 2. Service Workers & PWAs — offline + installable ──────────────────
 * A Service Worker is a proxy between your app and the network. It can CACHE
 * assets so the app works offline and loads instantly. Plus a manifest.json =
 * an installable Progressive Web App.
 *
 *   navigator.serviceWorker.register('/sw.js');
```

```
 *   // sw.js
 *   self.addEventListener('fetch', (e) =>
 *     e.respondWith(caches.match(e.request).then((c) => c ?? fetch(e.request))));
 */

/* ── 3. IndexedDB — a real database in the browser ──────────────────────
 * localStorage (lesson 33) holds small strings synchronously. IndexedDB stores
 * large, structured, queryable data (objects, blobs) asynchronously.
 *
 *   const req = indexedDB.open('myDB', 1);
 *   req.onupgradeneeded = (e) => e.target.result.createObjectStore('users', { keyPath: 'id' });
 *   req.onsuccess = (e) => { const db = e.target.result;  ... transactions ... };
 * Libraries like `idb` make its callback API far nicer.
 */

/* ── 4. Observer APIs — react to changes efficiently ────────────────────
 * • IntersectionObserver — "is this element on screen?" → lazy-load images,
 *   infinite scroll, fade-in-on-scroll (no scroll-event spam).
 *     new IntersectionObserver(es => es.forEach(e => e.isIntersecting && load(e.target)))
 *       .observe(img);
 * • MutationObserver — watch the DOM for added/removed/changed nodes.
 * • ResizeObserver — react when an element's SIZE changes.
 */

/* ── 5. Web Components — reusable custom elements (framework-free) ──────
 * Define your own HTML tags with encapsulated markup/style via Shadow DOM.
 *
 *   class MyCounter extends HTMLElement {
 *     connectedCallback() {
 *       const root = this.attachShadow({ mode: 'open' }); // isolated DOM/CSS
 *       root.innerHTML = `<button>+</button><span>0</span>`;
 *     }
 *   }
 *   customElements.define('my-counter', MyCounter);  // <my-counter></my-counter>
 */

/* ── 6. requestAnimationFrame — smooth 60fps animation ──────────────────
 * Don't animate with setInterval. rAF runs your callback right before the next
 * paint, syncing to the display and pausing in background tabs.
 *
 *   function tick(t) { move(); requestAnimationFrame(tick); }
 *   requestAnimationFrame(tick);
 */

/* ── 7. Device & misc APIs (ask permission; HTTPS only) ─────────────────
 *   navigator.geolocation.getCurrentPosition(pos => ...);   // location
 *   await navigator.clipboard.writeText('copied!');         // clipboard
 *   new Notification('Hi!');                                // notifications
 *   matchMedia('(prefers-color-scheme: dark)').matches;     // dark mode
 *   navigator.onLine;                                       // online status
```

```
 */

/* PRACTICE (in the browser) ------------------------------------------------
 *   1. Use IntersectionObserver to log when a bottom element scrolls into view.
 *   2. Define a <hello-world> Web Component that renders your name.
 *   3. Animate a box across the screen with requestAnimationFrame.
 * ------------------------------------------------------------------------ */
```

47 · REAL-TIME NETWORKING & PRODUCTION OPS

```
/* ============================================================================
 * 47 · REAL-TIME NETWORKING & PRODUCTION OPS
 * ============================================================================
 * Run:  node lessons/47-realtime-and-production.js
 *
 * WHAT YOU'LL LEARN
 *   • Pushing data in real time: WebSockets vs Server-Sent Events vs polling
 *   • Running software in production: logging, monitoring, config, flags
 *   • Accessibility (a11y) & internationalization (i18n) basics
 *
 * "It works on my machine" isn't done. Production code must be observable,
 * configurable, resilient, and usable by everyone.
 * ========================================================================== */

// ── 1. Real-time: three ways to get fresh data ──────────────────────────────
// • POLLING ........ ask repeatedly on a timer. Simple, but wasteful/laggy.
//     setInterval(() => fetch('/api/messages'), 5000);
// • SSE (EventSource) ... server → client, ONE-WAY stream over plain HTTP.
//     const es = new EventSource('/stream');
//     es.onmessage = (e) => console.log('update:', e.data);   // great for feeds
// • WEBSOCKETS ..... full DUPLEX (both directions), low latency. Chat, games,
//     live cursors. A persistent connection, not request/response.
//     const ws = new WebSocket('wss://example.com');
//     ws.onopen    = () => ws.send('hello');
//     ws.onmessage = (e) => console.log('got:', e.data);
//   On the server (Node) you'd use the `ws` library or Socket.IO.
console.log('real-time: poll (simple) < SSE (one-way) < WebSocket (two-way)');

// Quick decision guide:
function chooseTransport(need) {
  if (need === 'two-way') return 'WebSocket';
  if (need === 'server-push') return 'SSE';
  return 'polling';
}
console.log(chooseTransport('two-way'));    // => WebSocket
console.log(chooseTransport('server-push')); // => SSE
```

```
// ── 2. Logging — structured, leveled ──────────────────────────────────────
// Don't ship console.log spam. Use levels and structured (JSON) logs so they're
// searchable in a log platform. (Libraries: pino, winston.)
const LEVELS = { error: 0, warn: 1, info: 2, debug: 3 };
function log(level, msg, meta = {}) {
  if (LEVELS[level] <= LEVELS.info) {           // current threshold = info
    console.log(JSON.stringify({ level, msg, ...meta }));
  }
}
log('info', 'user signed up', { userId: 42 });
// => {"level":"info","msg":"user signed up","userId":42}
log('debug', 'noisy detail'); // suppressed (below threshold)


// ── 3. Monitoring & error tracking ────────────────────────────────────────
// • Catch and REPORT errors from real users (Sentry, etc.) instead of losing them:
//     window.addEventListener('error', (e) => report(e));
//     window.addEventListener('unhandledrejection', (e) => report(e.reason));
//     process.on('uncaughtException', report); // Node
// • Track metrics (latency, error rate, throughput) and set alerts.
//   "If you can't see it, you can't fix it."


// ── 4. Configuration & secrets ────────────────────────────────────────────
// • Config comes from ENVIRONMENT VARIABLES, not hard-coded values (lesson 45).
// • Secrets (API keys, DB passwords) live in a secret manager / .env that is
//   GIT-IGNORED — never committed (lesson 39).
const config = {
  apiUrl: process.env.API_URL ?? 'http://localhost:3000',
  logLevel: process.env.LOG_LEVEL ?? 'info',
};
console.log('config:', config);


// ── 5. Feature flags & graceful failure ───────────────────────────────────
// • Feature flags toggle features without redeploying (gradual rollout, kill-switch).
const flags = { newCheckout: false };
console.log(flags.newCheckout ? 'new checkout' : 'old checkout'); // => old checkout
// • Resilience: retries with backoff (lesson 29), timeouts, circuit breakers,
//   and fallback UI so one failed call doesn't crash the whole app.


// ── 6. Accessibility (a11y) — usable by everyone ──────────────────────────
// • Semantic HTML first (<button>, <nav>, <label>) — it's accessible by default.
// • Provide alt text, keyboard navigation, visible focus, sufficient contrast.
// • Use ARIA only to fill gaps semantic HTML can't (aria-label, roles, live regions).
// • Test with a keyboard only and a screen reader. It's also often a legal requirement.
```

```
// ── 7. Internationalization (i18n) ──────────────────────────────────────
// • Don't concatenate translated strings; use a message catalog per locale.
// • Format with Intl (lessons 06/32) — numbers, currency, dates, plurals:
const price = new Intl.NumberFormat('de-DE', { style: 'currency', currency: 'EUR' });
console.log('localized price:', price.format(1234.5)); // => 1.234,50 €



/* THE PRODUCTION CHECKLIST ------------------------------------------------
 *   tested · linted · typed · secured · observable (logs+metrics+alerts) ·
 *   configurable (env) · resilient (retries/timeouts) · accessible · documented
 * ------------------------------------------------------------------------ */

/* PRACTICE ----------------------------------------------------------------
 *   1. Add a 'warn' log call and confirm it prints but 'debug' does not.
 *   2. Extend chooseTransport for a 'one-off request' → 'fetch'.
 *   3. Read LOG_LEVEL from env and make the log() threshold respect it.
 * ------------------------------------------------------------------------ */
```

48 · DATA STRUCTURES — the containers behind every program

```
/* ========================================================================
 * 48 · DATA STRUCTURES — the containers behind every program
 * ========================================================================
 * Run:  node lessons/48-data-structures.js
 *
 * WHAT YOU'LL LEARN
 *   The classic computer-science data structures, built in plain JS:
 *   • Stack (LIFO) and Queue (FIFO)
 *   • Linked list
 *   • Hash map (how Map/objects work under the hood)
 *   • Binary tree + binary search tree
 *   • Graph (adjacency list)
 *
 * WHY THIS MATTERS
 *   Arrays and objects (lessons 12–16) cover 90% of daily work. But knowing
 *   these structures — when each is fast, and WHY — is the difference between
 *   O(n) and O(1) code, and it's the backbone of every coding interview.
 *   Pair this with lesson 49 (algorithms) and lesson 41 (Big-O).
 * ======================================================================== */

// ── 1. STACK (LIFO — Last In, First Out) ────────────────────────────────
// Think a stack of plates: you add and remove from the TOP. Used for: undo,
// browser back, call stack, balancing brackets, depth-first traversal.
class Stack {
  #items = [];
  push(value) { this.#items.push(value); return this; } // add to top
  pop() { return this.#items.pop(); }                   // remove from top
  peek() { return this.#items.at(-1); }                 // look, don't remove
```

```
  get size() { return this.#items.length; }
  get isEmpty() { return this.#items.length === 0; }
}
const undo = new Stack();
undo.push('type A').push('type B').push('type C');
console.log('stack peek:', undo.peek()); // => type C
console.log('stack pop :', undo.pop());  // => type C  (last in, first out)
console.log('stack pop :', undo.pop());  // => type B

// Classic use: are the brackets balanced? "([])" ✅   "([)]" ❌
function balanced(str) {
  const pairs = { ')': '(', ']': '[', '}': '{' };
  const stack = [];
  for (const ch of str) {
    if (ch === '(' || ch === '[' || ch === '{') stack.push(ch);
    else if (ch in pairs) {
      if (stack.pop() !== pairs[ch]) return false; // mismatch
    }
  }
  return stack.length === 0; // nothing left unclosed
}
console.log('balanced ([]):', balanced('([])')); // => true
console.log('balanced ([):', balanced('([)]'));  // => false



// ── 2. QUEUE (FIFO — First In, First Out) ──────────────────────────────
// Think a line at a shop: first to arrive is first served. Used for: task
// queues, print spoolers, breadth-first traversal, rate limiting.
// NOTE: array.shift() is O(n) (it re-indexes). For big queues use two stacks or
// a ring buffer. We use shift() here for clarity.
class Queue {
  #items = [];
  enqueue(value) { this.#items.push(value); return this; } // add to back
  dequeue() { return this.#items.shift(); }                // remove from front
  peek() { return this.#items[0]; }
  get size() { return this.#items.length; }
}
const line = new Queue();
line.enqueue('Ana').enqueue('Bob').enqueue('Cy');
console.log('queue dequeue:', line.dequeue()); // => Ana  (first in, first out)
console.log('queue peek   :', line.peek());    // => Bob



// ── 3. LINKED LIST ─────────────────────────────────────────────────────
// A chain of nodes; each node holds a value + a pointer to the NEXT node.
// Strength: O(1) insert/remove at the front (no re-indexing like arrays).
// Weakness: O(n) to reach the middle (no random access). Great teaching tool.
class LinkedList {
  #head = null;
  #size = 0;
```

```
  prepend(value) {                      // add to front — O(1)
    this.#head = { value, next: this.#head };
    this.#size++;
    return this;
  }
  append(value) {                       // add to end — O(n)
    const node = { value, next: null };
    if (!this.#head) { this.#head = node; }
    else {
      let cur = this.#head;
      while (cur.next) cur = cur.next;   // walk to the last node
      cur.next = node;
    }
    this.#size++;
    return this;
  }
  toArray() {                           // walk the chain into an array
    const out = [];
    for (let cur = this.#head; cur; cur = cur.next) out.push(cur.value);
    return out;
  }
  get size() { return this.#size; }
}
const list = new LinkedList();
list.append('b').append('c').prepend('a');
console.log('linked list:', list.toArray()); // => [ 'a', 'b', 'c' ]


// ── 4. HASH MAP (how Map / objects work under the hood) ──────────────────
// A hash map turns a KEY into an array index via a hash function, giving ~O(1)
// lookup. Collisions (two keys → same bucket) are handled by "chaining": each
// bucket holds a list of [key, value] pairs. (In real code, just use `Map`.)
class HashMap {
  #buckets = Array.from({ length: 8 }, () => []);
  #hash(key) {                          // turn a string into a bucket index
    let h = 0;
    for (const ch of String(key)) h = (h * 31 + ch.charCodeAt(0)) % this.#buckets.length;
    return h;
  }
  set(key, value) {
    const bucket = this.#buckets[this.#hash(key)];
    const pair = bucket.find((p) => p[0] === key);
    if (pair) pair[1] = value;          // update existing
    else bucket.push([key, value]);     // or add new (collision → same bucket)
    return this;
  }
  get(key) {
    const bucket = this.#buckets[this.#hash(key)];
    return bucket.find((p) => p[0] === key)?.[1];
  }
```

```
}
const ages = new HashMap();
ages.set('Ana', 30).set('Bob', 25);
console.log('hashmap get Ana:', ages.get('Ana')); // => 30
console.log('hashmap get Zz :', ages.get('Zz'));  // => undefined



// ── 5. BINARY SEARCH TREE (BST) ──────────────────────────────────────────
// A tree where every node has up to two children, and for each node:
//   left subtree < node < right subtree.
// That ordering makes search/insert O(log n) on a balanced tree — like binary
// search (lesson 49) but in a structure you can keep adding to.
class BST {
  #root = null;
  insert(value) {
    this.#root = this.#insert(this.#root, value);
    return this;
  }
  #insert(node, value) {
    if (!node) return { value, left: null, right: null };
    if (value < node.value) node.left = this.#insert(node.left, value);
    else if (value > node.value) node.right = this.#insert(node.right, value);
    return node;                          // duplicates ignored
  }
  has(value) {
    let node = this.#root;
    while (node) {
      if (value === node.value) return true;
      node = value < node.value ? node.left : node.right; // go one way only
    }
    return false;
  }
  inOrder() {                             // in-order walk → SORTED values
    const out = [];
    (function walk(node) {
      if (!node) return;
      walk(node.left); out.push(node.value); walk(node.right);
    })(this.#root);
    return out;
  }
}
const tree = new BST();
[8, 3, 10, 1, 6, 14].forEach((n) => tree.insert(n));
console.log('BST sorted   :', tree.inOrder()); // => [ 1, 3, 6, 8, 10, 14 ]
console.log('BST has 6    :', tree.has(6));     // => true
console.log('BST has 7    :', tree.has(7));     // => false



// ── 6. GRAPH (adjacency list) ────────────────────────────────────────────
// A set of nodes ("vertices") connected by edges. The adjacency-list form maps
```

```
// each node → an array of its neighbours. Models networks: friends, roads,
// dependencies, the web. (Traversal — BFS/DFS — is in lesson 49.)
class Graph {
  #adj = new Map();
  addNode(node) {
    if (!this.#adj.has(node)) this.#adj.set(node, []);
    return this;
  }
  addEdge(a, b) {                       // undirected: link both ways
    this.addNode(a).addNode(b);
    this.#adj.get(a).push(b);
    this.#adj.get(b).push(a);
    return this;
  }
  neighbours(node) { return this.#adj.get(node) ?? []; }
  get nodes() { return [...this.#adj.keys()]; }
}
const social = new Graph();
social.addEdge('Ana', 'Bob').addEdge('Ana', 'Cy').addEdge('Bob', 'Dee');
console.log("graph Ana's friends:", social.neighbours('Ana')); // => [ 'Bob', 'Cy' ]


/* WHICH STRUCTURE, WHEN? ------------------------------------------------
 *   Stack ......... undo, parsing, depth-first work, the call stack
 *   Queue ......... scheduling, buffering, breadth-first work
 *   Linked list ... O(1) front insert/remove; teaching pointers
 *   Hash map ...... O(1) key→value lookup (you'll usually just use Map)
 *   BST ........... kept-sorted data with O(log n) search/insert (balanced)
 *   Graph ......... relationships/networks; pair with BFS/DFS (lesson 49)
 *
 * Big-O recap (lesson 41): array index = O(1), array search = O(n),
 *   Set/Map lookup = O(1), balanced BST search = O(log n).
 * ---------------------------------------------------------------------- */

/* PRACTICE -------------------------------------------------------------
 *   1. Add a `reverse()` method to LinkedList that flips the chain in place.
 *   2. Use your Stack to reverse the characters of a string.
 *   3. Add `min()`/`max()` to the BST (hint: go all the way left / right).
 *   4. Add a directed-edge option to Graph (link only a → b, not both ways).
 * ---------------------------------------------------------------------- */
```

49 · ALGORITHMS — the problem-solving toolkit

```
/* ========================================================================
 * 49 · ALGORITHMS — the problem-solving toolkit
 * ========================================================================
 * Run:  node lessons/49-algorithms.js
 *
```

```
 * WHAT YOU'LL LEARN
 *   The algorithm patterns that show up in real code AND every interview:
 *   • Searching: linear vs binary search
 *   • Sorting: bubble (to learn) vs merge & quick (to use)
 *   • Two-pointer & sliding-window techniques
 *   • Recursion & dynamic programming (memoization)
 *   • Graph traversal: BFS and DFS (uses lesson 48's structures)
 *
 * Read each one with its Big-O (lesson 41) in mind: the GOAL is rarely "make it
 * work" — it's "make it work without blowing up as the input grows."
 * ========================================================================== */

// ── 1. LINEAR SEARCH — O(n) ──────────────────────────────────────────────
// Check items one by one. Works on ANY array. Slow on large data.
function linearSearch(arr, target) {
  for (let i = 0; i < arr.length; i++) if (arr[i] === target) return i;
  return -1; // not found
}
console.log('linear:', linearSearch([5, 3, 8, 1], 8)); // => 2


// ── 2. BINARY SEARCH — O(log n) ──────────────────────────────────────────
// MUCH faster, but the array MUST be sorted. Halve the search range each step:
// look at the middle, then throw away the half that can't contain the target.
function binarySearch(sorted, target) {
  let lo = 0, hi = sorted.length - 1;
  while (lo <= hi) {
    const mid = Math.floor((lo + hi) / 2);
    if (sorted[mid] === target) return mid;
    if (sorted[mid] < target) lo = mid + 1; // target is in the right half
    else hi = mid - 1;                      // target is in the left half
  }
  return -1;
}
console.log('binary:', binarySearch([1, 3, 5, 8, 13, 21], 13)); // => 4
// 1M items: linear ≈ 1,000,000 checks; binary ≈ 20. That's the power of O(log n).


// ── 3. BUBBLE SORT — O(n²) — learn it, don't ship it ─────────────────────
// Repeatedly swap adjacent out-of-order pairs until sorted. Easy to understand,
// too slow for real data — but it teaches what sorting *is*.
function bubbleSort(arr) {
  const a = [...arr]; // copy — don't mutate the caller's array (lesson 12)
  for (let i = 0; i < a.length; i++) {
    for (let j = 0; j < a.length - 1 - i; j++) {
      if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]]; // swap
    }
  }
  return a;
}
```

```
console.log('bubble:', bubbleSort([5, 2, 9, 1, 5, 6])); // => [1, 2, 5, 5, 6, 9]


// ── 4. MERGE SORT — O(n log n) — a real "divide & conquer" sort ──────────────
// Split the array in half, sort each half (recursively), then MERGE the two
// sorted halves. Reliable O(n log n) — the kind of algorithm engines use.
function mergeSort(arr) {
  if (arr.length <= 1) return arr;                 // base case
  const mid = Math.floor(arr.length / 2);
  const left = mergeSort(arr.slice(0, mid));       // sort left half
  const right = mergeSort(arr.slice(mid));         // sort right half
  return merge(left, right);
}
function merge(left, right) {
  const out = [];
  let i = 0, j = 0;
  while (i < left.length && j < right.length) {    // take the smaller front item
    out.push(left[i] <= right[j] ? left[i++] : right[j++]);
  }
  return out.concat(left.slice(i), right.slice(j)); // drain whatever's left
}
console.log('merge :', mergeSort([5, 2, 9, 1, 5, 6])); // => [1, 2, 5, 5, 6, 9]
// (In practice: arr.toSorted() — lesson 37. These show HOW sorting works.)


// ── 5. QUICK SORT — O(n log n) average — pick a pivot, partition ─────────────
// Choose a "pivot", put smaller items left and larger right, recurse on each.
function quickSort(arr) {
  if (arr.length <= 1) return arr;
  const [pivot, ...rest] = arr;
  const smaller = rest.filter((n) => n < pivot);
  const larger = rest.filter((n) => n >= pivot);
  return [...quickSort(smaller), pivot, ...quickSort(larger)];
}
console.log('quick :', quickSort([5, 2, 9, 1, 5, 6])); // => [1, 2, 5, 5, 6, 9]


// ── 6. TWO-POINTER technique ─────────────────────────────────────────────────
// Walk two indices toward each other. Turns many O(n²) problems into O(n).
// Example: does a SORTED array contain two numbers that sum to a target?
function hasPairSum(sorted, target) {
  let lo = 0, hi = sorted.length - 1;
  while (lo < hi) {
    const sum = sorted[lo] + sorted[hi];
    if (sum === target) return [sorted[lo], sorted[hi]];
    if (sum < target) lo++;   // need a bigger sum → move left pointer up
    else hi--;                // need a smaller sum → move right pointer down
  }
  return null;
}
```

```
console.log('two-pointer:', hasPairSum([1, 3, 4, 7, 9], 11)); // => [4, 7]


// ── 7. SLIDING WINDOW ──────────────────────────────────────────────
// Keep a moving "window" over the data instead of recomputing from scratch.
// Example: max sum of any k consecutive numbers — O(n) instead of O(n*k).
function maxWindowSum(arr, k) {
  let windowSum = 0;
  for (let i = 0; i < k; i++) windowSum += arr[i]; // first window
  let best = windowSum;
  for (let i = k; i < arr.length; i++) {
    windowSum += arr[i] - arr[i - k];              // slide: add new, drop oldest
    best = Math.max(best, windowSum);
  }
  return best;
}
console.log('sliding window:', maxWindowSum([2, 1, 5, 1, 3, 2], 3)); // => 9 (5+1+3)


// ── 8. DYNAMIC PROGRAMMING (memoization) ──────────────────────────
// Naive recursive fibonacci is O(2ⁿ) — it recomputes the same values endlessly.
// DP = remember sub-results so each is computed ONCE. Drops it to O(n).
function fib(n, memo = new Map()) {
  if (n <= 1) return n;
  if (memo.has(n)) return memo.get(n);             // already solved → reuse
  const result = fib(n - 1, memo) + fib(n - 2, memo);
  memo.set(n, result);
  return result;
}
console.log('fib(40):', fib(40)); // => 102334155  (instant, thanks to memoization)


// ── 9. GRAPH TRAVERSAL — BFS & DFS ─────────────────────────────────
// Visit every node reachable from a start. BFS uses a QUEUE (lesson 48) and
// explores level by level (→ shortest path in unweighted graphs). DFS uses a
// STACK / recursion and dives deep first.
const graph = {
  A: ['B', 'C'],
  B: ['A', 'D', 'E'],
  C: ['A', 'F'],
  D: ['B'], E: ['B', 'F'], F: ['C', 'E'],
};

function bfs(graph, start) {
  const visited = new Set([start]);
  const queue = [start];          // FIFO
  const order = [];
  while (queue.length) {
    const node = queue.shift();   // take from the FRONT
    order.push(node);
```

```
    for (const next of graph[node]) {
      if (!visited.has(next)) { visited.add(next); queue.push(next); }
    }
  }
  return order;
}

function dfs(graph, start, visited = new Set(), order = []) {
  visited.add(start);
  order.push(start);
  for (const next of graph[start]) {
    if (!visited.has(next)) dfs(graph, next, visited, order); // dive deep first
  }
  return order;
}
console.log('BFS from A:', bfs(graph, 'A')); // => [ 'A', 'B', 'C', 'D', 'E', 'F' ]
console.log('DFS from A:', dfs(graph, 'A')); // => [ 'A', 'B', 'D', 'E', 'F', 'C' ]



/* COMPLEXITY CHEAT SHEET ----------------------------------------------------
 *   linear search ... O(n)        binary search ... O(log n)  (needs sorted)
 *   bubble sort ..... O(n²)       merge/quick ..... O(n log n)
 *   two-pointer ..... O(n)        sliding window .. O(n)
 *   naive fib ....... O(2ⁿ)       memoized fib .... O(n)
 *   BFS / DFS ....... O(V + E)    (vertices + edges)
 *
 * INTERVIEW PLAYBOOK:
 *   • Sorted input?  → think binary search / two-pointer.
 *   • "Subarray/substring of length k"? → sliding window.
 *   • Overlapping sub-problems? → memoize (DP).
 *   • Graph/tree/grid? → BFS (shortest path) or DFS (explore all).
 *   • Always state the Big-O of your solution out loud.
 * ------------------------------------------------------------------------- */

/* PRACTICE ------------------------------------------------------------------
 *   1. Write isPalindrome(str) with the two-pointer technique.
 *   2. Implement insertionSort and compare its output to mergeSort.
 *   3. Use BFS to find the SHORTEST path (list of nodes) between two graph nodes.
 *   4. Solve "climbing stairs" (1 or 2 steps at a time) with memoized recursion.
 * ------------------------------------------------------------------------- */
```

50 · BINARY DATA — bytes, buffers, and files

```
/* ============================================================================
 * 50 · BINARY DATA — bytes, buffers, and files
 * ============================================================================
 * Run:  node lessons/50-binary-data.js
 *       (the Blob/File/FormData parts are BROWSER APIs — shown as comments;
```

```
 *        the ArrayBuffer/TypedArray parts run in Node too.)
 *
 * WHAT YOU'LL LEARN
 *   How JavaScript handles RAW BYTES — not just strings and numbers:
 *   • ArrayBuffer — a fixed block of memory
 *   • Typed arrays (Uint8Array, Float64Array, …) — views over that memory
 *   • DataView — read/write mixed types at exact byte offsets
 *   • Blob / File — chunks of binary data in the browser
 *   • FormData — sending files and fields to a server
 *   • TextEncoder / TextDecoder — text ↔ bytes
 *   • SharedArrayBuffer + Atomics — memory shared across threads
 *
 * WHY THIS MATTERS
 *   File uploads/downloads, images, audio, WebSockets (lesson 47), streaming
 *   fetch responses, WebGL, and WebAssembly all move BYTES. Strings can't
 *   represent that efficiently — typed arrays can.
 * ========================================================================== */

// ── 1. ArrayBuffer — a raw block of memory ──────────────────────────────────
// An ArrayBuffer is just N bytes. You can't read it directly — you need a VIEW.
const buffer = new ArrayBuffer(8); // 8 bytes, all zero
console.log('buffer byteLength:', buffer.byteLength); // => 8



// ── 2. Typed arrays — views over a buffer ───────────────────────────────────
// A typed array reads the buffer's bytes as numbers of a fixed size/type.
//   Uint8Array   → 1 byte,  0..255          (the workhorse for raw bytes)
//   Int32Array   → 4 bytes, signed integers
//   Float64Array → 8 bytes, decimals (same precision as a normal JS number)
const bytes = new Uint8Array(buffer); // view the 8-byte buffer as 8 × uint8
bytes[0] = 255;
bytes[1] = 256;  // wraps! 256 doesn't fit in a byte → becomes 0
bytes[2] = 42;
console.log('uint8 view:', bytes); // => Uint8Array(8) [ 255, 0, 42, 0, 0, 0, 0, 0 ]

// Multiple views can share ONE buffer — they read the same bytes differently:
const ints = new Int32Array(buffer); // 8 bytes → two 32-bit ints
console.log('same bytes as int32:', ints); // values depend on byte order

// Typed arrays have most array methods (map/filter/reduce, lesson 13) but a
// FIXED length and a single number type — that's what makes them fast & compact.
const doubled = Uint8Array.from([1, 2, 3], (n) => n * 2);
console.log('typed map:', doubled); // => Uint8Array(3) [ 2, 4, 6 ]



// ── 3. DataView — precise, mixed-type access ────────────────────────────────
// When a binary format packs different types at known offsets (file headers,
// network protocols), DataView reads/writes them exactly — and lets you control
// "endianness" (byte order).
const dv = new DataView(new ArrayBuffer(8));
```

```
dv.setInt16(0, 300);          // write a 16-bit int at byte 0
dv.setFloat32(2, 3.14, true); // write a 32-bit float at byte 2 (true = little-endian)
console.log('DataView int16 :', dv.getInt16(0));        // => 300
console.log('DataView f32   :', dv.getFloat32(2, true).toFixed(2)); // => 3.14



// ── 4. Text ↔ bytes (TextEncoder / TextDecoder) ──────────────────────────
// Strings are UTF-16 in memory; on the wire/disk text is usually UTF-8 bytes.
const encoder = new TextEncoder();
const encoded = encoder.encode('Hi 👋'); // string → Uint8Array of UTF-8 bytes
console.log('encoded bytes:', encoded);   // emoji takes 4 bytes

const decoder = new TextDecoder();
console.log('decoded back:', decoder.decode(encoded)); // => Hi 👋



// ── 5. Blob & File (BROWSER) — chunks of binary data ─────────────────────
/* A Blob is an immutable bag of bytes with a MIME type. A File is a Blob with a
 * name (what you get from <input type="file">). Both are how the browser holds
 * images, downloads, and uploads.
 *
 *   // Make a Blob and trigger a download:
 *   const blob = new Blob(['name,score\nAna,10'], { type: 'text/csv' });
 *   const url = URL.createObjectURL(blob);
 *   const a = Object.assign(document.createElement('a'),
 *                           { href: url, download: 'data.csv' });
 *   a.click();
 *   URL.revokeObjectURL(url);   // free the memory when done
 *
 *   // Read a file the user picked:
 *   input.addEventListener('change', async () => {
 *     const file = input.files[0];          // a File (Blob + name + size)
 *     const text  = await file.text();       // read as string
 *     const bytes = await file.arrayBuffer(); // read as raw bytes
 *     console.log(file.name, file.size, text);
 *   });
 */



// ── 6. FormData (BROWSER) — send files + fields to a server ──────────────
/* FormData builds a multipart request body — the standard way to upload files
 * alongside text fields. fetch (lesson 25) sets the right headers automatically.
 *
 *   const form = new FormData();
 *   form.append('username', 'ana');
 *   form.append('avatar', fileInput.files[0]); // a File/Blob
 *   await fetch('/upload', { method: 'POST', body: form });
 *   //  ^ do NOT set Content-Type yourself — the browser adds the boundary.
 *
 *   // FormData can also read an entire <form> at once:
```

```
 *   const form2 = new FormData(document.querySelector('form'));
 *   for (const [key, value] of form2) console.log(key, value);
 */



// ── 7. Streaming binary (concept) ──────────────────────────────────────
/* Large responses don't have to load all at once. fetch bodies are streams of
 * Uint8Array chunks — read them progressively (great for progress bars):
 *
 *   const res = await fetch('/big-file');
 *   const reader = res.body.getReader();      // a stream of byte chunks
 *   let received = 0;
 *   while (true) {
 *     const { done, value } = await reader.read(); // value = Uint8Array chunk
 *     if (done) break;
 *     received += value.length;
 *     console.log('received', received, 'bytes');
 *   }
 *  (In Node, see streams in lesson 45 — same idea, different API.)
 */



// ── 8. SharedArrayBuffer + Atomics — memory shared between threads ─────────
// A normal ArrayBuffer is copied (transferred) when sent to a Web Worker
// (lesson 46) / Node worker_thread (lesson 45). A SharedArrayBuffer is the SAME
// memory seen by BOTH threads at once — true shared state, no copying.
// (Browsers require special COOP/COEP headers to enable it, for security.)
const shared = new SharedArrayBuffer(8); // 8 bytes, shareable across threads
const sharedView = new Int32Array(shared);

// When two threads touch the same memory you get RACE CONDITIONS. `Atomics`
// provides operations that are guaranteed to complete without interruption:
Atomics.store(sharedView, 0, 100);          // safely write slot 0
Atomics.add(sharedView, 0, 5);              // atomic read-modify-write → 105
console.log('Atomics value:', Atomics.load(sharedView, 0)); // => 105

// Atomics.wait / Atomics.notify let one thread SLEEP until another signals it —
// the building block of locks and cross-thread coordination:
//   // worker:  Atomics.wait(sharedView, 0, 105);   // block while slot 0 === 105
//   // main:    Atomics.store(sharedView, 0, 0); Atomics.notify(sharedView, 0);
// ⚠ This is advanced/rare. You only need it for high-performance parallel work
// (image/audio processing, simulations, WASM threads). For normal worker
// communication, plain postMessage (lesson 46) is simpler and safer.



/* MENTAL MODEL ------------------------------------------------------------
 *   ArrayBuffer = the memory (bytes).   Typed array / DataView = how you READ it.
 *   SharedArrayBuffer = memory shared across threads; Atomics = safe access to it.
 *   Blob/File   = bytes + a type/name (browser).   FormData = a way to SEND them.
```

```
 *   TextEncoder/Decoder = the bridge between text and UTF-8 bytes.
 *
 *   Reach for these only when you actually handle binary: uploads, media,
 *   protocols, WebGL/WASM. For everyday data, JSON + objects (lessons 14/25)
 *   are simpler and correct.
 * ------------------------------------------------------------------------ */

/* PRACTICE ---------------------------------------------------------------
 *   1. Make a Uint8Array of [72,105] and TextDecoder.decode it (what word?).
 *   2. Use a DataView to store two Float32 values in one 8-byte buffer, read back.
 *   3. (Browser) Build a CSV string, wrap it in a Blob, and download it.
 *   4. (Browser) Add a file <input>, read the chosen file with file.text().
 * ------------------------------------------------------------------------ */
```

51 · BIGINT & LANGUAGE CORNERS

```
/* ========================================================================
 * 51 · BIGINT & LANGUAGE CORNERS
 * ========================================================================
 * Run:  node lessons/51-bigint-and-language-corners.js
 *
 * WHAT YOU'LL LEARN
 *   The last few language features that round out "I know ALL of JavaScript":
 *   • BigInt — integers bigger than Number can safely hold
 *   • Object.fromEntries — build an object from key/value pairs
 *   • Array.prototype.flatMap — map + flatten in one step
 *   • Labeled statements — break/continue an OUTER loop
 *   • eval & the Function constructor — running code from a string (and why
 *     you almost never should)
 *
 * These are niche — you won't reach for them daily — but knowing they exist
 * (and their gotchas) is the difference between "most of JS" and "all of JS".
 * ======================================================================== */

// ── 1. BigInt — arbitrary-precision integers ──────────────────────────────
// Regular numbers are 64-bit floats. They can only represent integers EXACTLY
// up to Number.MAX_SAFE_INTEGER. Past that, math silently goes wrong:
console.log(Number.MAX_SAFE_INTEGER);       // => 9007199254775807 (2^53 - 1)
console.log(9007199254740991 + 1);          // => 9007199254740992 ✅
console.log(9007199254740991 + 2);          // => 9007199254740992 ❌ (should be ...93!)

// BigInt fixes this — write `n` after an integer literal, or call BigInt():
const big = 9007199254740991n;
console.log(big + 2n);                       // => 9007199254740993n ✅ exact
console.log(BigInt(42), typeof 42n);         // => 42n 'bigint'

// Arithmetic works as usual — but division TRUNCATES (BigInt has no decimals):
console.log(10n / 3n);                       // => 3n  (not 3.333…)
```

```
console.log(2n ** 64n);                     // => 18446744073709551616n (huge, exact)

// ⚠ You CANNOT mix BigInt and Number in math — it throws on purpose:
try { 1n + 1; } catch (e) { console.log('mix error:', e.message); } // TypeError
console.log(1n + BigInt(1));                // => 2n   (convert first)

// Comparisons DO work across types (no error):
console.log(2n > 1, 2n === 2, 2n == 2);     // => true false true
//                       ^ === is false: different TYPES (bigint vs number)

// Gotchas: no Math.* (Math.sqrt(4n) throws); JSON.stringify can't serialize it:
try { JSON.stringify({ id: 5n }); } catch (e) { console.log('json error:', e.message); }
// Fix: convert to string in a replacer → JSON.stringify(obj, (k,v) =>
//      typeof v === 'bigint' ? v.toString() : v)
// USE CASES: database BIGINT ids, Twitter/Discord "snowflake" ids, nanosecond
// timestamps, cryptography — anywhere integers exceed 2^53.



// ── 2. Object.fromEntries — [[k,v]] → { k: v } ──────────────────────────────
// The inverse of Object.entries (lesson 14). Turns pairs back into an object.
const pairs = [['name', 'Ana'], ['age', 30]];
console.log(Object.fromEntries(pairs));     // => { name: 'Ana', age: 30 }

// Killer combo: transform an object by going object → entries → map → object.
const prices = { apple: 1, pear: 2 };
const doubled = Object.fromEntries(
  Object.entries(prices).map(([k, v]) => [k, v * 2])
);
console.log(doubled);                       // => { apple: 2, pear: 4 }

// Also converts a Map (lesson 16) or URLSearchParams (lesson 25) into an object:
console.log(Object.fromEntries(new Map([['a', 1]]))); // => { a: 1 }



// ── 3. Array.prototype.flatMap — map, then flatten one level ────────────────
// Equivalent to arr.map(fn).flat(1), but in a single pass. Great when each item
// maps to ZERO, ONE, or MANY results.
console.log([1, 2, 3].flatMap((n) => [n, n * 10])); // => [1, 10, 2, 20, 3, 30]

// Return [] to DROP an item, [x] to keep it → map + filter in one step:
console.log([1, -2, 3, -4].flatMap((n) => (n > 0 ? [n] : []))); // => [1, 3]

// Split sentences into words across an array:
console.log(['a b', 'c d'].flatMap((s) => s.split(' '))); // => ['a','b','c','d']
// (Note: flatMap only flattens ONE level; use .flat(Infinity) for deep nesting.)



// ── 4. Labeled statements — break/continue an OUTER loop ────────────────────
// By default break/continue affect the NEAREST loop. A label lets you target an
```

```
// outer one — handy for breaking out of nested loops without a flag variable.
outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i + j === 3) {
      console.log(`breaking outer at i=${i}, j=${j}`);
      break outer;                // jumps ALL the way out, not just the inner loop
    }
  }
}
// `continue label` similarly skips to the next iteration of the labeled loop.
// Use sparingly — often a helper function with an early `return` is clearer.



// ── 5. eval & the Function constructor — code from strings (avoid!) ──────────
// eval() runs a string as code IN THE CURRENT SCOPE. It works…
console.log(eval('2 + 3'));                  // => 5
// …but it's a security hole (runs ANY code, incl. attacker input), it's slow
// (defeats engine optimizations), and it can read/modify your local variables.
// RULE: never eval untrusted input. You almost never need eval at all —
//   JSON.parse for data, optional chaining/bracket access for dynamic props.

// The Function constructor is slightly less dangerous: it builds a function from
// a string but runs in the GLOBAL scope (it can't see your locals):
const addFn = new Function('a', 'b', 'return a + b');
console.log(addFn(4, 5));                    // => 9
// Same security/perf caveats apply. Legit uses are rare (templating engines,
// sandboxes, REPLs) — for everyday code, treat both as "do not use".



/* QUICK REFERENCE ---------------------------------------------------------
 *   BigInt ............ 123n / BigInt(x) — exact integers past 2^53; no mixing
 *                       with Number, no decimals, no Math, no direct JSON.
 *   Object.fromEntries  pairs → object; pair with Object.entries to transform.
 *   flatMap ........... map + flat(1); return [] to drop, [x] to keep.
 *   label: loop ....... break/continue an outer loop by name.
 *   eval/Function ..... run strings as code — security + perf risk. Avoid.
 * ------------------------------------------------------------------------- */

/* PRACTICE ----------------------------------------------------------------
 *   1. Compute 100! (factorial) with BigInt — impossible to do exactly with Number.
 *   2. Use Object.entries + map + Object.fromEntries to UPPERCASE every value
 *      of { a: 'x', b: 'y' }.
 *   3. Use flatMap to turn [{tags:['js','ts']},{tags:['css']}] into one tag list.
 *   4. Use a labeled loop to find the first pair (i,j) in two arrays that sums to 7.
 * ------------------------------------------------------------------------- */
```

52 · INTERNATIONALIZATION (the full Intl API)

```
/* ============================================================================
 * 52 · INTERNATIONALIZATION (the full Intl API)
 * ============================================================================
 * Run:  node lessons/52-internationalization.js
 *
 * WHAT YOU'LL LEARN
 *   The built-in `Intl` API formats data the way each LANGUAGE/REGION expects —
 *   no libraries needed. You met NumberFormat (lesson 06) & DateTimeFormat
 *   (lesson 32); here are the rest:
 *   • Intl.Collator — locale-aware sorting & comparison
 *   • Intl.PluralRules — pick the right plural form
 *   • Intl.RelativeTimeFormat — "3 days ago", "in 2 hours"
 *   • Intl.ListFormat — "A, B, and C"
 *   • Intl.Segmenter — split text into words/sentences (any language)
 *
 * BIG IDEA: never hand-build dates, numbers, plurals, or "x ago" strings — the
 * platform already knows the rules for ~hundreds of locales. (See lesson 47 for
 * i18n in the bigger picture: translations, RTL, etc.)
 * ============================================================================ */

// ── Recap: the two you already know ──────────────────────────────────────────
console.log(new Intl.NumberFormat('en-IN', { style: 'currency', currency:
'INR' }).format(250000));
// => ₹2,50,000.00   (note the Indian digit grouping — Intl handles it)
console.log(new Intl.DateTimeFormat('en-GB', { dateStyle: 'full' }).format(new Date('2026-06-
05')));
// => Friday, 5 June 2026



// ── 1. Intl.Collator — sort/compare the way a language does ──────────────────
// String sort with < or default .sort() is by code-point, which is WRONG for
// accents and non-English alphabets. Collator sorts correctly per locale.
const words = ['zebra', 'apple', 'Ärger', 'apple pie', 'Apfel'];
const collator = new Intl.Collator('de', { sensitivity: 'base' }); // German rules
console.log('collator sort:', [...words].sort(collator.compare));
// 'ä' sorts near 'a' (correct in German) — a naive sort would misplace it.

// `numeric: true` makes "file2" come before "file10" (human/natural order):
const files = ['file10', 'file2', 'file1'];
console.log('natural sort:', [...files].sort(new Intl.Collator(undefined, { numeric:
true }).compare));
// => [ 'file1', 'file2', 'file10' ]   (not file1, file10, file2)



// ── 2. Intl.PluralRules — choose the correct plural CATEGORY ─────────────────
// English has 2 forms (1 item / N items). Other languages have up to 6. Don't
// hard-code `n === 1 ? 'item' : 'items'` — ask PluralRules which form to use.
```

```
const enPlural = new Intl.PluralRules('en-US');
function items(n) {
  const form = enPlural.select(n);            // 'one' | 'other' (for English)
  const label = { one: 'item', other: 'items' }[form];
  return `${n} ${label}`;
}
console.log(items(1), '/', items(5));         // => 1 item / 5 items
// Ordinals too (1st, 2nd, 3rd, 4th):
const ordinal = new Intl.PluralRules('en-US', { type: 'ordinal' });
const suffix = { one: 'st', two: 'nd', few: 'rd', other: 'th' };
console.log([1, 2, 3, 4, 11, 21].map((n) => `${n}${suffix[ordinal.select(n)]}`).join(' '));
// => 1st 2nd 3rd 4th 11th 21st


// ── 3. Intl.RelativeTimeFormat — "3 days ago" / "in 2 hours" ──────────────
// Stop writing your own "time ago" helper — this one is locale-correct.
const rtf = new Intl.RelativeTimeFormat('en', { numeric: 'auto' });
console.log(rtf.format(-1, 'day'));           // => yesterday   ('auto' → words)
console.log(rtf.format(3, 'day'));            // => in 3 days
console.log(rtf.format(-2, 'hour'));          // => 2 hours ago

// A tiny helper: turn a past Date into a friendly relative string.
function timeAgo(date) {
  const seconds = Math.round((date - new Date('2026-06-05T12:00:00')) / 1000);
  const units = [['day', 86400], ['hour', 3600], ['minute', 60], ['second', 1]];
  for (const [unit, secs] of units) {
    if (Math.abs(seconds) >= secs || unit === 'second') {
      return rtf.format(Math.round(seconds / secs), unit);
    }
  }
}
console.log(timeAgo(new Date('2026-06-04T12:00:00'))); // => yesterday


// ── 4. Intl.ListFormat — join a list the way humans write it ──────────────
// "A, B, and C" (note the Oxford comma + "and") — and it localizes.
const fruits = ['apples', 'oranges', 'pears'];
console.log(new Intl.ListFormat('en', { type: 'conjunction' }).format(fruits));
// => apples, oranges, and pears
console.log(new Intl.ListFormat('en', { type: 'disjunction' }).format(fruits));
// => apples, oranges, or pears


// ── 5. Intl.Segmenter — split text into words/sentences/graphemes ─────────
// "Just split on spaces" breaks for many languages (Chinese/Japanese have no
// spaces) and for emoji. Segmenter splits text correctly anywhere.
const seg = new Intl.Segmenter('en', { granularity: 'word' });
const segments = [...seg.segment('Hello, world! 👋')];
const realWords = segments.filter((s) => s.isWordLike).map((s) => s.segment);
console.log('words:', realWords);             // => [ 'Hello', 'world' ]
```

```
// Grapheme granularity counts USER-PERCEIVED characters (emoji = 1, not many):
const graphemes = [...new Intl.Segmenter().segment('a👨‍👩‍👧b')];
console.log('grapheme count:', graphemes.length); // => 3  ('.length' would say 8+)



/* WHEN TO REACH FOR EACH ---------------------------------------------------
 *   NumberFormat ...... currency, percent, units, digit grouping (lesson 06)
 *   DateTimeFormat .... locale dates & times (lesson 32)
 *   Collator .......... sorting/searching strings correctly (accents, numeric)
 *   PluralRules ....... "1 item" vs "5 items", ordinals (1st/2nd/3rd)
 *   RelativeTimeFormat  "3 days ago", "in 2 hours"
 *   ListFormat ........ "A, B, and C"
 *   Segmenter ......... counting/splitting words & characters in ANY language
 *
 * All of Intl is built into Node and every modern browser — zero dependencies.
 * ------------------------------------------------------------------------- */

/* PRACTICE -----------------------------------------------------------------
 *   1. Sort ['café','car','cab'] with an Intl.Collator and compare to a plain sort.
 *   2. Write likes(n) → "1 like" / "N likes" using Intl.PluralRules.
 *   3. Format "in 5 minutes" and "2 weeks ago" with Intl.RelativeTimeFormat.
 *   4. Join ['HTML','CSS','JS'] as "HTML, CSS, and JS" with Intl.ListFormat.
 *   5. Use Intl.Segmenter to correctly count the characters in '👍🎉'.
 * ------------------------------------------------------------------------- */
```